

UNSUPERVISED MACHINE LEARNING PARADIGMS FOR THE REPRESENTATION OF MUSIC SIMILARITY AND STRUCTURE

Axel Marmoret, IRISA, PhD defense,

*Under the supervision of
Frédéric Bimbot and Jérémy E. Cohen*



December 2nd, 2022

PhD, in a nutshell

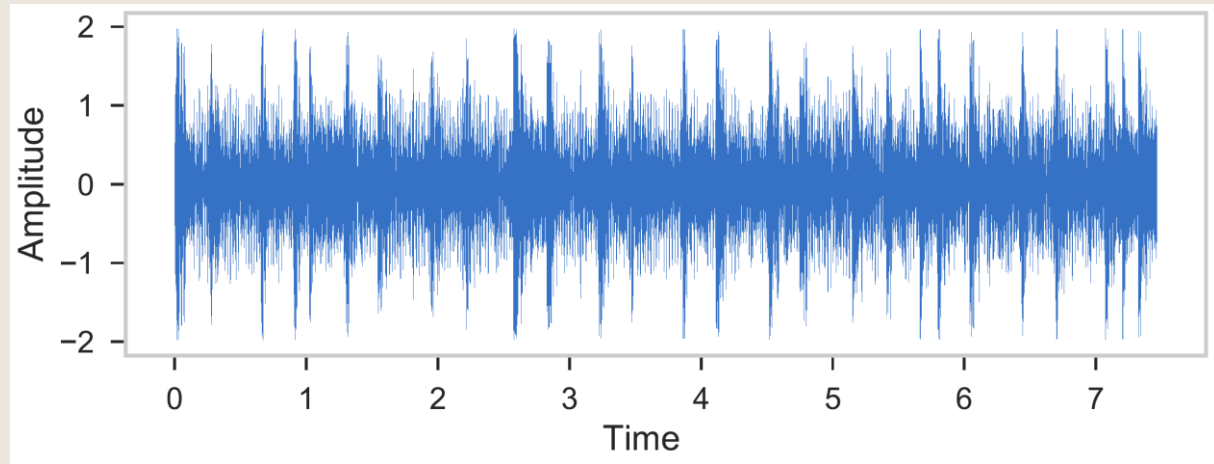
Frédéric Bimbot



Jérémy E. Cohen



Music audio signal

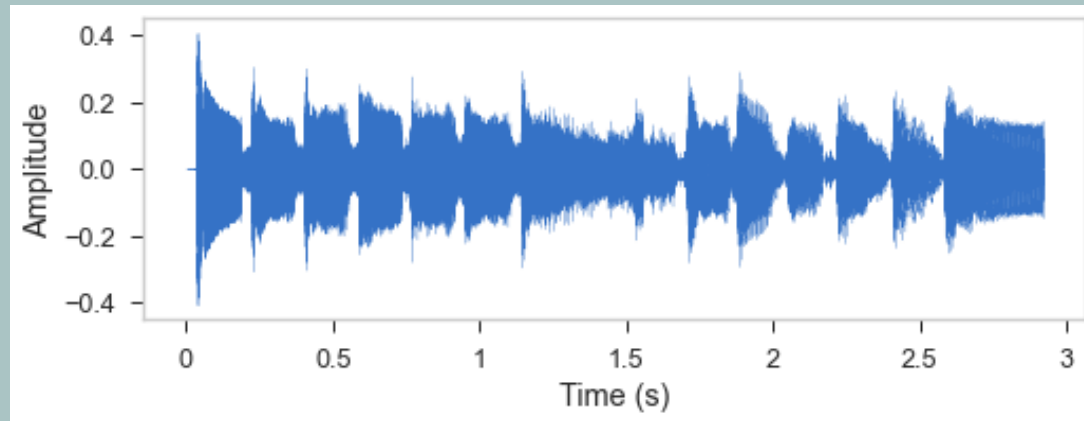


Music structure

Verse	Chorus	Verse	Solo	Chorus
-------	--------	-------	------	--------

Music

Audio signal



Symbolic representation

- Human-intended representations
 - *Score, tablature, sheet music...*



- Digital Format
 - *MIDI, MusicXML, ...*

Music structure?

- Segmenting a song in structural elements

Song organisation
(ground-truth)

Verse	Chorus	Verse	Solo	Chorus
-------	--------	-------	------	--------

■ Segmenting a song in structural elements

Verse	Chorus	Verse	Solo	Chorus
-------	--------	-------	------	--------

A	B	A	C	B'
---	---	---	---	----

a	b	c	c	a	b	d	e	f	c	c'
---	---	---	---	---	---	---	---	---	---	----

1	2	3	3	4	5	4	5	1	2	3	3	6	7	8	8	4	5	4	9
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

[illegible]

Music structure?

■ Segmenting a song in structural elements

Song organisation (ground-truth)	Verse	Chorus	Verse	Solo	Chorus
-------------------------------------	-------	--------	-------	------	--------

[illegible]

Music structure?

■ Segmenting a song in structural elements

Song organisation
(ground-truth)



Boundary retrieval

High-scale
(sectional level)



Low-scale



Why structural segmentation?

For structure analysis

- Music analysis/Musicology
- Automatic summary
- ...

For indirect applications

- Genre/music recognition
- Recommendation
- Cover detection
- ...

As a proxy task

Quantitatively evaluates the relevance of a music analysis model

How are sections estimated?

- Two principal criteria [Nie+20]:

Homogeneity/Novelty



- *Locally stationary*

Repetition

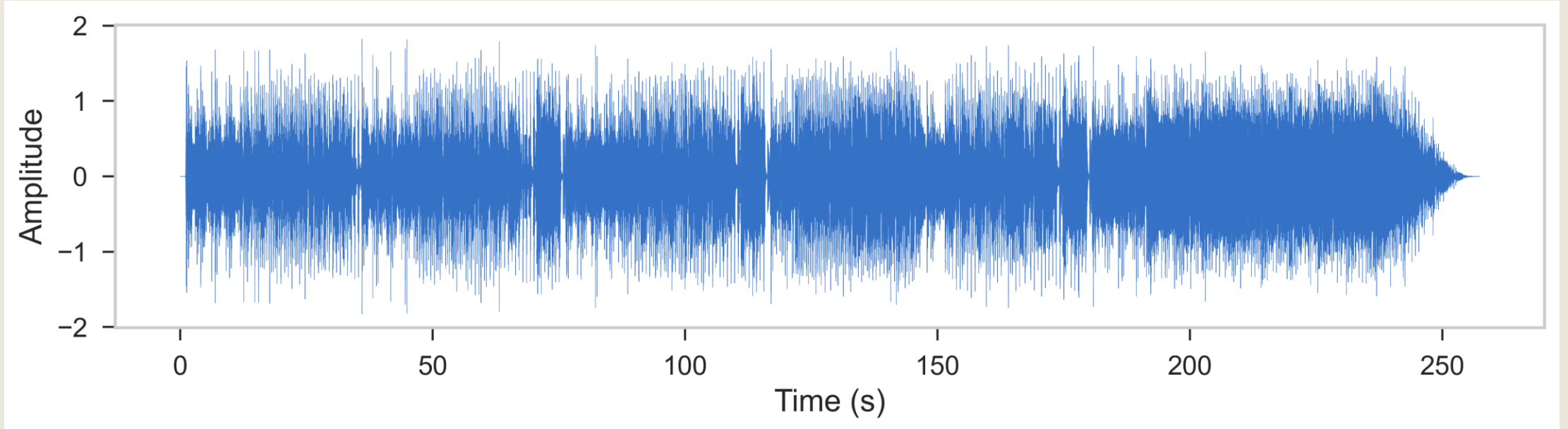


- *Repeated accross the song*

➔ Notion of **similarity** and **dissimilarity** accross the song

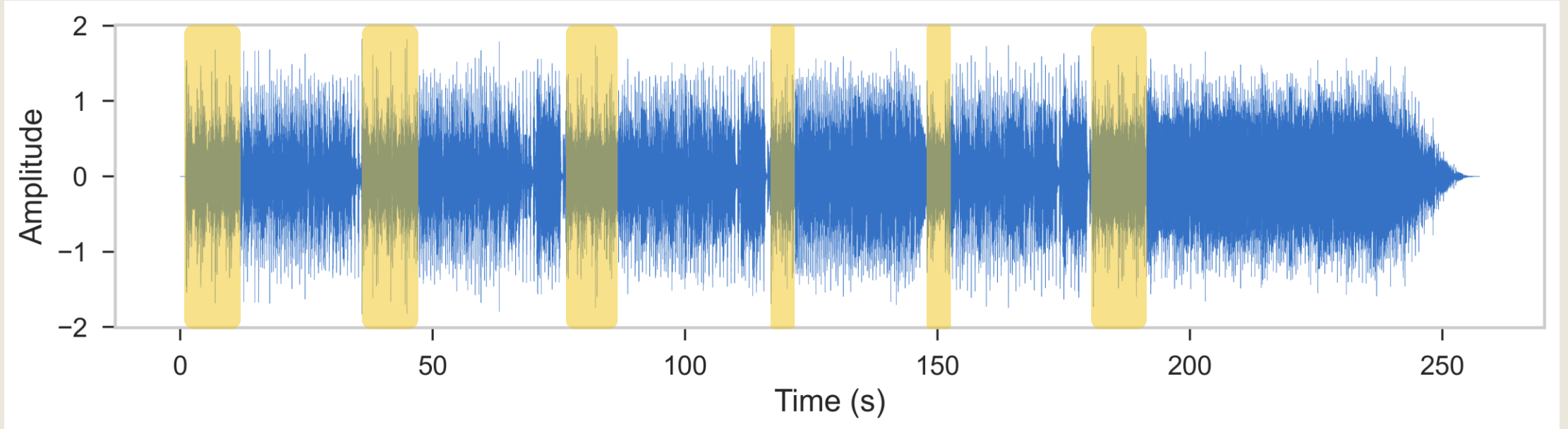
Practical example

"Come Together" – *The Beatles*



Practical example

"Come Together" – *The Beatles*

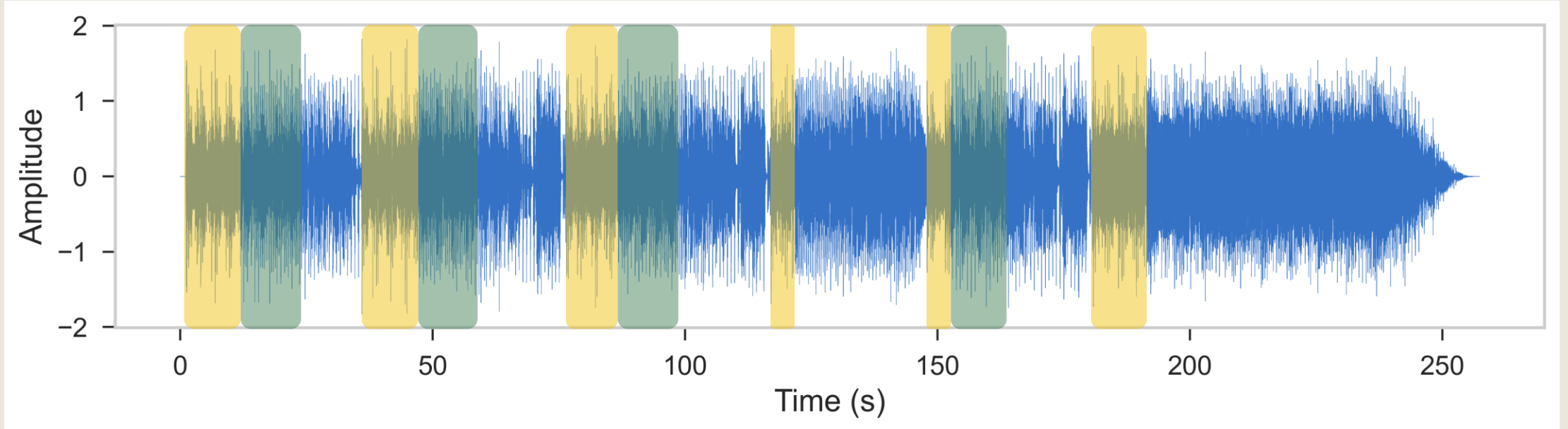


Intro



Practical example

"Come Together" – *The Beatles*



Intro

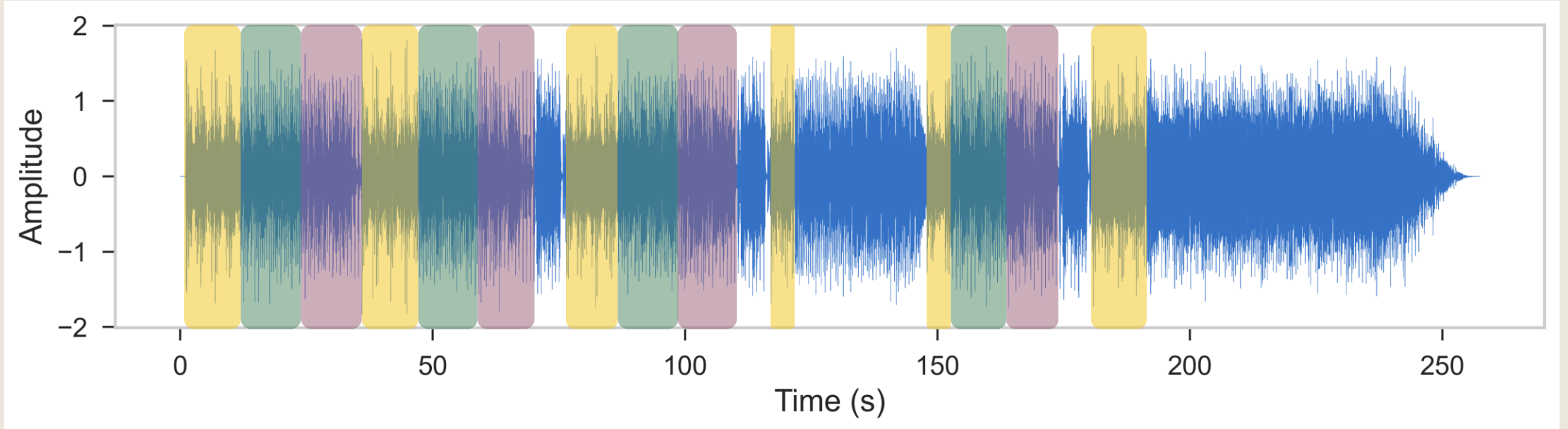


Verse
1



Practical example

"Come Together" – *The Beatles*



Intro



Verse
2

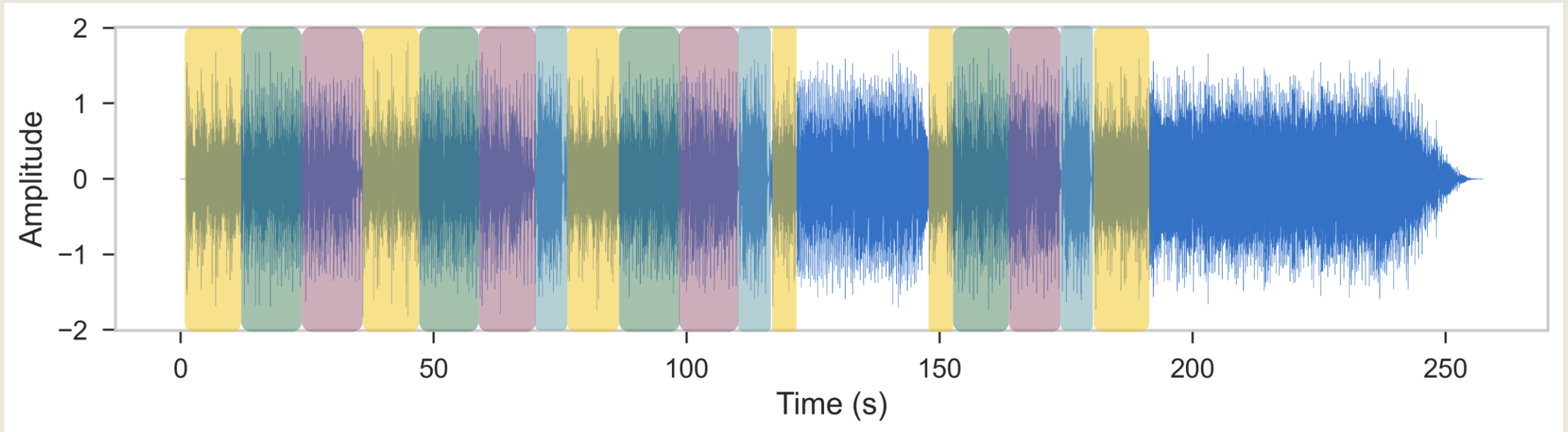


Verse
1



Practical example

"Come Together" – *The Beatles*



Intro



Verse
2



Verse
1

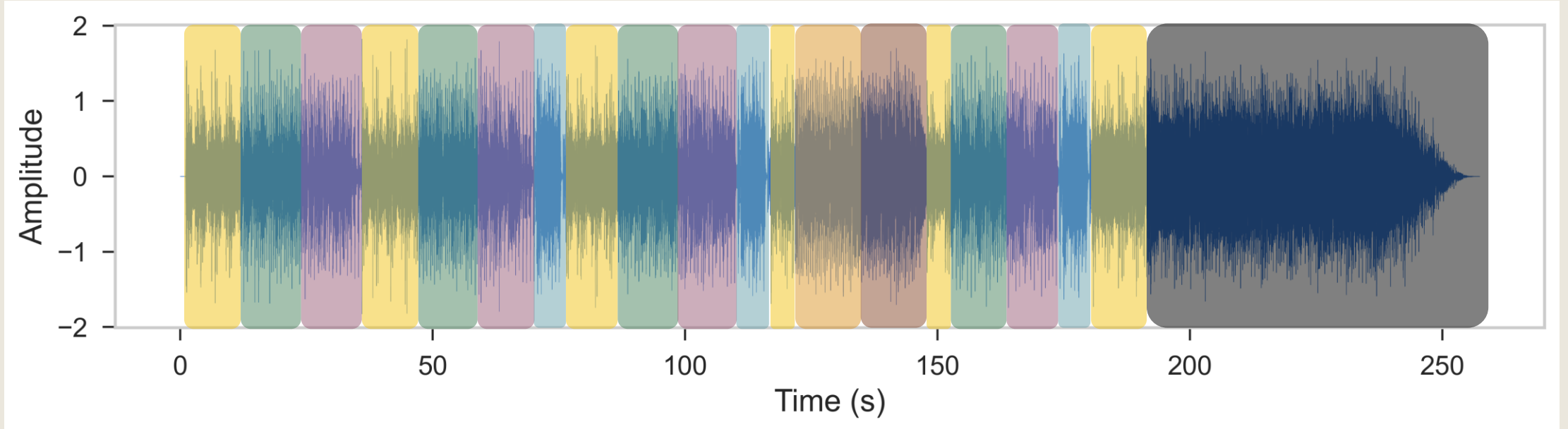


Chorus



Practical example

"Come Together" – *The Beatles*



Intro



Verse
2



Solo
1



Outro



Verse
1



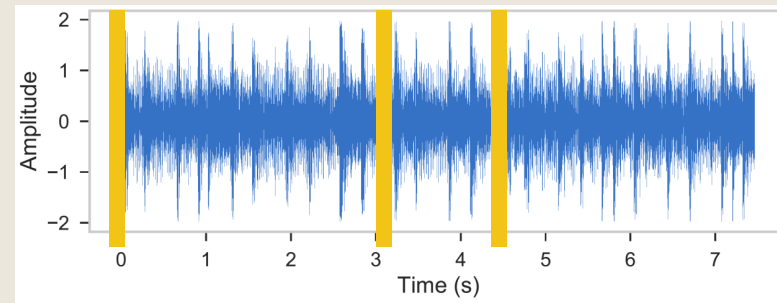
Chorus



Solo
2



Machine Learning?



Segmentation
algorithm

List of boundaries
(e.g. [0, 3.1, 4.5])

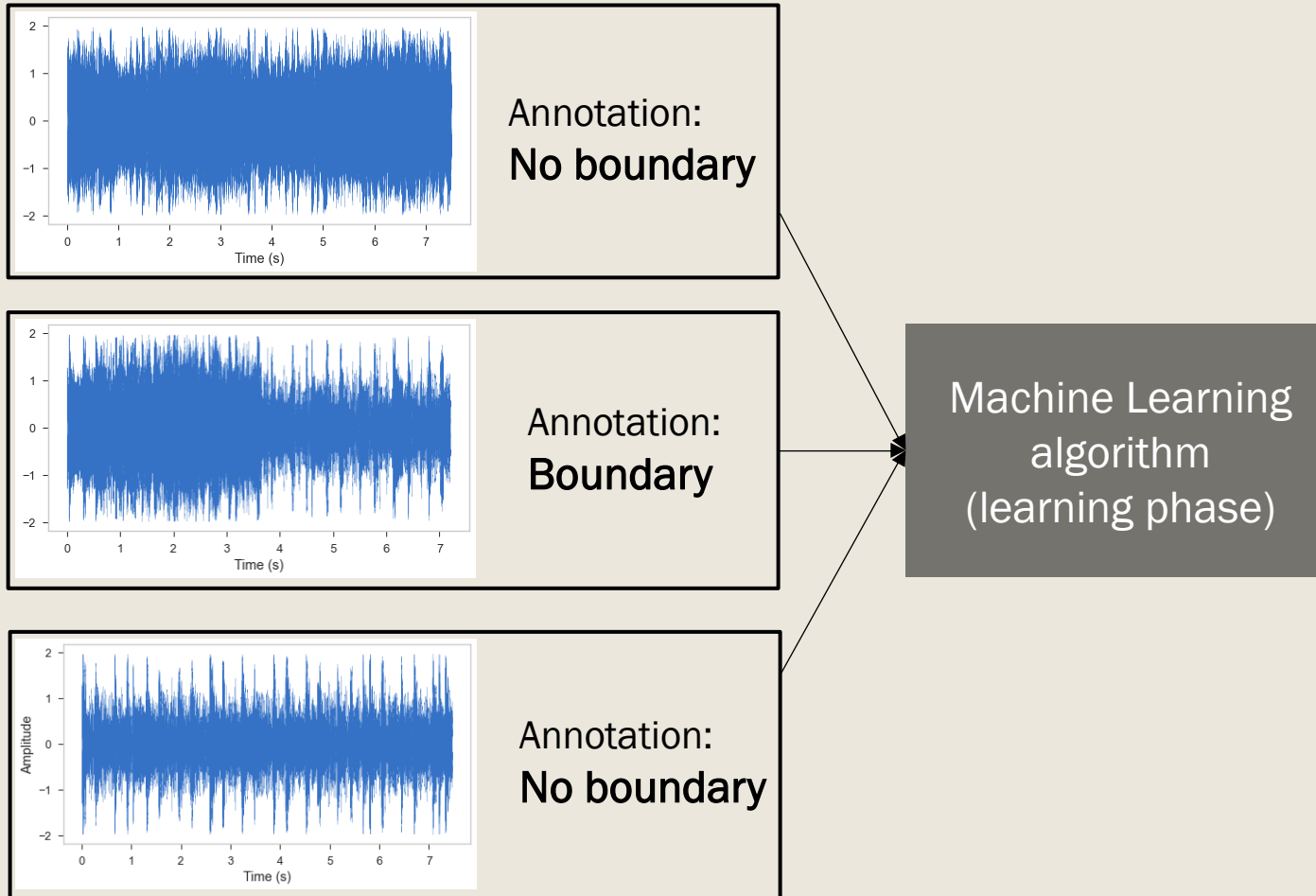
Conventional
algorithm

Decision: **determined by a human** (coded)

Machine
Learning

Decision: **inferred from data** ("learning")

Supervised Machine Learning



- Learning: data → annotation
 - Requires an annotated database
 - RWC Pop [Got+02]
 - SALAMI [Smi+11]

deep learning [GS15]

- ✓ State-of-the-Art
- ✗ Time and energy consuming
- ✗ Annotations: *Hard to collect* and *subjective*

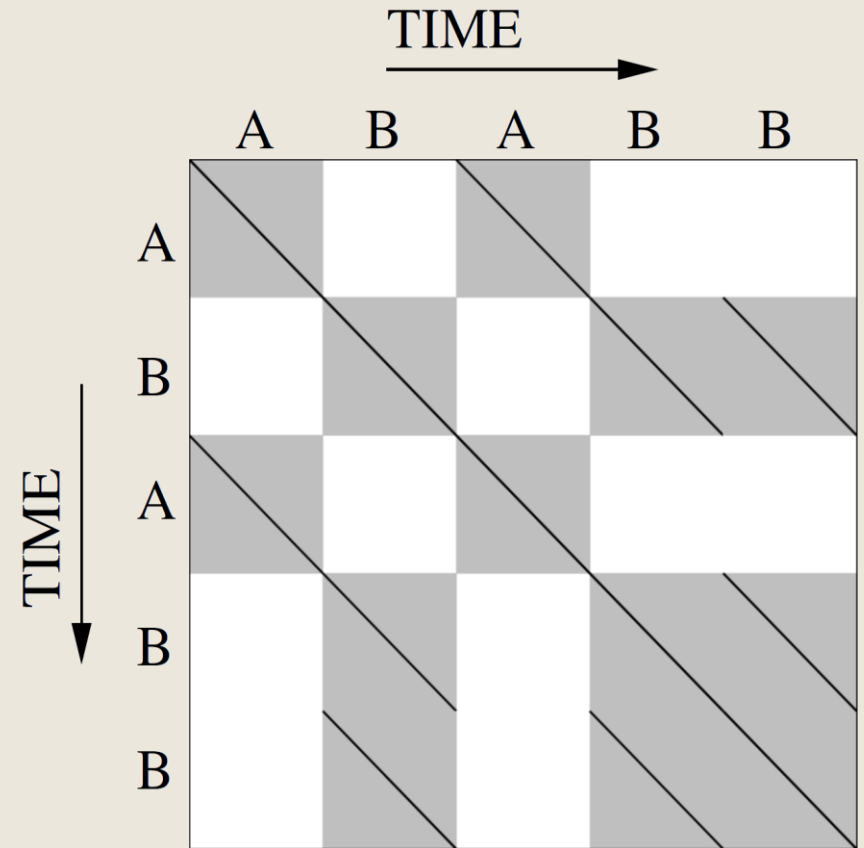
[GS15] Grill, T. and Schlüter, J., « Music Boundary Detection Using Neural Networks on Combined Features and Two-Level Annotations. », in: *ISMIR*, 2015, pp. 531–537

[Got+02] Goto, M., Hashiguchi, H., Nishimura, T., and Oka, R., « RWC Music Database: Popular, Classical and Jazz Music Databases », in: *ISMIR*, vol. 2, 2002, pp. 287–288.

[Smi+11] Smith, J.B.L., Burgoyne, J.A., Fujinaga, I., De Roure, D., and Downie, J.S., « Design and creation of a large-scale database of structural annotations. », in: *ISMIR*, vol. 11, Miami, FL, 2011, pp. 555–560.

Unsupervised Machine Learning

- Based on the similarity of the data
 - *Does not depend on annotation*
- Autosimilarity matrix:
 - *similarity accross time*
 - *Most of unsupervised state-of-the-art techniques*
[Foo00, ME14, Ser+14]



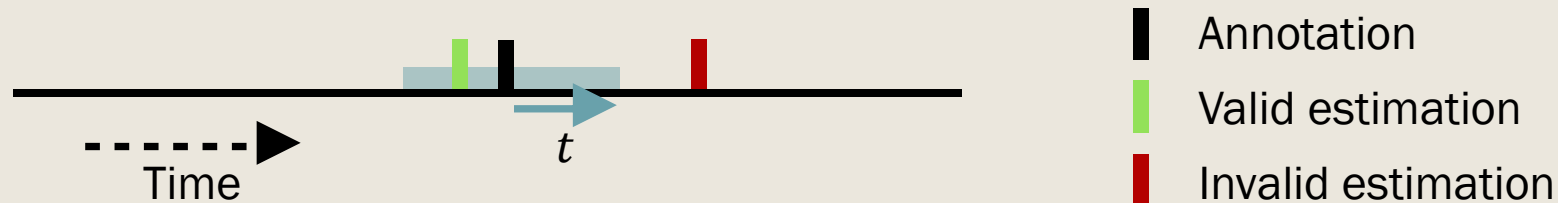
[Foo00] Foote, J. Automatic Audio Segmentation using a Measure of Audio Novelty. In 2000 *Proc. IEEE International Conference on Multimedia and Expo*.

[ME14] McFee, B., & Ellis, D. Analyzing Song Structure with Spectral Clustering. In 2014 *ISMIR*.

[Ser+14] Serra, J., Müller, M., Grosche, P., and Arcos, J.LI., « Unsupervised music structure annotation by time series structure features and segment similarity », in: *IEEE Transactions on Multimedia* 16.5 (2014), pp. 1229–1240.

How is structure evaluated?

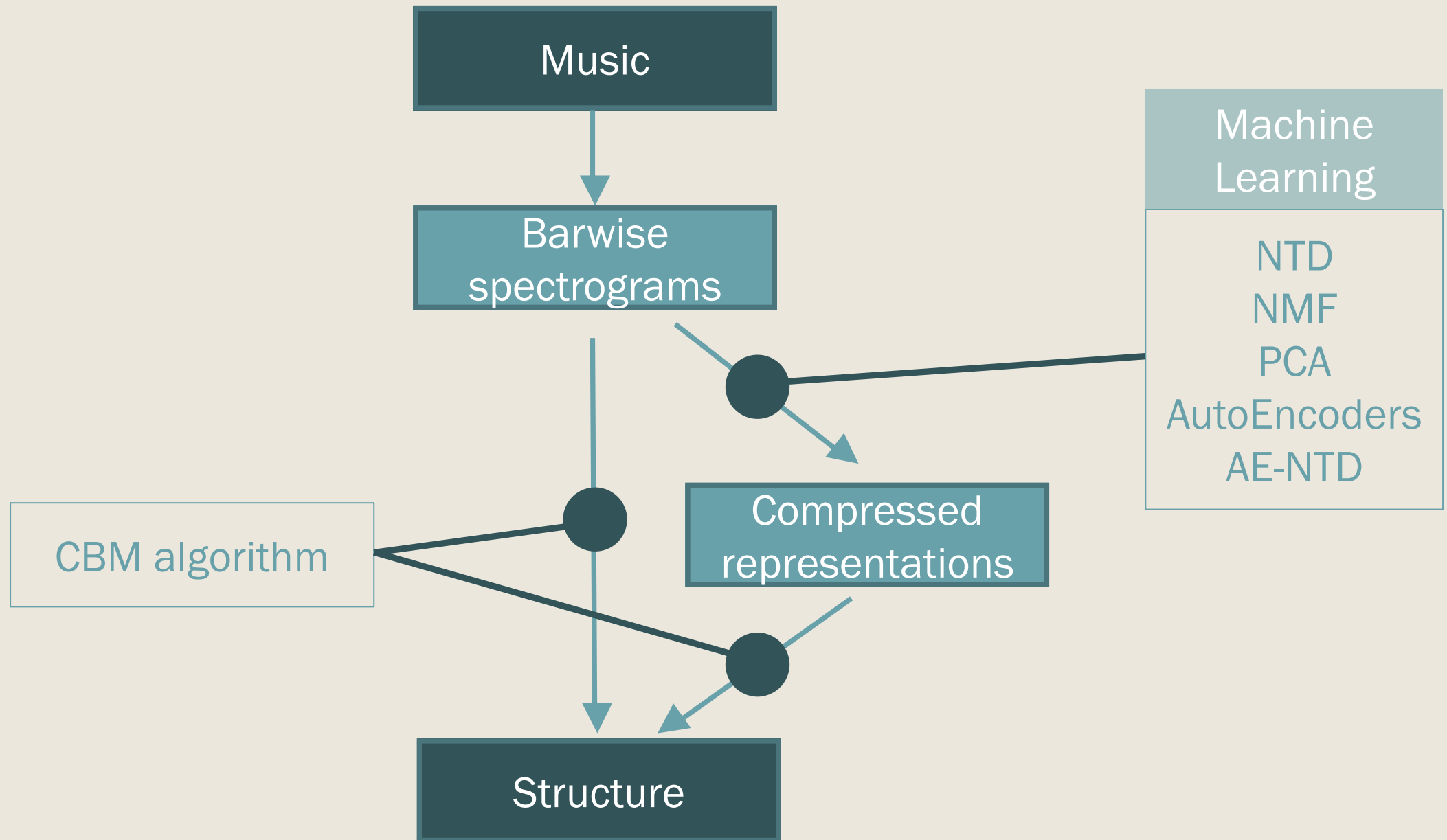
- Accordance of two sets of boundaries (in **seconds**)
 - *Estimated boundaries* (e.g. [0, 3.1, 4.5])
 - *Annotated boundaries* (e.g. [0, 4.7, 5.2])
- Boundary: valid if close to annotation, with a tolerance t

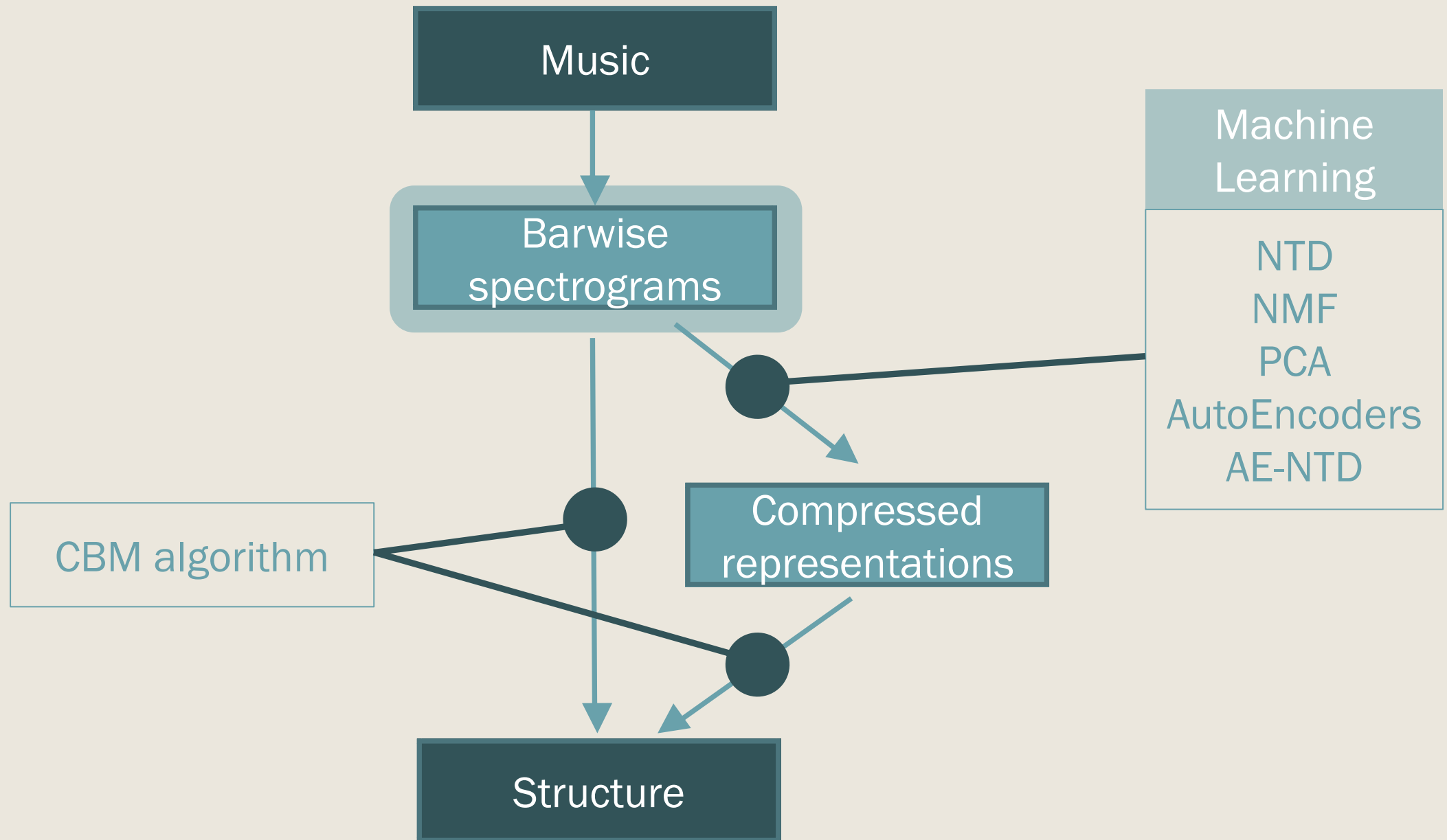


- Metrics: ($t = 0.5s$ and $t = 3s$)
 - $Precision (P_t) = \frac{\text{Valid estimation}}{\text{Valid estimation} + \text{Invalid estimation}}$
 - $Recall (R_t) = \frac{\text{Valid estimation}}{\text{Annotation}}$
 - $F\text{-measure} (F_t) = \frac{2PR}{P+R}$

$F_{0.5}$
 F_3

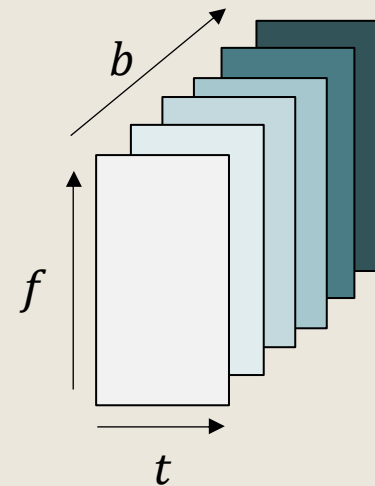
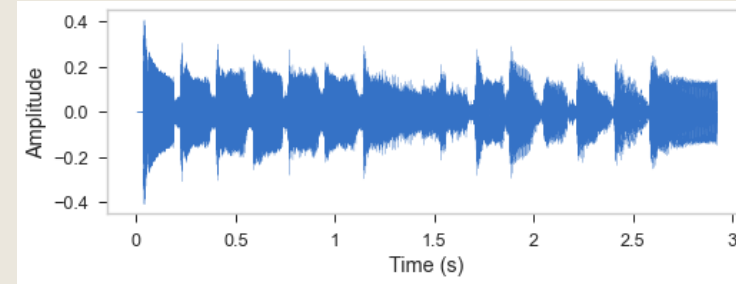
(MIREX standard)



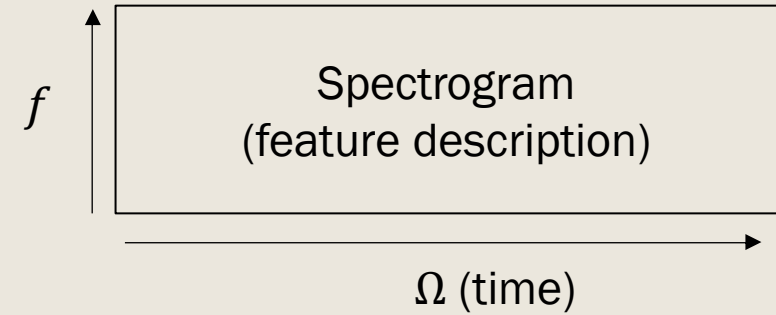
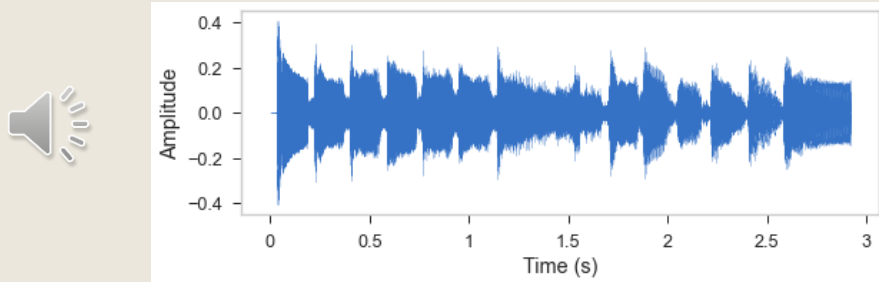


Barwise spectrograms

Barwise spectrograms

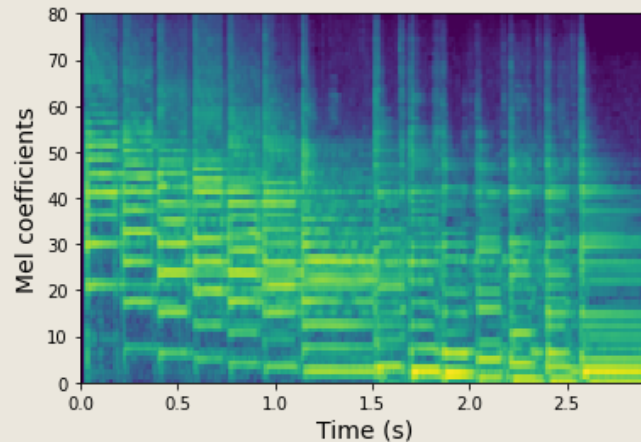


Spectrogram?

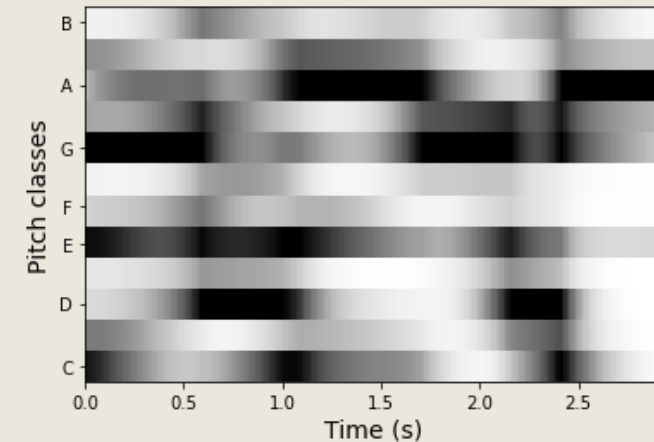


- Features: frequency content (*Fourier Analysis*)

Frequency scale
(Log Mel Spectrogram)



Musical scale
(Chromagram)



Barwise scale

Für Elise
Clavierstück in A Minor - WoO 59
Ludwig van Beethoven

Poco moto.
pp

Public Domain

Barwise scale

Für Elise
Clavierstück in A minor - WoO 39
Ludwig van Beethoven

Poco moto.
pp

Public Domain

Barwise scale

Für Elise
Clavierstück in A minor - WoO 39
Ludwig van Beethoven

pp *Poco moto.*

The image shows the first page of the musical score for 'Für Elise' by Ludwig van Beethoven. The score is in 3/8 time, A minor, and is marked 'pp' (pianissimo) and 'Poco moto.' (moderately). The score is divided into measures, with measures 1 through 29 visible. Each measure is highlighted with a light blue background. A pair of scissors icon is placed above each measure, indicating a cut point. The score is written for piano, with a treble and bass staff. The key signature has two flats (B-flat and E-flat). The tempo and dynamics are indicated at the beginning. The score is in the public domain.

Public Domain

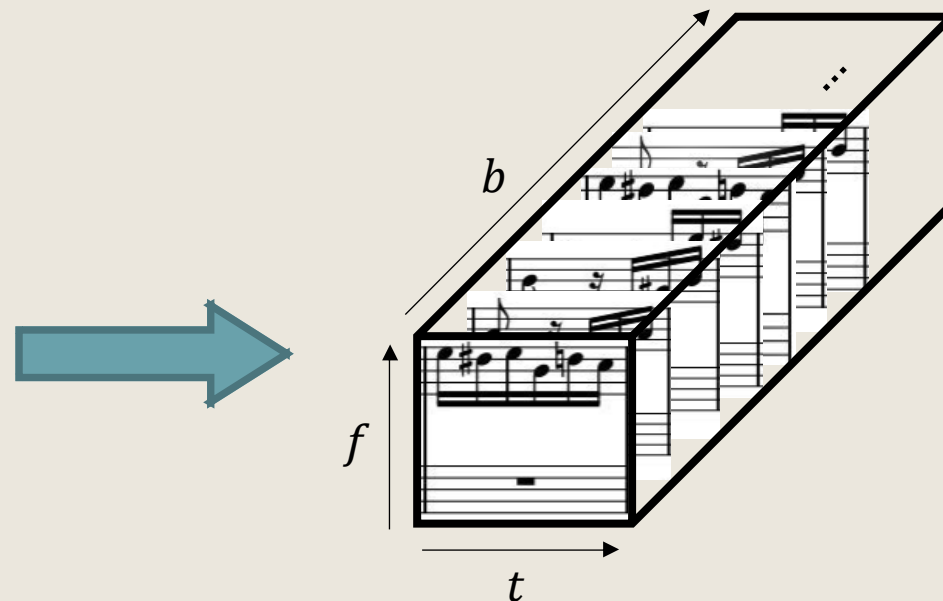


Barwise scale

Für Elise
Clavierstück in A minor - WoO 39
Ludwig van Beethoven

pp *mo*

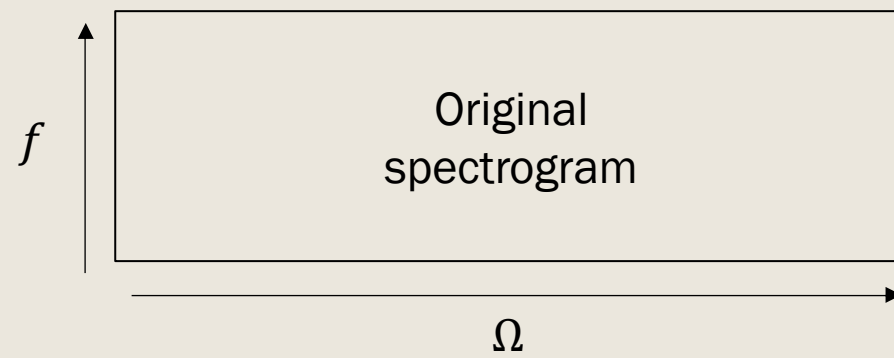
The image shows a musical score for 'Für Elise' by Ludwig van Beethoven. The score is written for piano in 3/8 time, in A minor. It is divided into measures, with some measures containing multiple notes. Vertical blue bars are placed at the beginning of each measure, indicating a barwise segmentation. Scissors icons are placed above each bar, suggesting a 'cutting' or 'segmenting' process. The score is labeled 'Für Elise', 'Clavierstück in A minor - WoO 39', and 'Ludwig van Beethoven'. The dynamic markings 'pp' (pianissimo) and 'mo' (mezzo-forte) are present. The score is marked 'Public Domain' at the bottom.

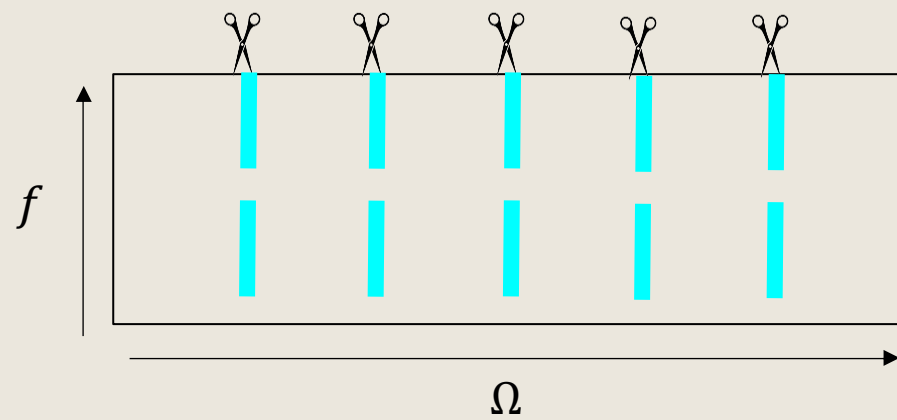


Hypotheses for barwise processing

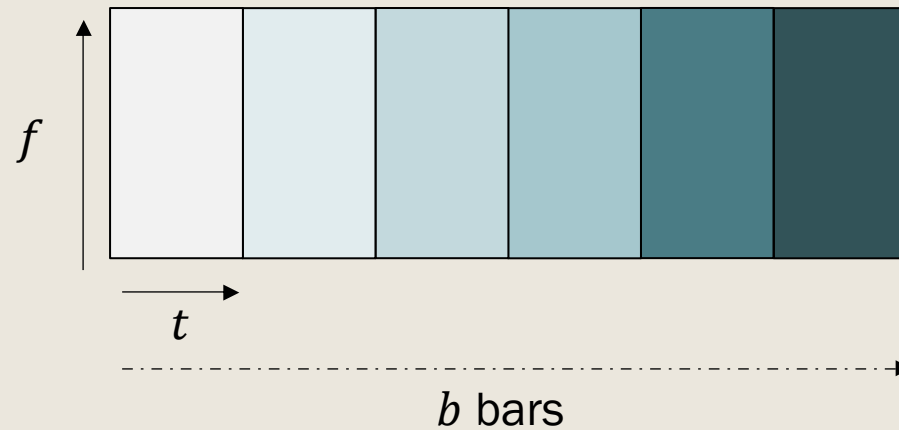
- Hypothesis 1: **patterns tend to develop at the bar scale**
- Hypothesis 2: **boundaries are located on bars**

Bars are estimated via an external toolbox (madmom [Böc+16])

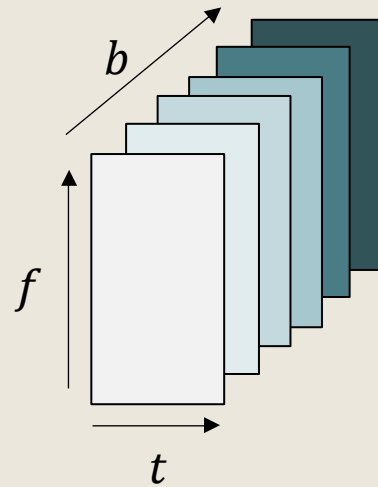




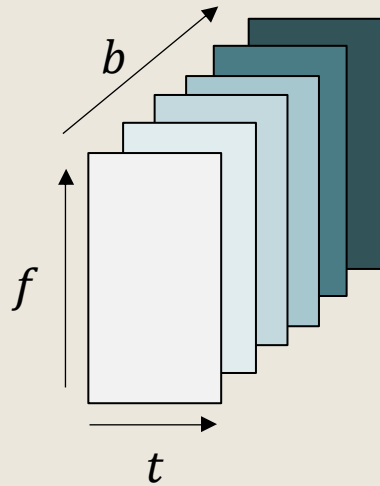
Barwise spectrograms



Time-Frequency-Bar Tensor (TFB)



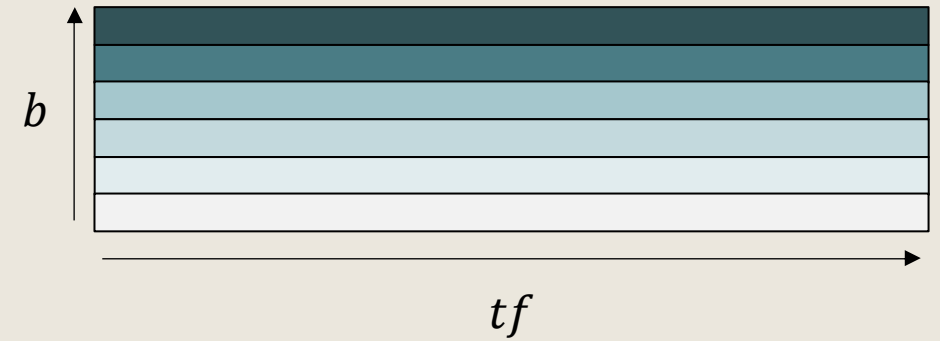
Time-Frequency-Bar Tensor (TFB)

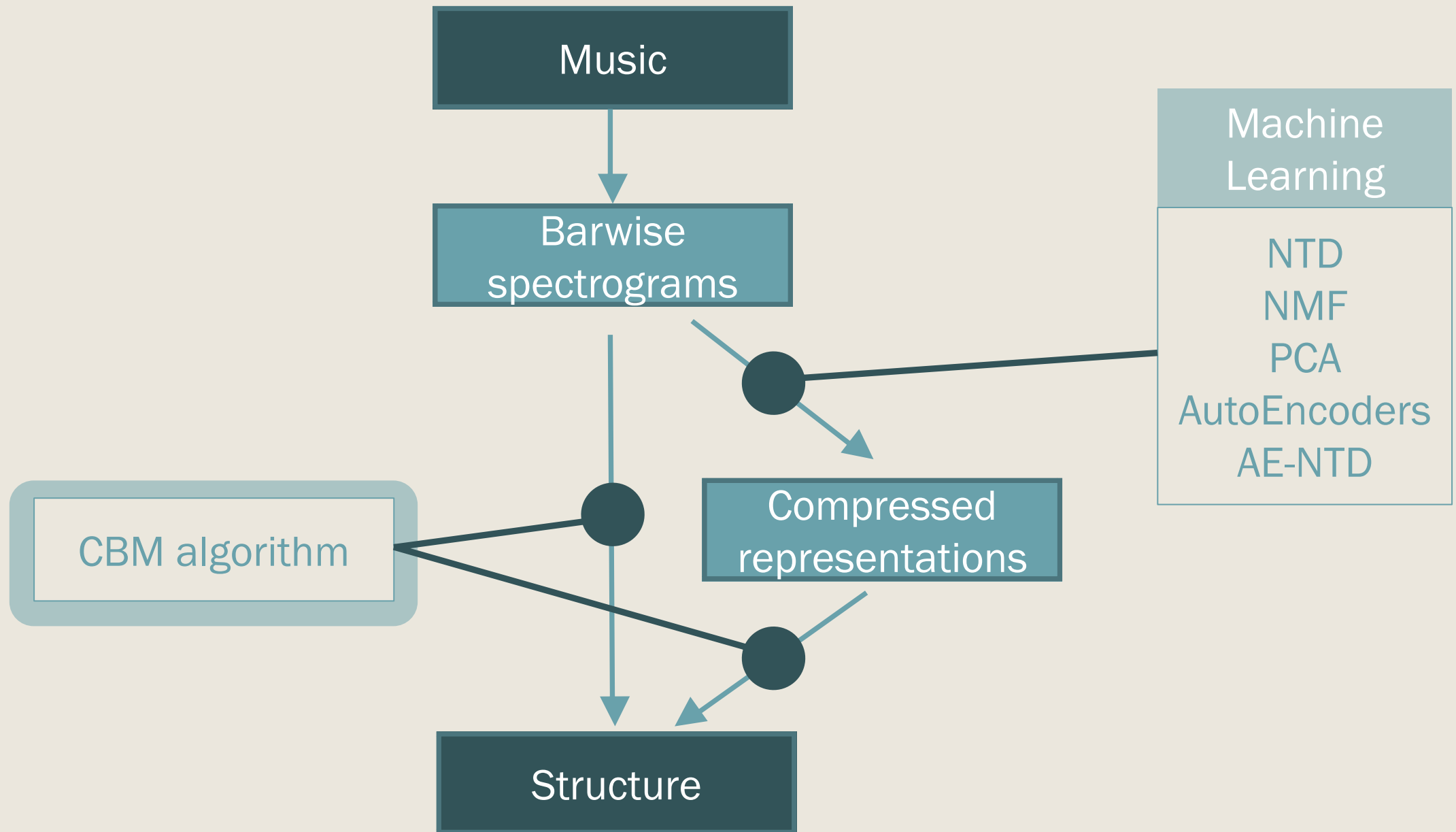


"Matricisation"



Barwise TF matrix





Convolutional Block-Matching (CBM) algorithm

[MCB22a]

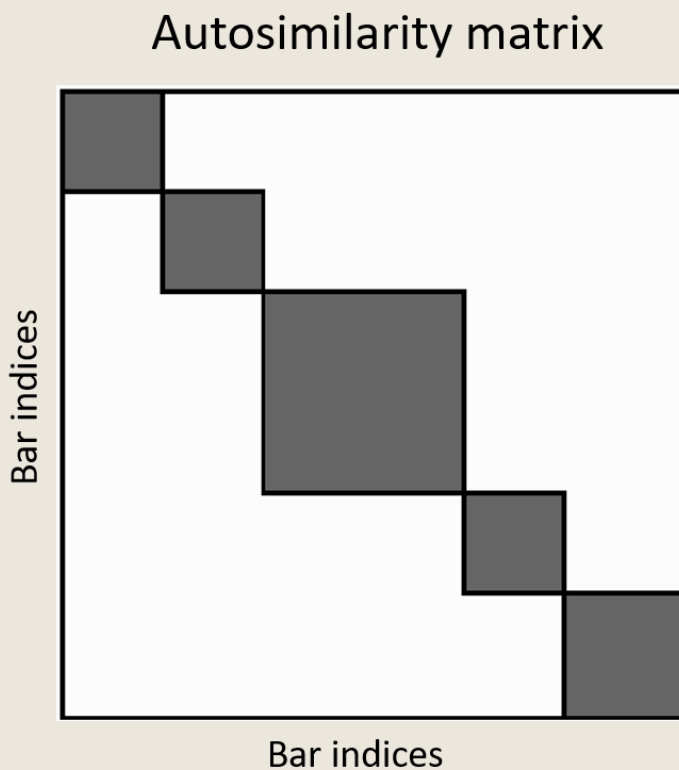
(with open-source code)

[MCB22a] Marmoret, A., Cohen, J. E., & Bimbot, F. (2022). Convolutional Block-Matching Segmentation Algorithm with Application to Music Structure Analysis. *arXiv preprint arXiv:2210.15356*.
(Submitted to ICASSP 2023)

CBM algorithm

[MCB22a]

(with open-source code)

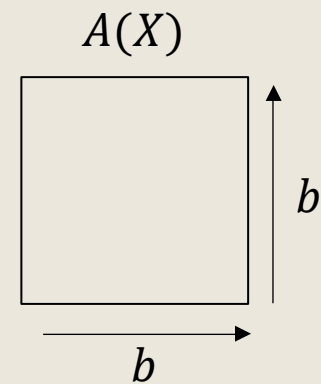
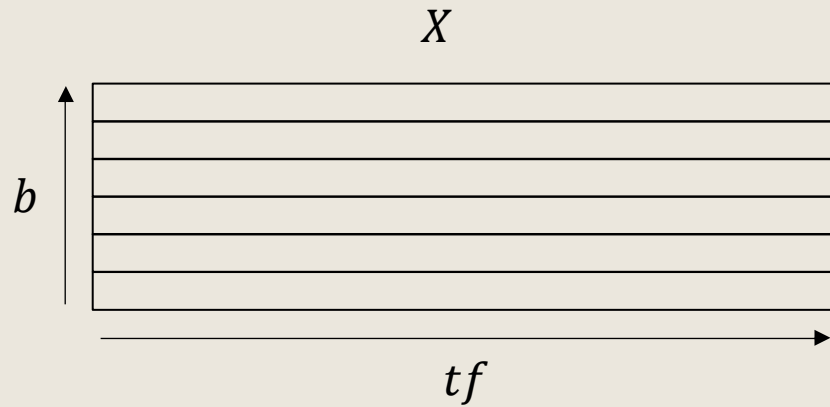


[0,4,8,16,20,25]

[MCB22a] Marmoret, A., Cohen, J. E., & Bimbot, F. (2022). Convolutional Block-Matching Segmentation Algorithm with Application to Music Structure Analysis. *arXiv preprint arXiv:2210.15356*. (Submitted to ICASSP 2023)

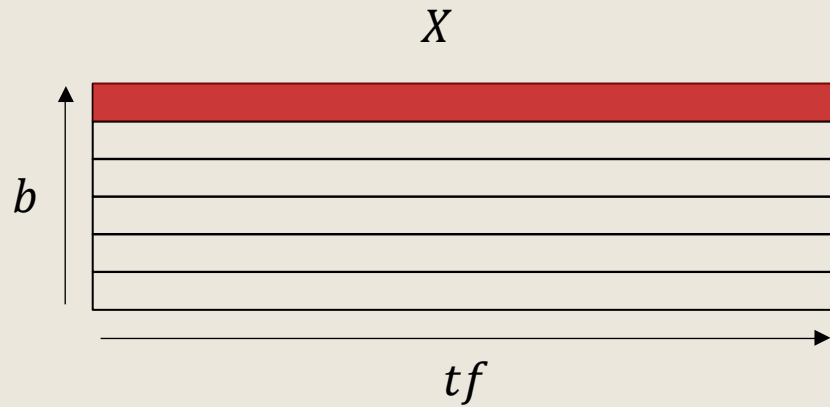
Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$

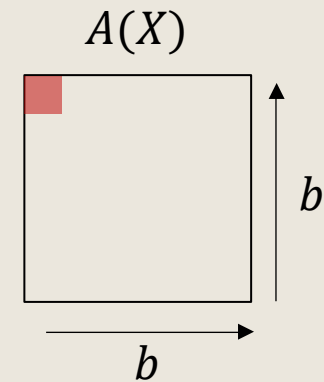


Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$

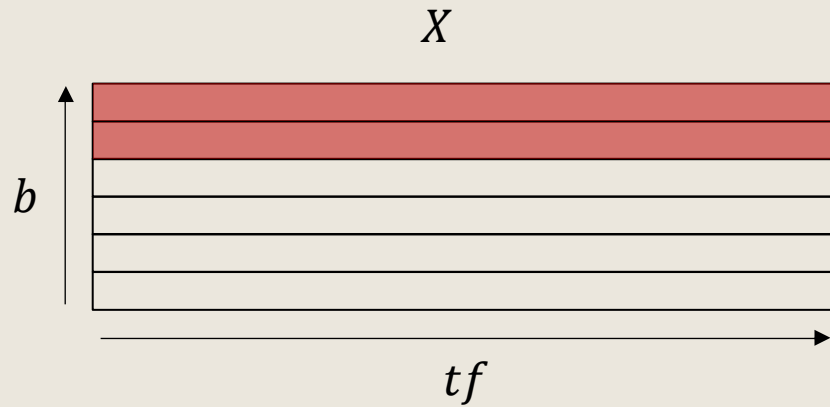


$$A(X)_{11} = s(X_1, X_1)$$

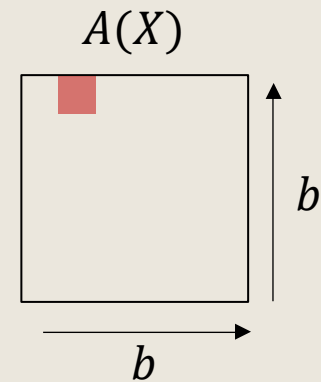


Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$

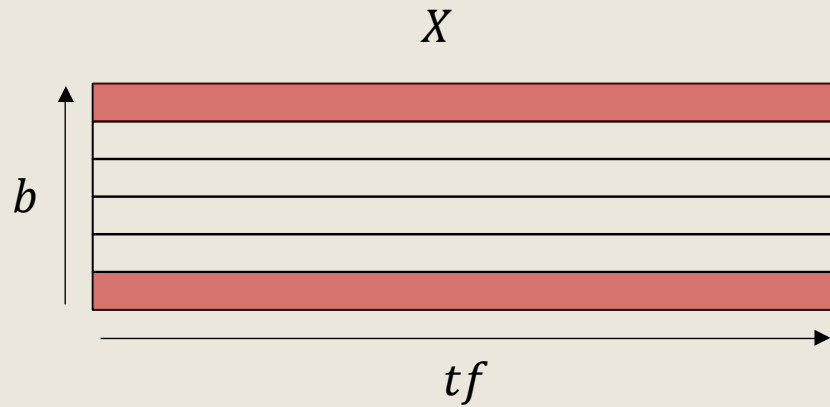


$$A(X)_{12} = s(X_1, X_2)$$

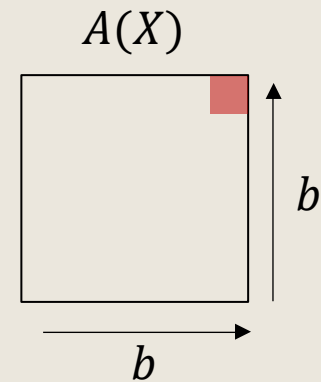


Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$

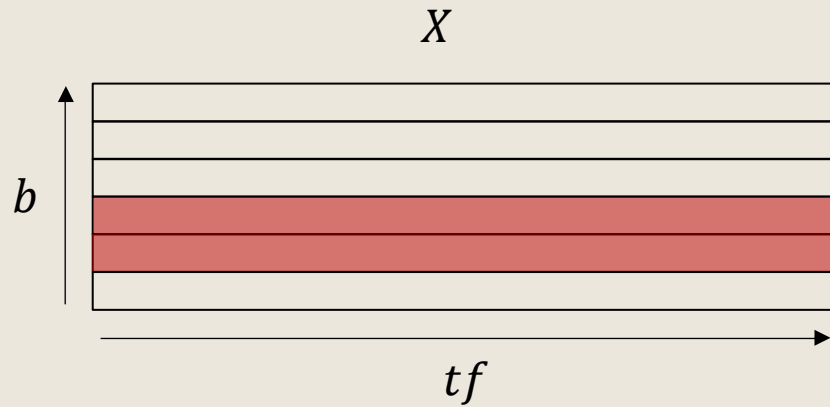


$$A(X)_{16} = s(X_1, X_6)$$

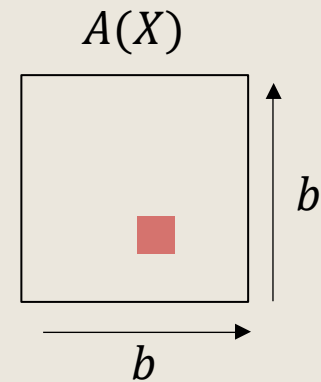


Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$

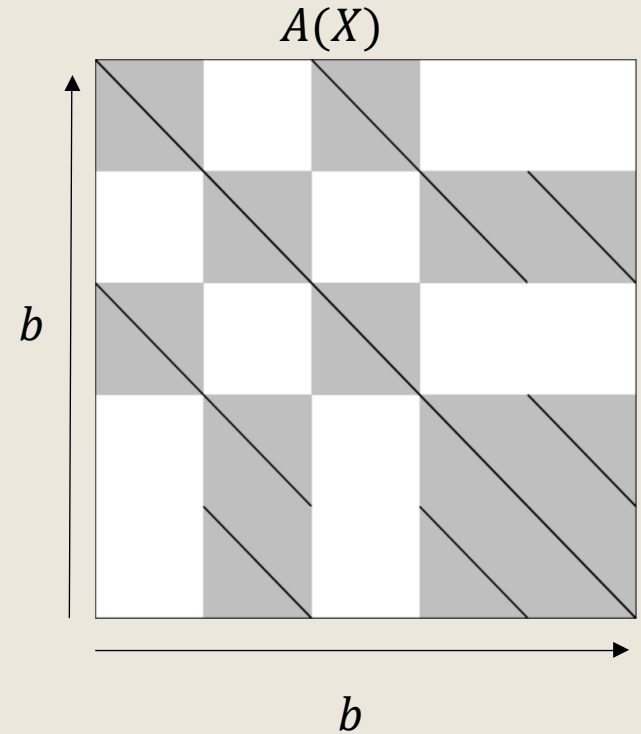
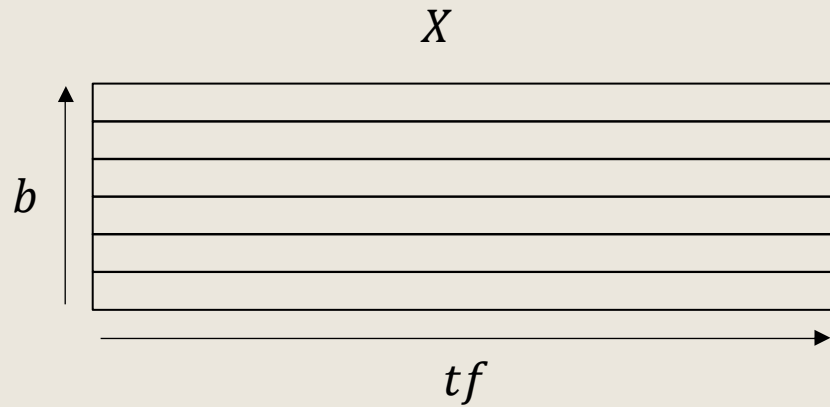


$$A(X)_{45} = s(X_4, X_5)$$

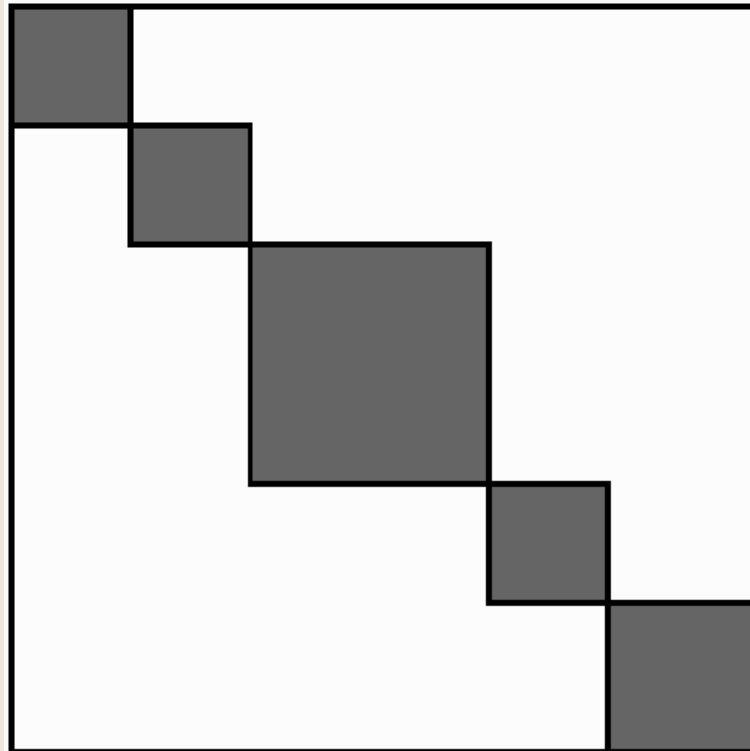


Barwise autosimilarity

- Similarity between pairs of bars X_i, X_j
 - Depends on a similarity function $s()$



Convolutional Block-Matching (CBM) algorithm



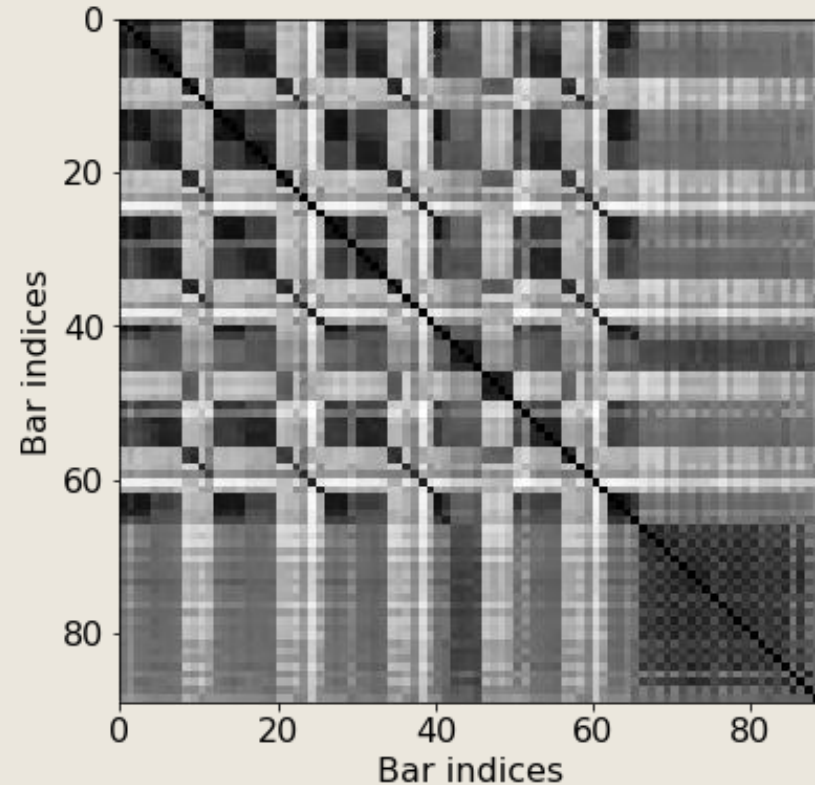
[0,4,8,16,20,25]

- Idea: framing square blocks of high similarity in the diagonal
- Principle:
 - **score** $\gamma(S_i)$ for **each segment** (weighted average)
 - **maximizing** the segmentation (set of segments) via **dynamic programming**

$$Z^* = \operatorname{argmax}_Z \sum_{S_i \in Z} \gamma(S_i)$$

Cosine autosimilarity

"Come Together"

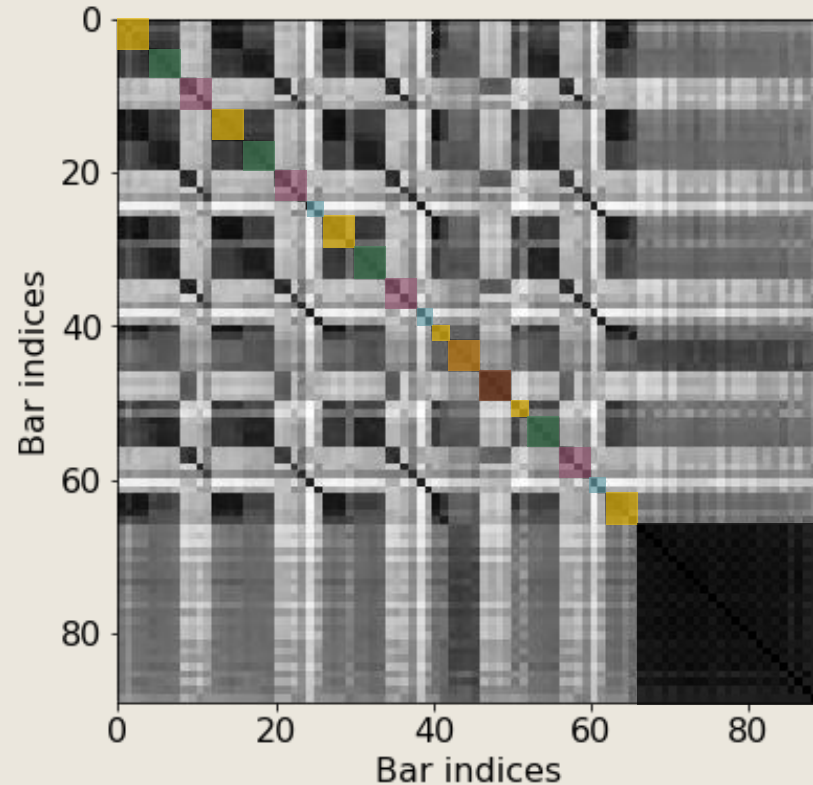


- $s(X_i, X_j) = \frac{\langle X_i, X_j \rangle}{\|X_i\| \|X_j\|}$

- Standard autosimilarity in literature [Foo00]

Cosine autosimilarity

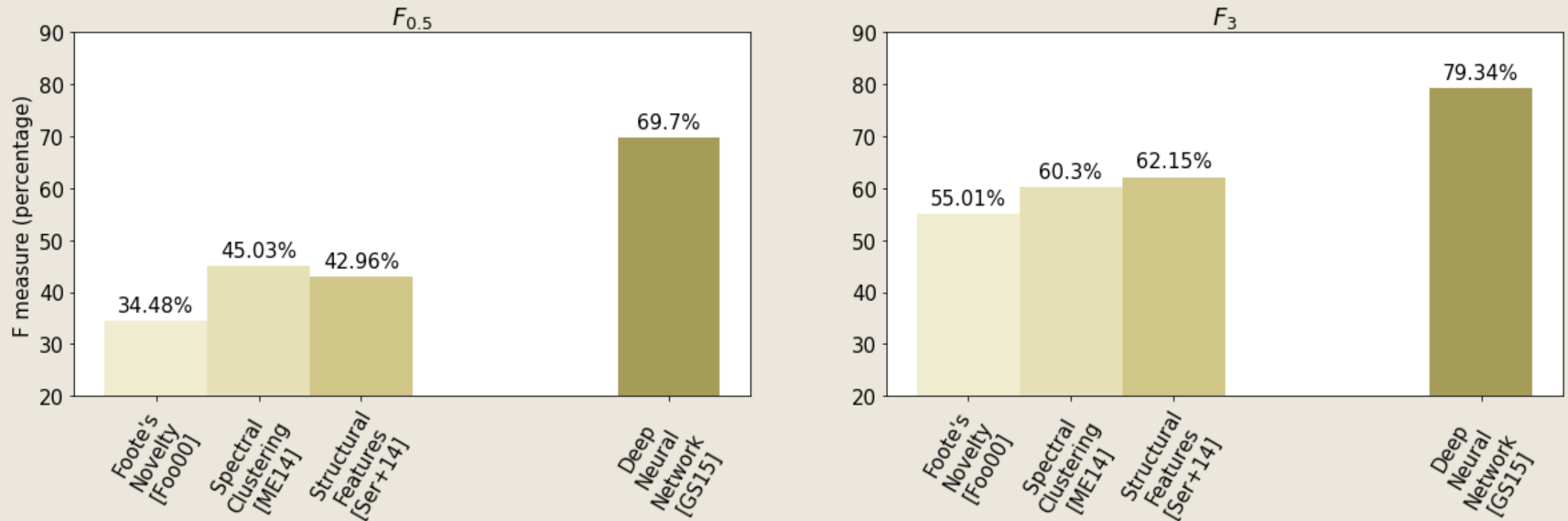
"Come Together"



- $s(X_i, X_j) = \frac{\langle X_i, X_j \rangle}{\|X_i\| \|X_j\|}$

- Standard autosimilarity in literature [Foo00]

CBM algorithm, in practice



Segmentation results, compared with the State-of-the-Art (RWC Pop dataset [Got+02])

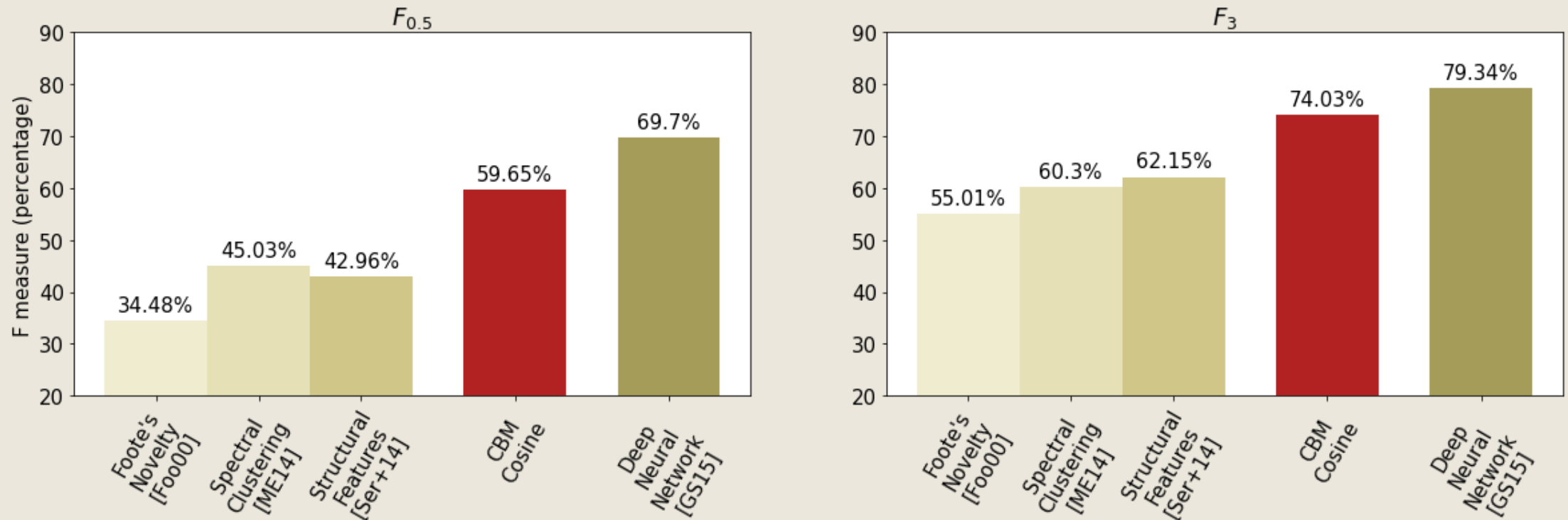
[Foo00] Foote, J. Automatic Audio Segmentation using a Measure of Audio Novelty. In 2000 *Proc. IEEE International Conference on Multimedia and Expo*.

[ME14] McFee, B., & Ellis, D. Analyzing Song Structure with Spectral Clustering. In 2014 *ISMIR*.

[Ser+14] Serra, J., Müller, M., Grosche, P., and Arcos, J.L., « Unsupervised music structure annotation by time series structure features and segment similarity », in: *IEEE Transactions on Multimedia* 16.5 (2014), pp. 1229–1240.

[GS15] Grill, T., & Schlüter, J. Music Boundary Detection Using Neural Networks on Combined Features and Two-Level Annotations. In 2015 *ISMIR*.

CBM algorithm, in practice



Segmentation results, compared with the State-of-the-Art (RWC Pop dataset [Got+02])

- Outperforms the unsupervised State-of-the-Art algorithms

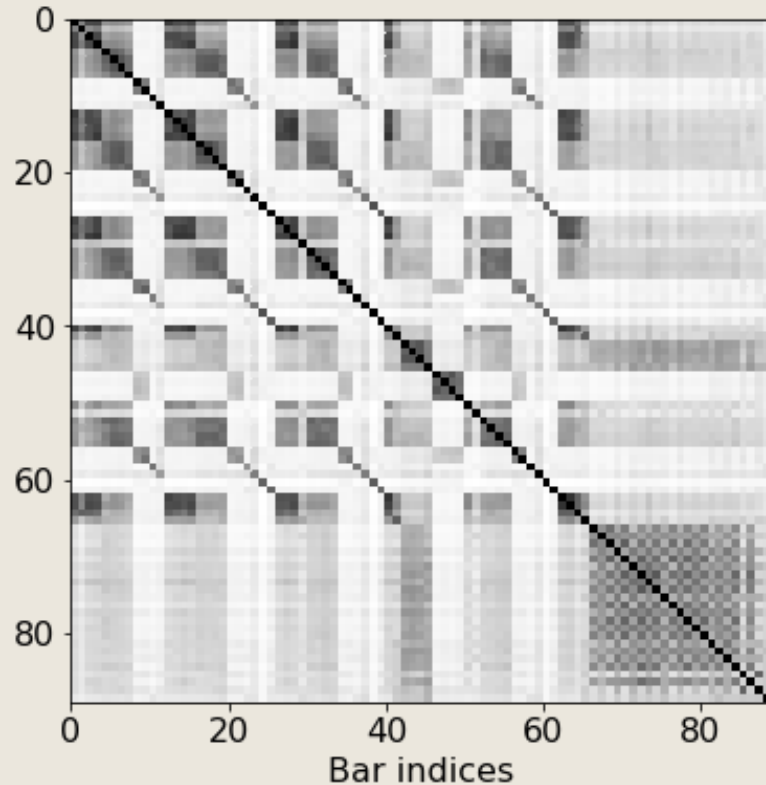
[Foo00] Foote, J. Automatic Audio Segmentation using a Measure of Audio Novelty. In 2000 *Proc. IEEE International Conference on Multimedia and Expo*.

[ME14] McFee, B., & Ellis, D. Analyzing Song Structure with Spectral Clustering. In 2014 *ISMIR*.

[Ser+14] Serra, J., Müller, M., Grosche, P., and Arcos, J.L., « Unsupervised music structure annotation by time series structure features and segment similarity », in: *IEEE Transactions on Multimedia* 16.5 (2014), pp. 1229–1240.

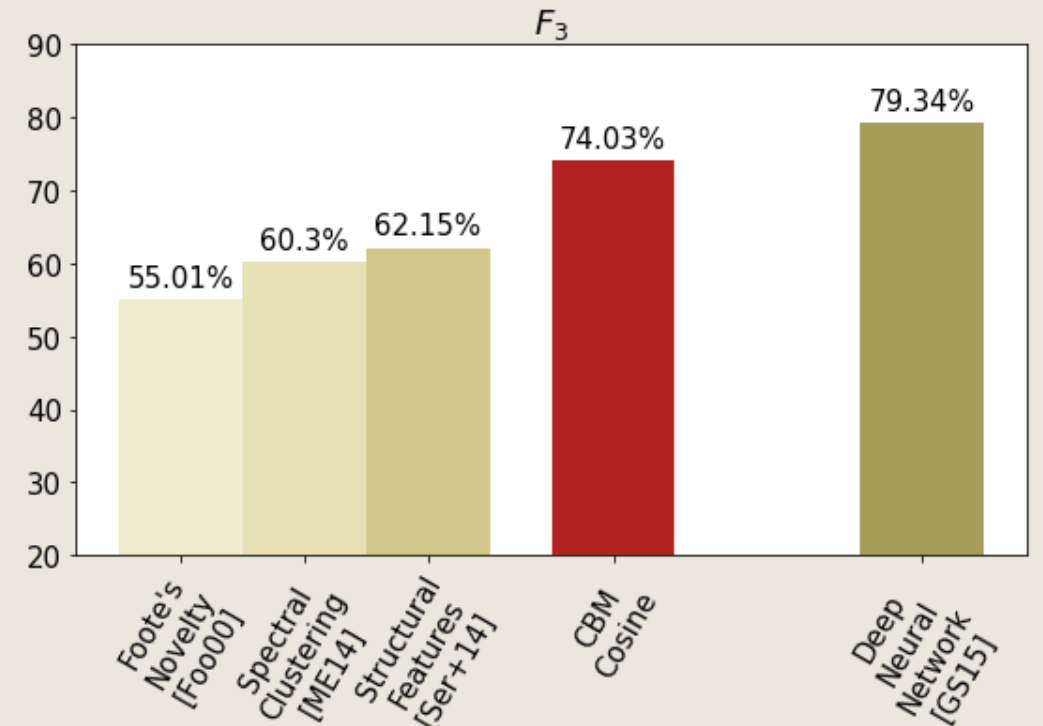
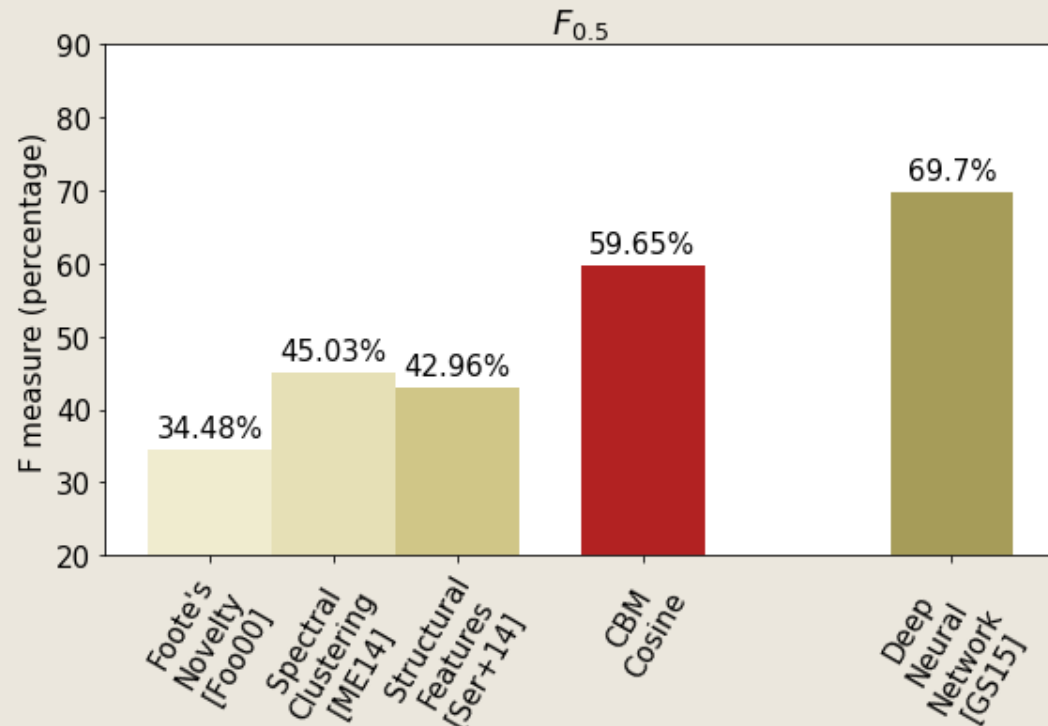
[GS15] Grill, T., & Schlüter, J. Music Boundary Detection Using Neural Networks on Combined Features and Two-Level Annotations. In 2015 *ISMIR*.

RBF autosimilarity



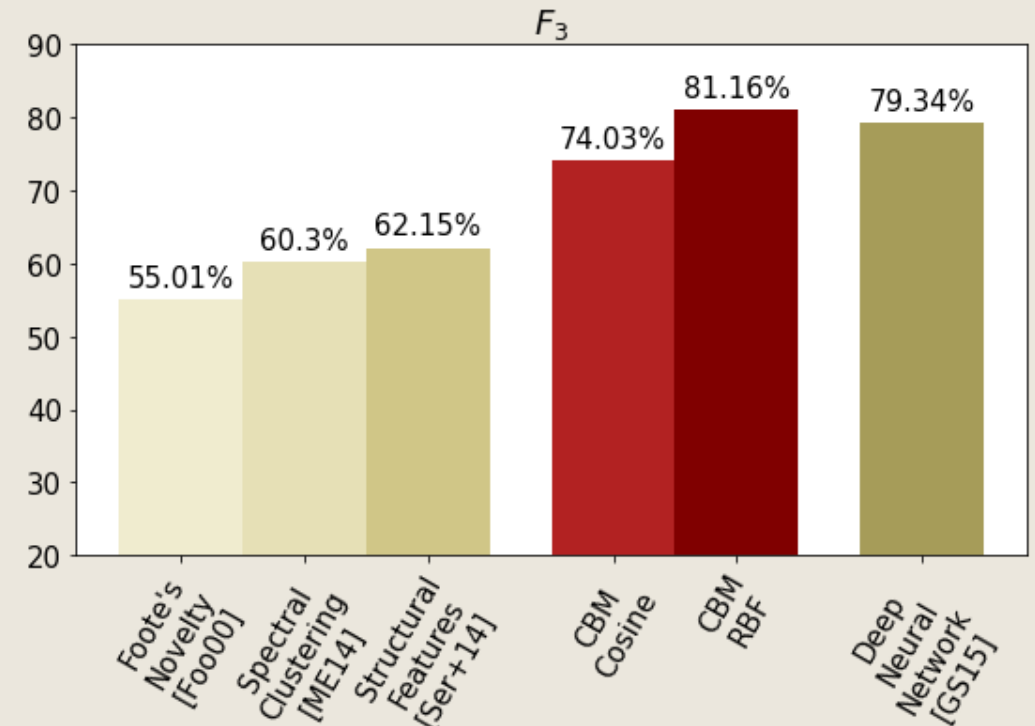
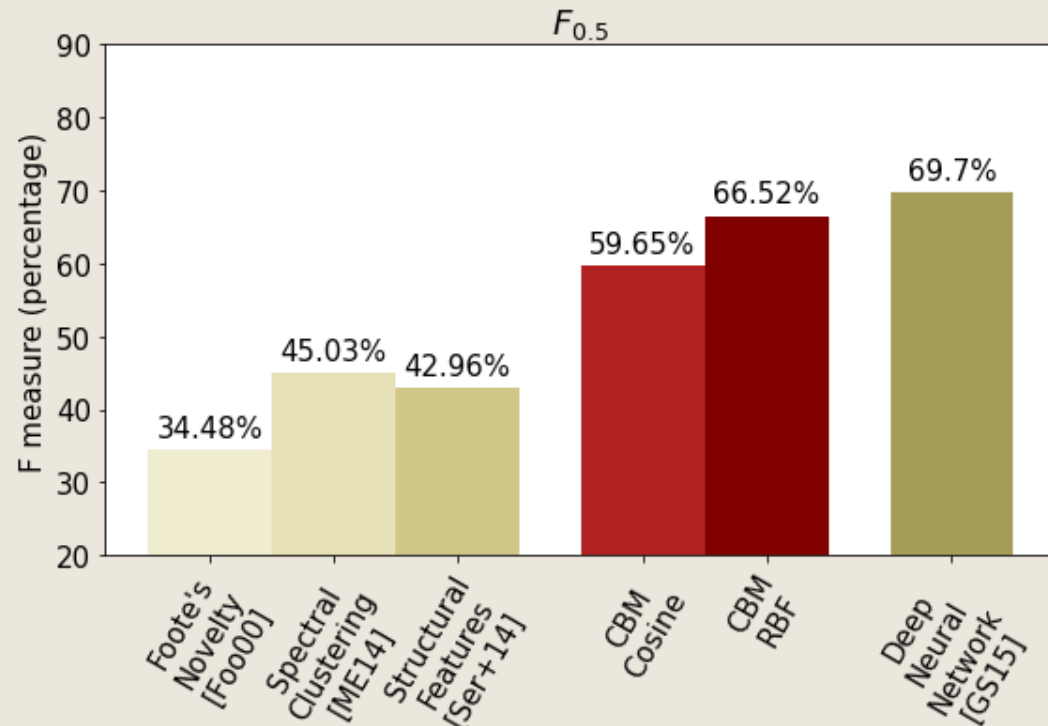
- $s(X_i, X_j) = e^{-\frac{1}{2\sigma} \left\| \frac{X_i}{\|X_i\|} - \frac{X_j}{\|X_j\|} \right\|_2}$
- Better represents the **dissimilarity** between bars
 - *Higher "contrast"*

CBM algorithm, RBF



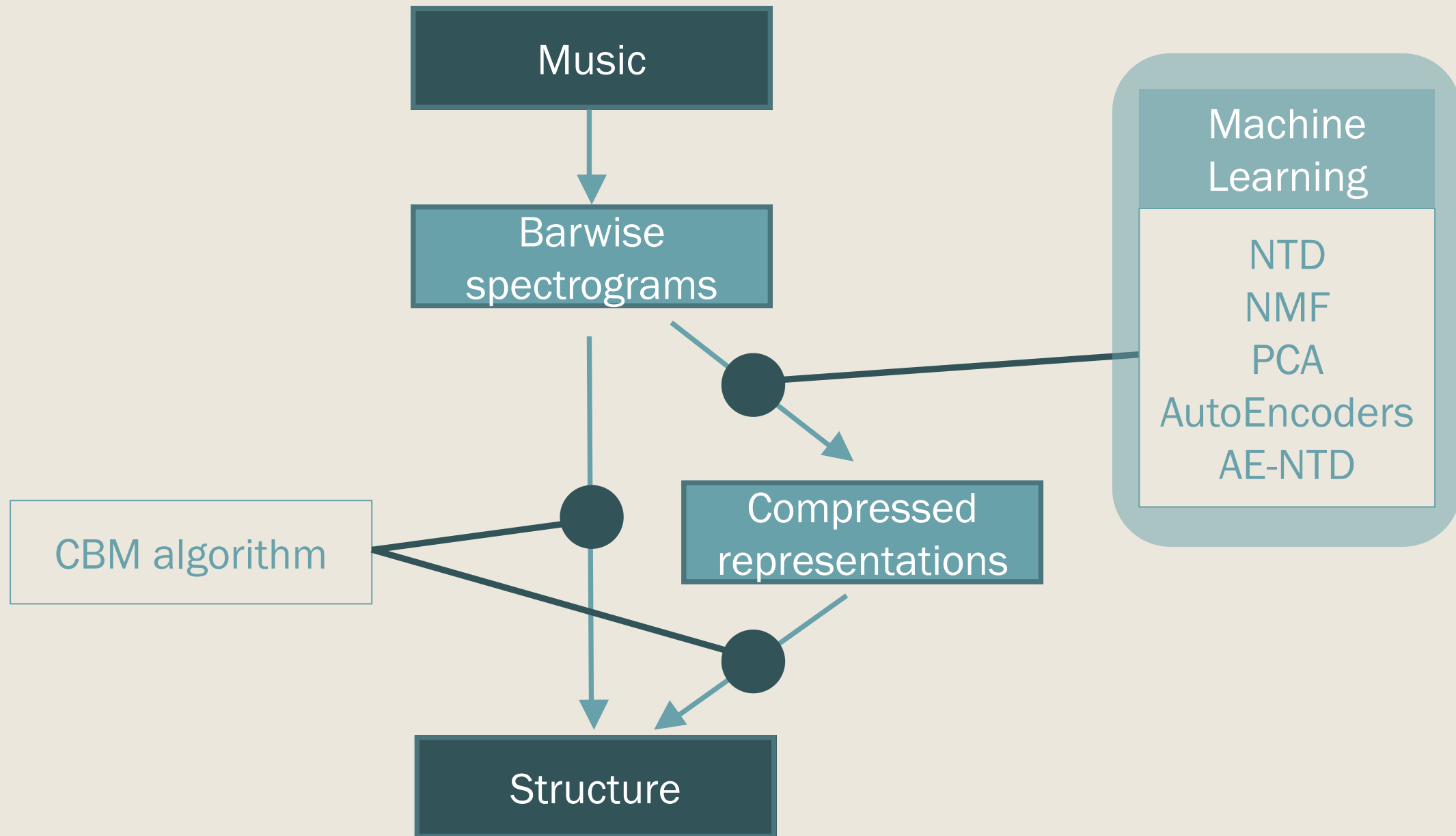
Segmentation results, compared with the State-of-the-Art (RWC Pop dataset)

CBM algorithm, RBF



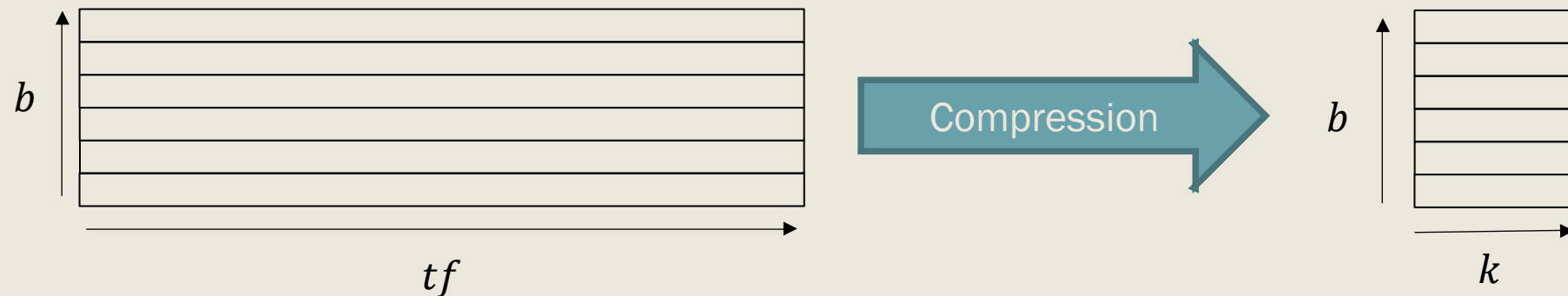
Segmentation results, compared with the State-of-the-Art (RWC Pop dataset)

- RBF results outperforms Cosine ones
 - Due to higher **contrast**
- Levels of performance close to those of the State-of-the-Art **without supervision**

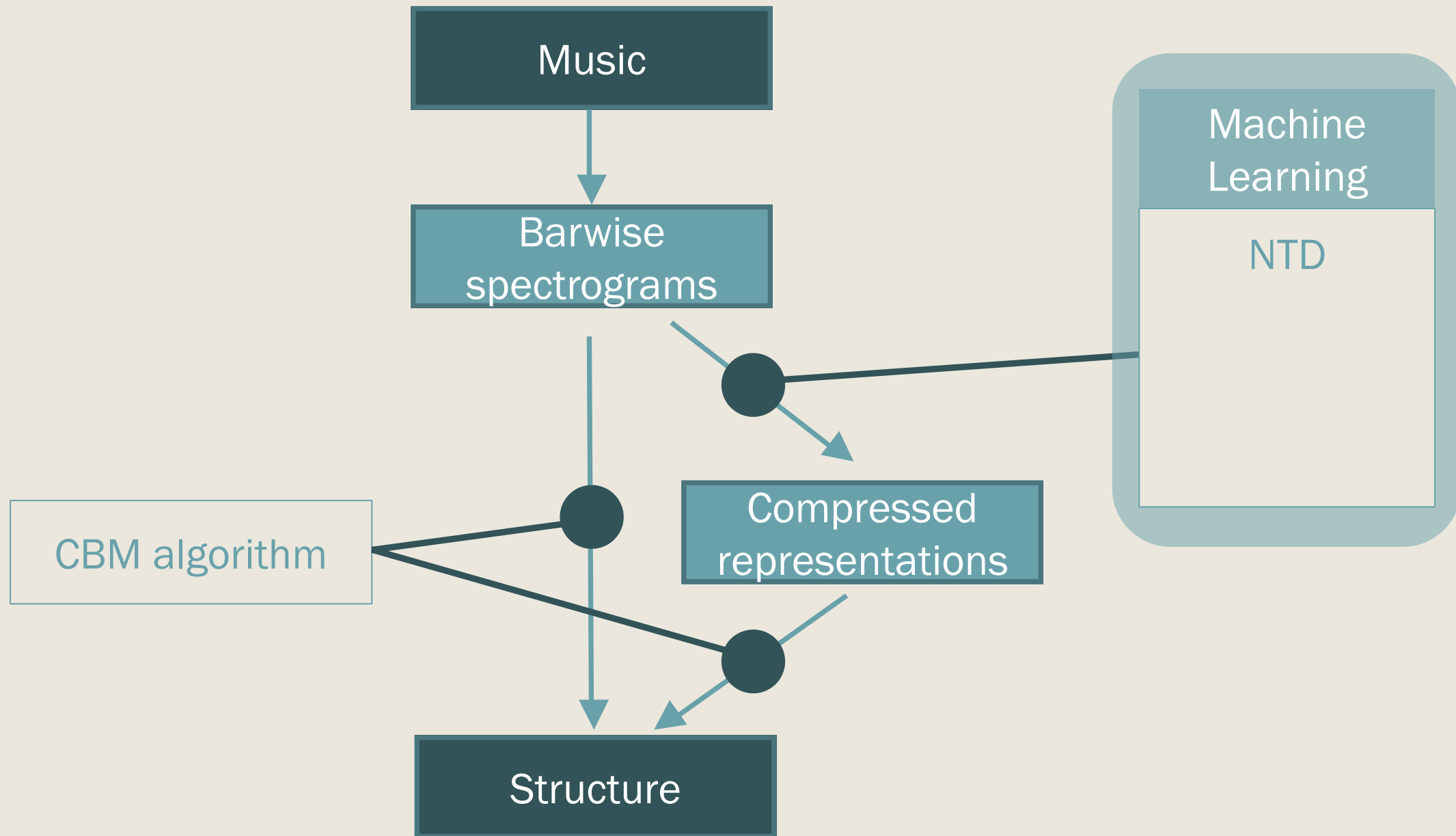


Machine Learning used for compression

- Summing up each bar to a few important components
 - *Compression must ensure accurate reconstruction*



- Objective: disambiguating the barwise dissimilarity
 - *i.e. increasing the contrast (similarly to RBF)*



Nonnegative Tucker Decomposition

[KC07, SG18, Mar+20, Mar+22]

[KC07] Kim, Y. D., & Choi, S. Nonnegative tucker decomposition. In *2007 IEEE conference on computer vision and pattern recognition*.

[SG18] Smith, J. B., & Goto, M.. Nonnegative tensor factorization for source separation of loops in audio. In *2018 IEEE ICASSP*.

[Mar+20] Marmoret, A., Cohen, J.E., Bertin, N., & Bimbot, F. Uncovering Audio Patterns in Music with Nonnegative Tucker Decomposition for Structural Segmentation. In *2020 ISMIR*.

[Mar+22] Marmoret, A., et al.. Nonnegative Tucker Decomposition with Beta-divergence for Music Structure Analysis of audio signals. *GRETSI2022*.

Nonnegative Tucker Decomposition

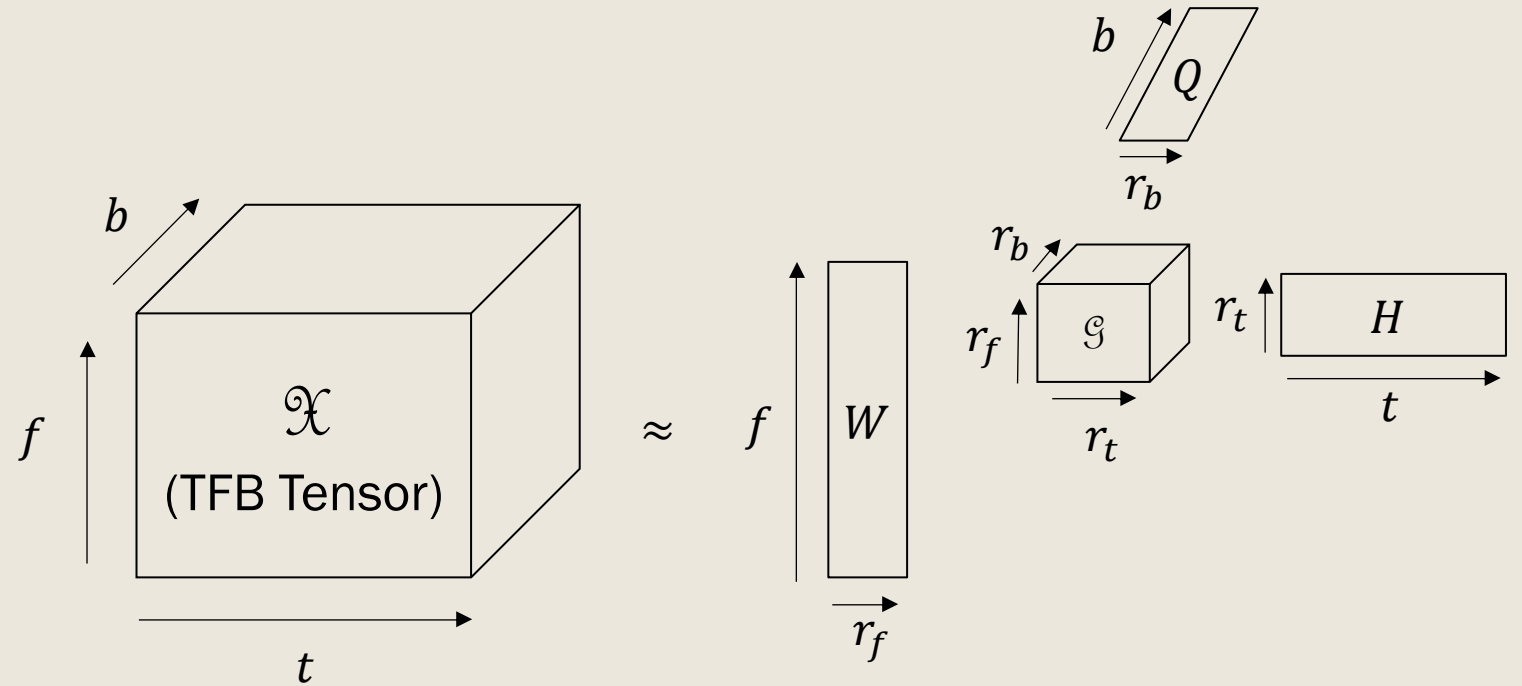
[KC07, SG18, Mar+20, Mar+22]

[KC07] Kim, Y. D., & Choi, S. Nonnegative Tucker decomposition. In 2007 IEEE conference on computer vision and pattern recognition.

[SG18] Smith, J. B., & Goto, M.. Nonnegative tensor factorization for source separation of loops in audio. In 2018 IEEE ICASSP.

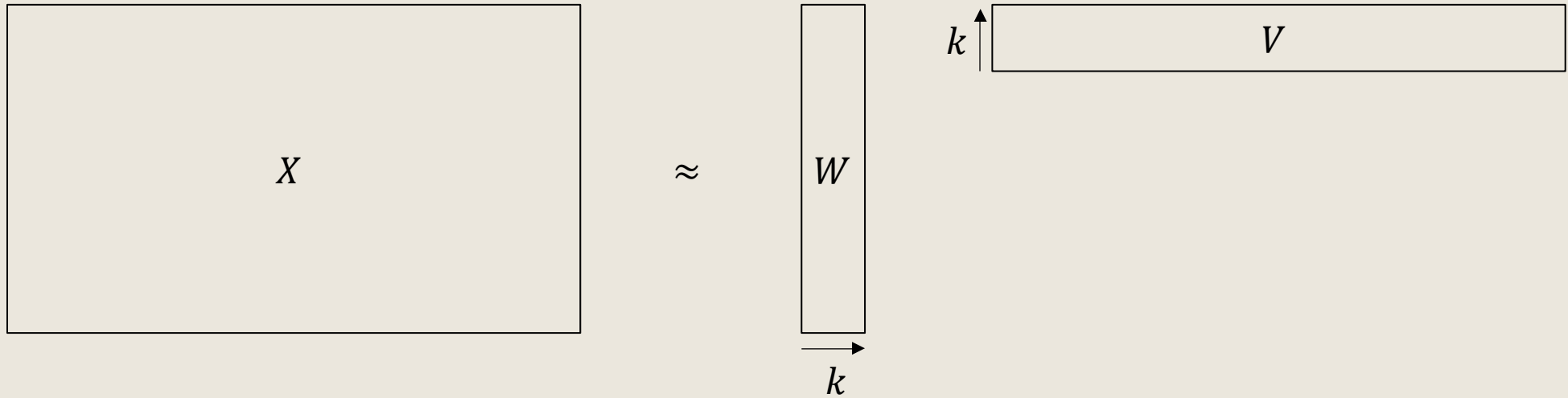
[Mar+20] Marmoret, A., Cohen, J.E., Bertin, N., & Bimbot, F. Uncovering Audio Patterns in Music with Nonnegative Tucker Decomposition for Structural Segmentation. In 2020 ISMIR.

[Mar+22] Marmoret, A., et al.. Nonnegative Tucker Decomposition with Beta-divergence for Music Structure Analysis of audio signals. GRETSI2022.



$$\mathcal{X} = \mathcal{G} \times_1 W \times_2 H \times_3 Q$$

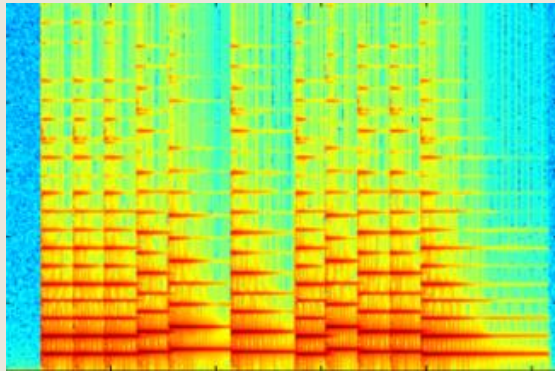
Nonnegative Matrix Factorization (NMF)



$$X \approx WV$$

NMF for Music Transcription [SB03]

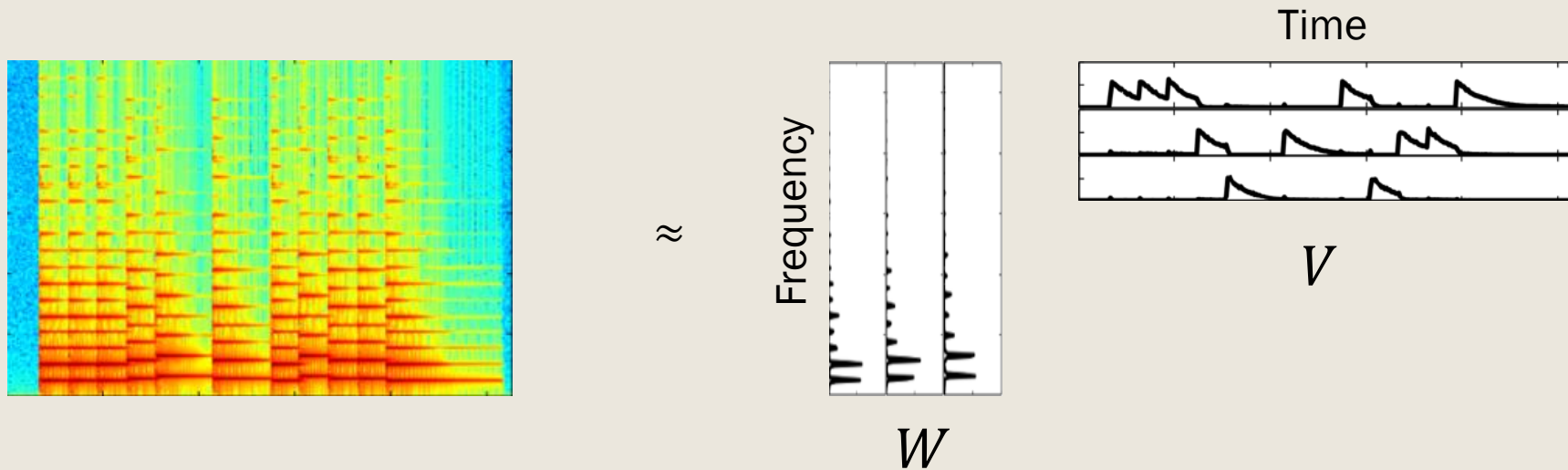
Example adapted from [Wu+22]



$$X \approx WV$$

NMF for Music Transcription [SB03]

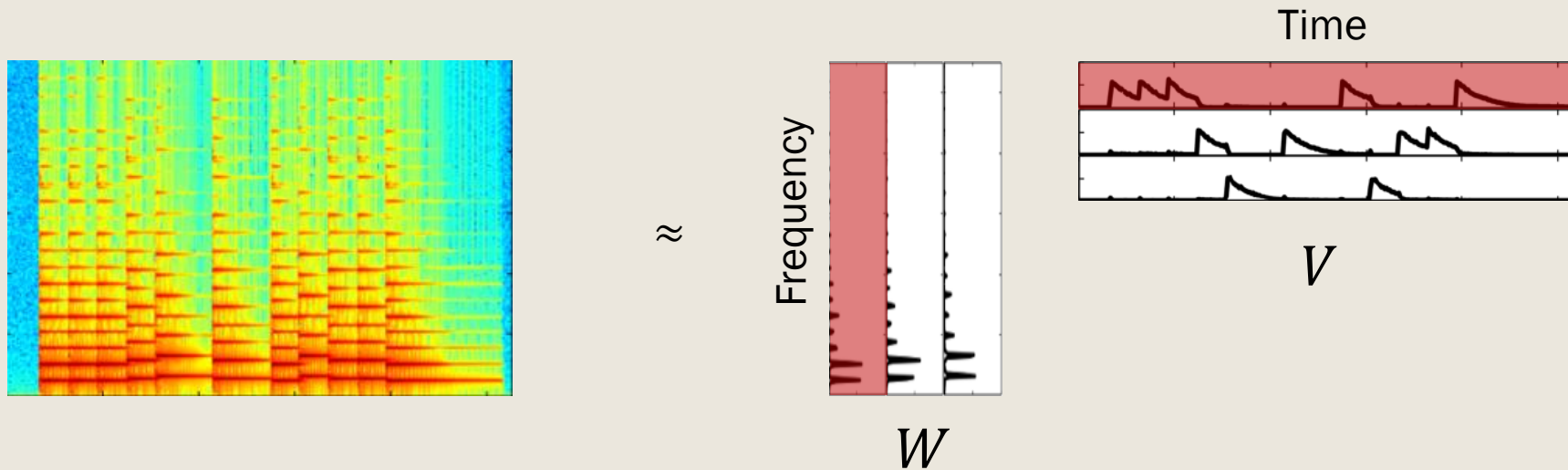
Example adapted from [Wu+22]



$$X \approx WV$$

NMF for Music Transcription [SB03]

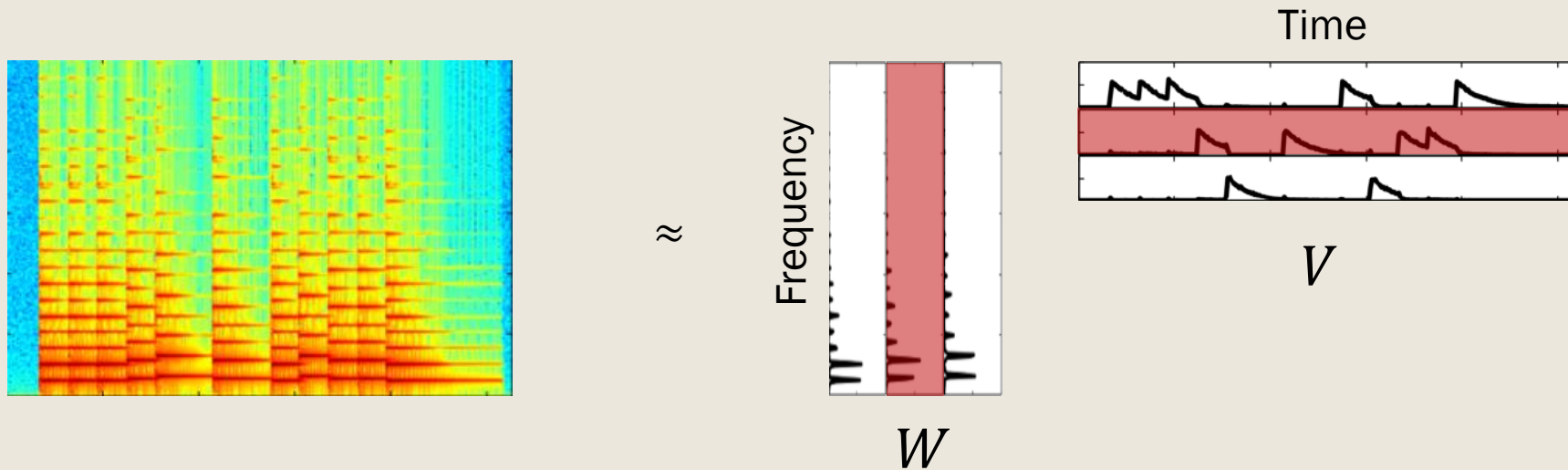
Example adapted from [Wu+22]



$$X \approx WV$$

NMF for Music Transcription [SB03]

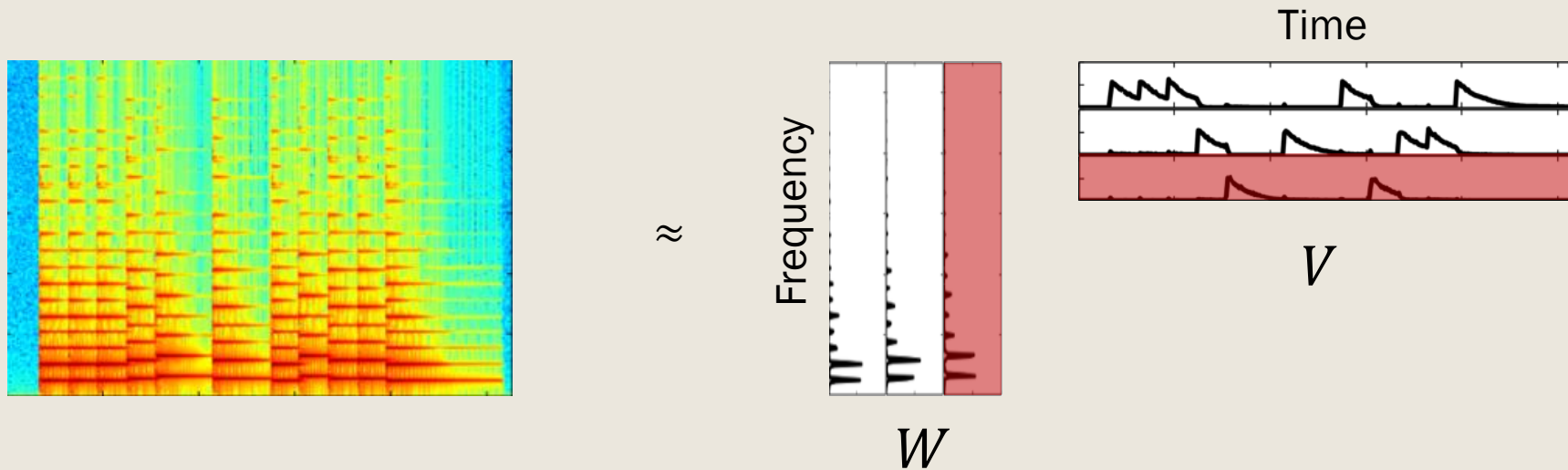
Example adapted from [Wu+22]



$$X \approx WV$$

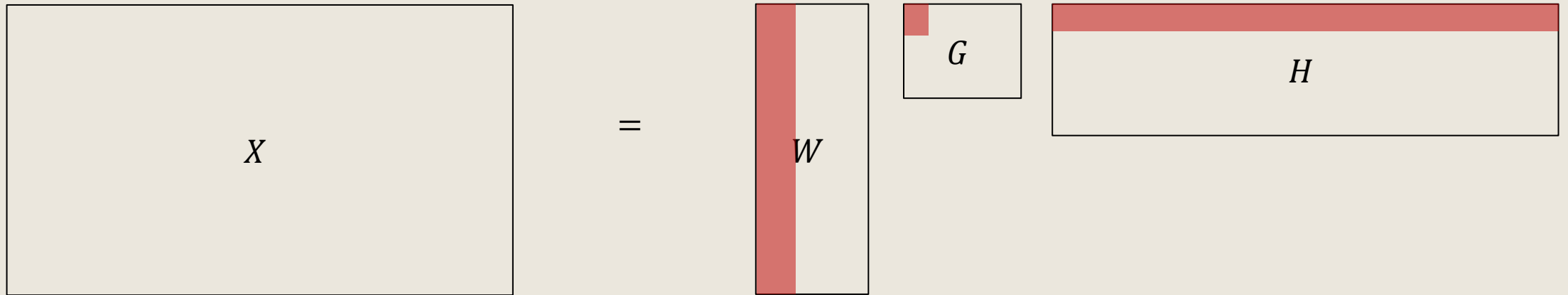
NMF for Music Transcription [SB03]

Example adapted from [Wu+22]



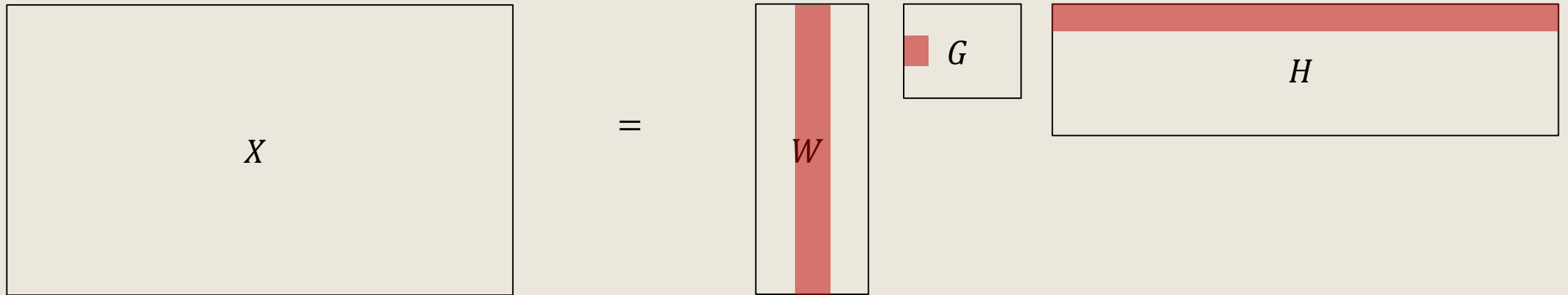
$$X \approx WV$$

Mixing templates



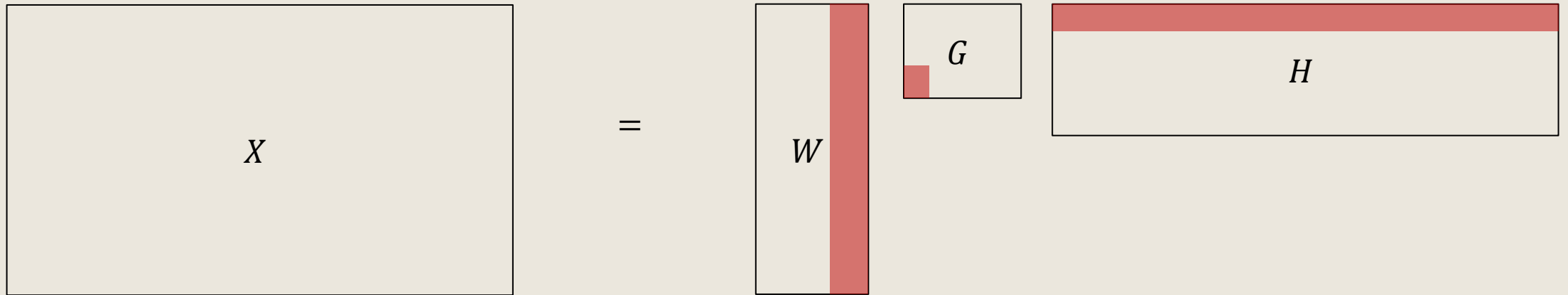
$$X = WGH$$

Mixing templates



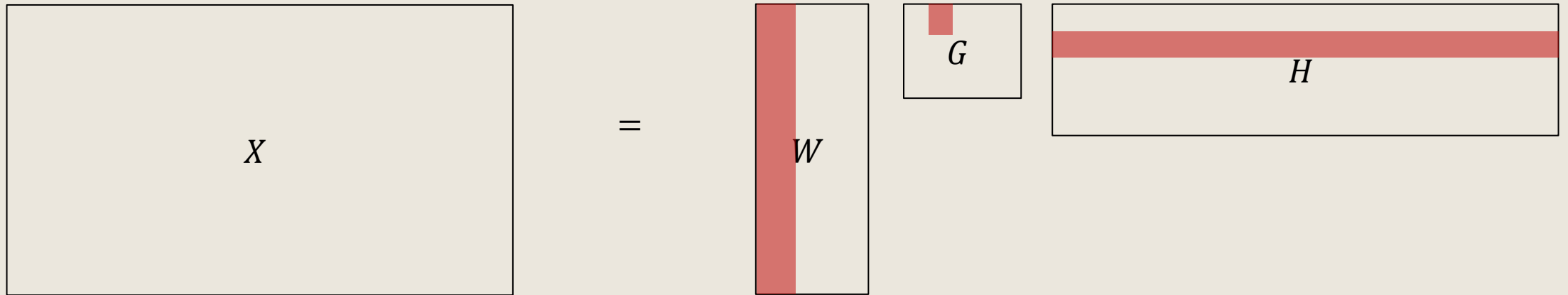
$$X = WGH$$

Mixing templates



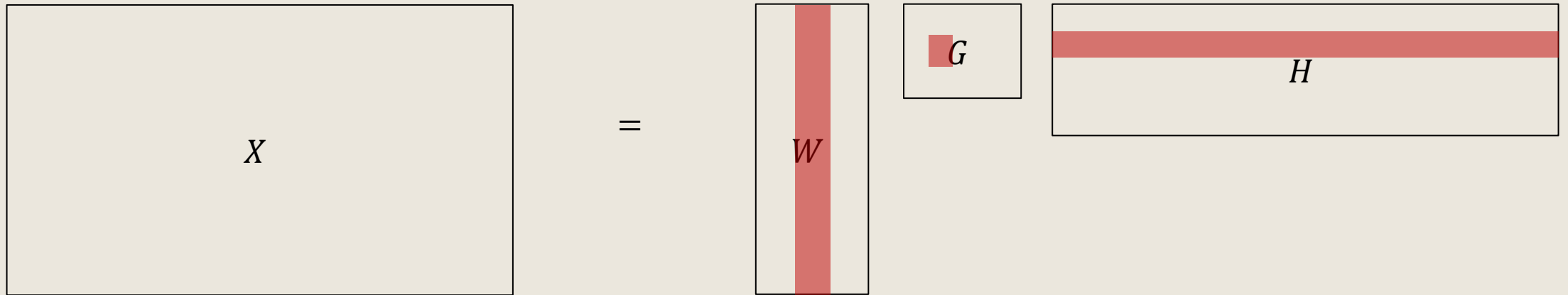
$$X = WGH$$

Mixing templates



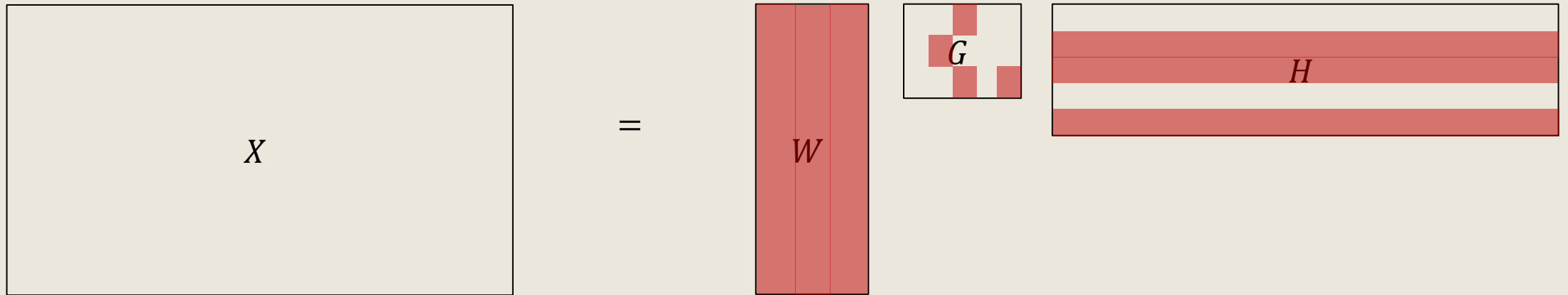
$$X = WGH$$

Mixing templates



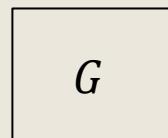
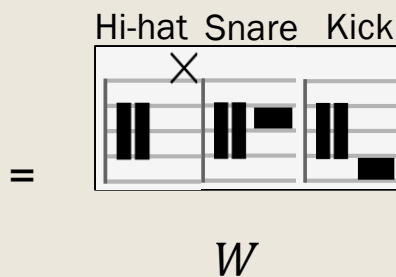
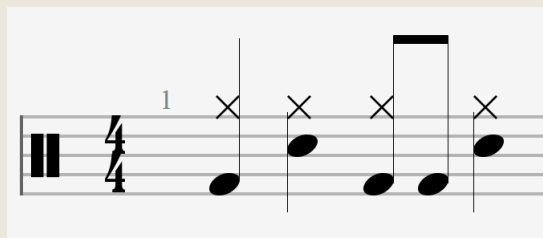
$$X = WGH$$

Mixing templates



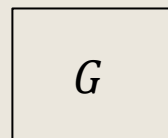
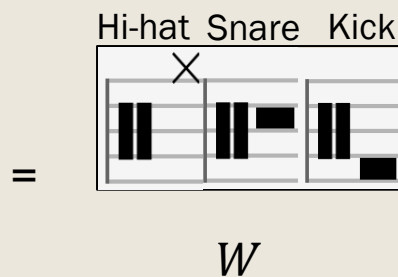
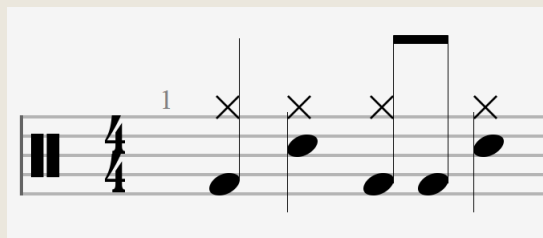
$$X = WGH$$

Barwise musical pattern: WGH



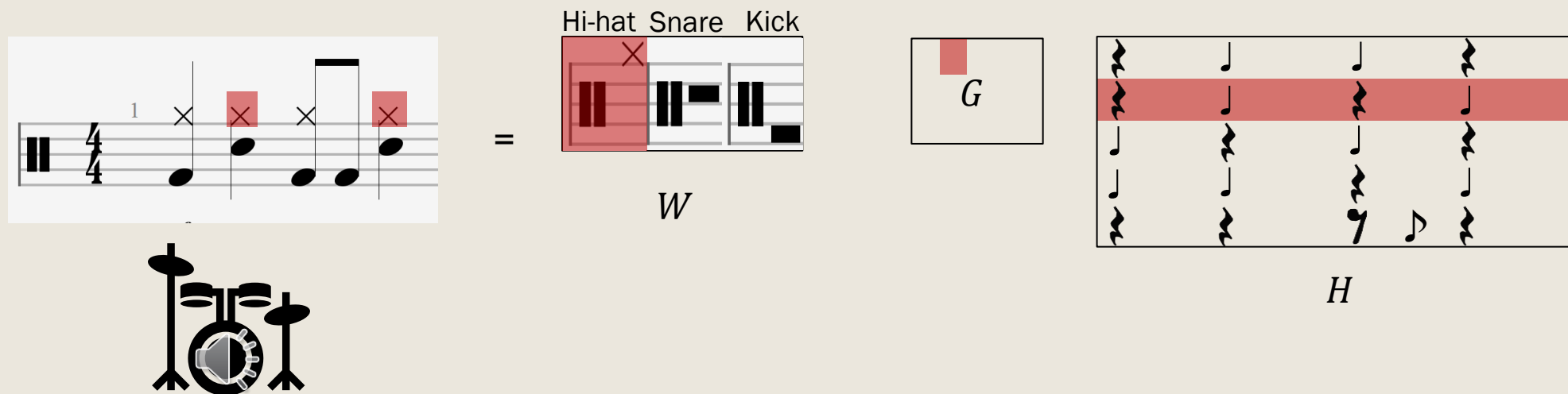
H

Barwise musical pattern: WGH

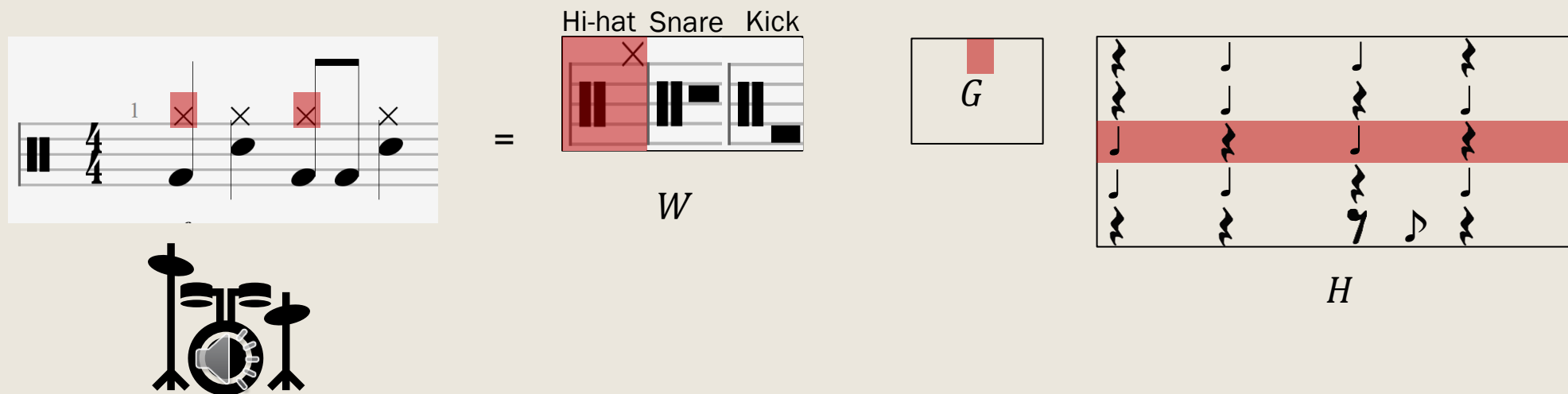


H

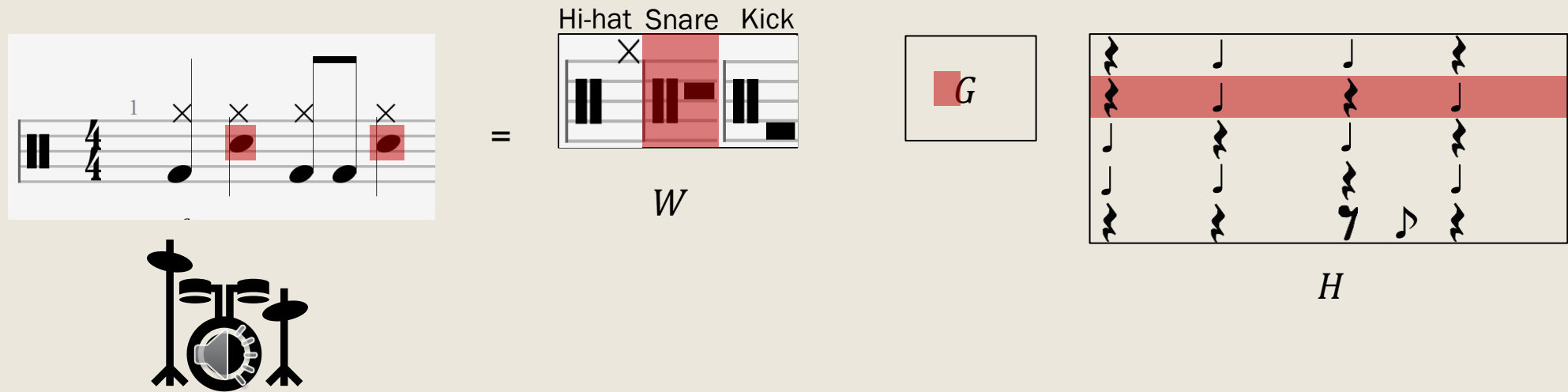
Barwise musical pattern: WGH



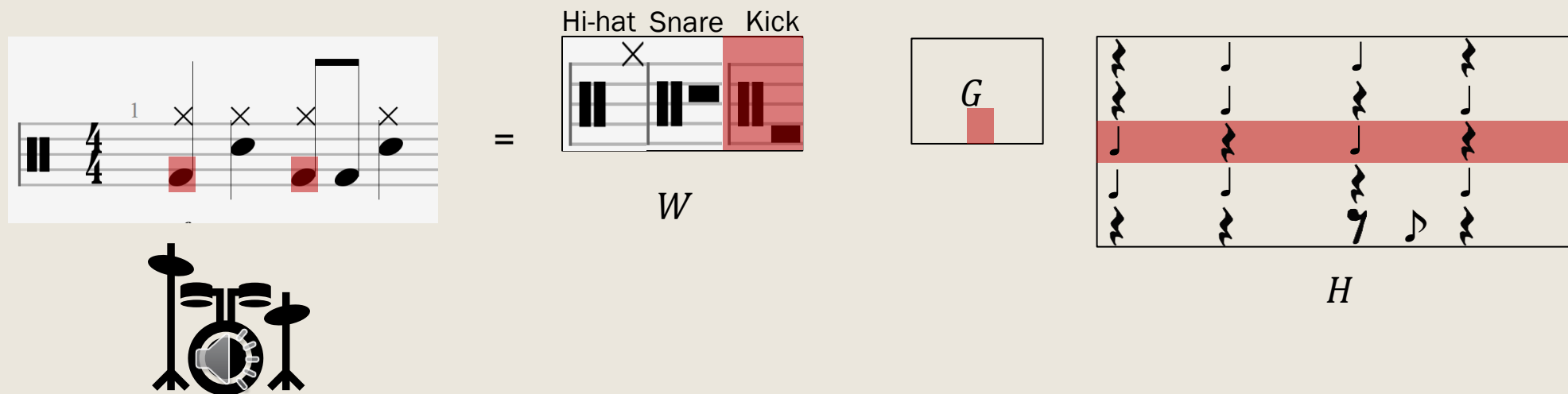
Barwise musical pattern: WGH



Barwise musical pattern: WGH



Barwise musical pattern: WGH

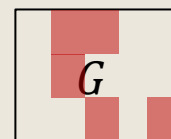
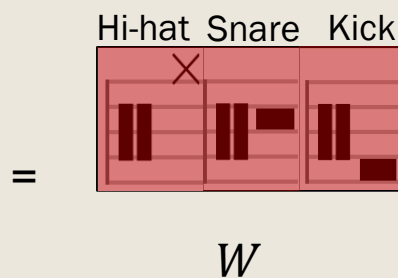
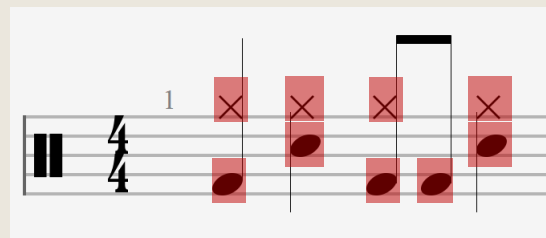


Barwise musical pattern: WGH

The diagram illustrates the decomposition of a musical score into three components: W (Drum), G (Bass), and H (Melody).

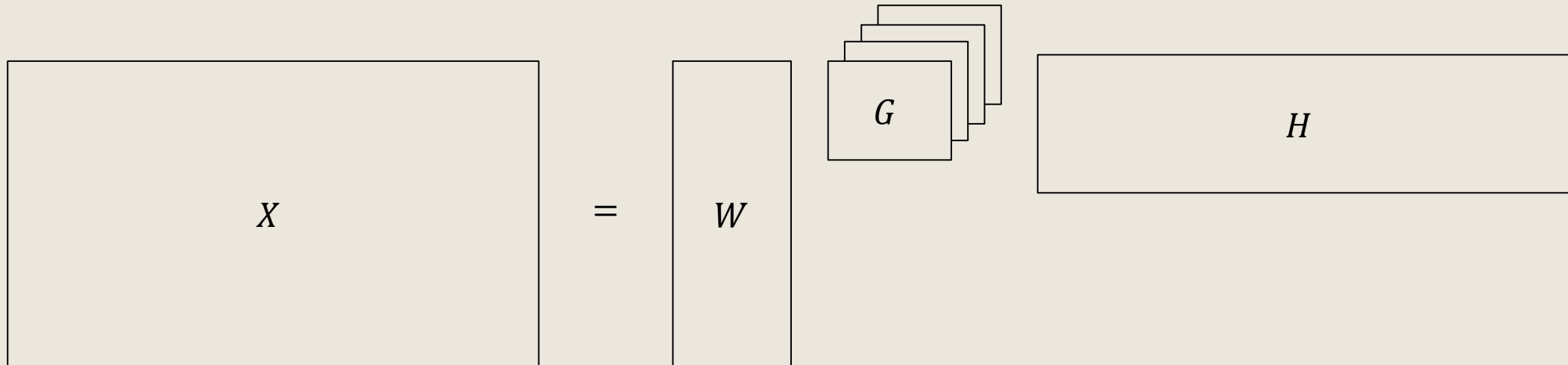
- Top Left:** A musical score snippet in 4/4 time. The melody consists of quarter notes: C4, D4, E4, F4, G4. The drum part includes a kick drum on the first beat, a snare drum on the second beat, and a hi-hat on the third beat. The bass line is a simple eighth-note pattern: C4, D4, E4, F4, G4.
- Top Middle:** The score is decomposed into three parts: W (Drum), G (Bass), and H (Melody). The W part is a drum pattern with a kick drum on the first beat, a snare drum on the second beat, and a hi-hat on the third beat. The G part is a bass line with a simple eighth-note pattern: C4, D4, E4, F4, G4. The H part is a melody with a simple eighth-note pattern: C4, D4, E4, F4, G4.
- Top Right:** A diagram showing the decomposition of the score into three parts: W (Drum), G (Bass), and H (Melody). The W part is a drum pattern with a kick drum on the first beat, a snare drum on the second beat, and a hi-hat on the third beat. The G part is a bass line with a simple eighth-note pattern: C4, D4, E4, F4, G4. The H part is a melody with a simple eighth-note pattern: C4, D4, E4, F4, G4.
- Bottom Left:** A diagram showing the decomposition of the score into three parts: W (Drum), G (Bass), and H (Melody). The W part is a drum pattern with a kick drum on the first beat, a snare drum on the second beat, and a hi-hat on the third beat. The G part is a bass line with a simple eighth-note pattern: C4, D4, E4, F4, G4. The H part is a melody with a simple eighth-note pattern: C4, D4, E4, F4, G4.

Barwise musical pattern: WGH

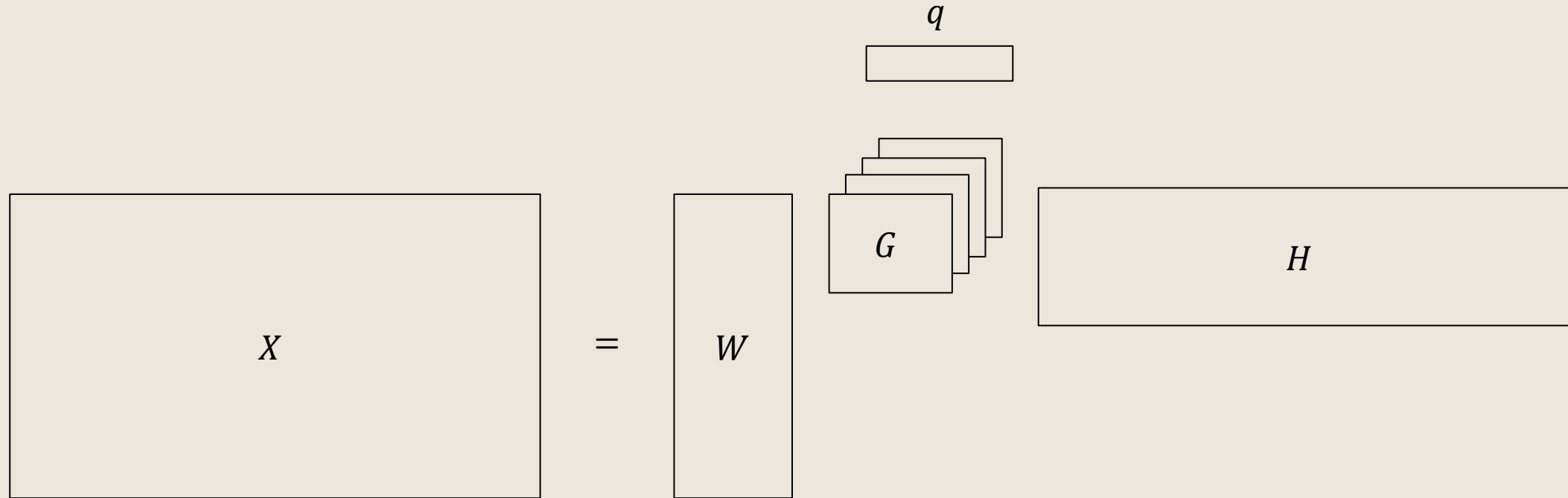


H

Combining patterns

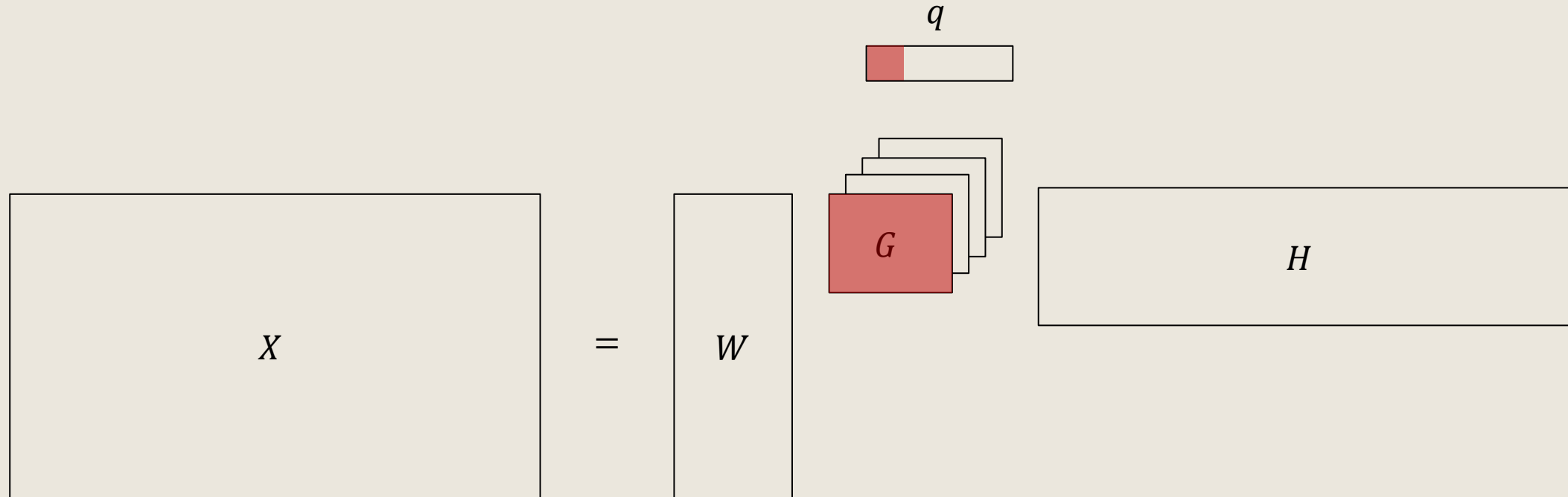


Combining patterns



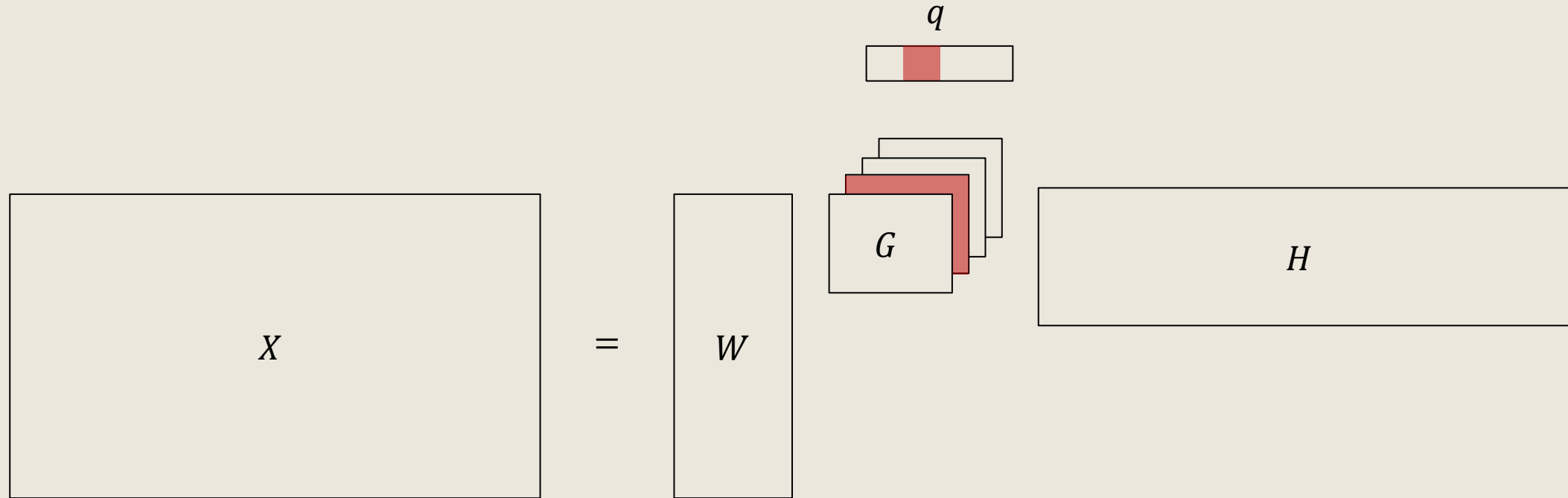
$$X = W \left(\sum_i q_{[i]} G_{[:,i]} \right) H$$

Combining patterns



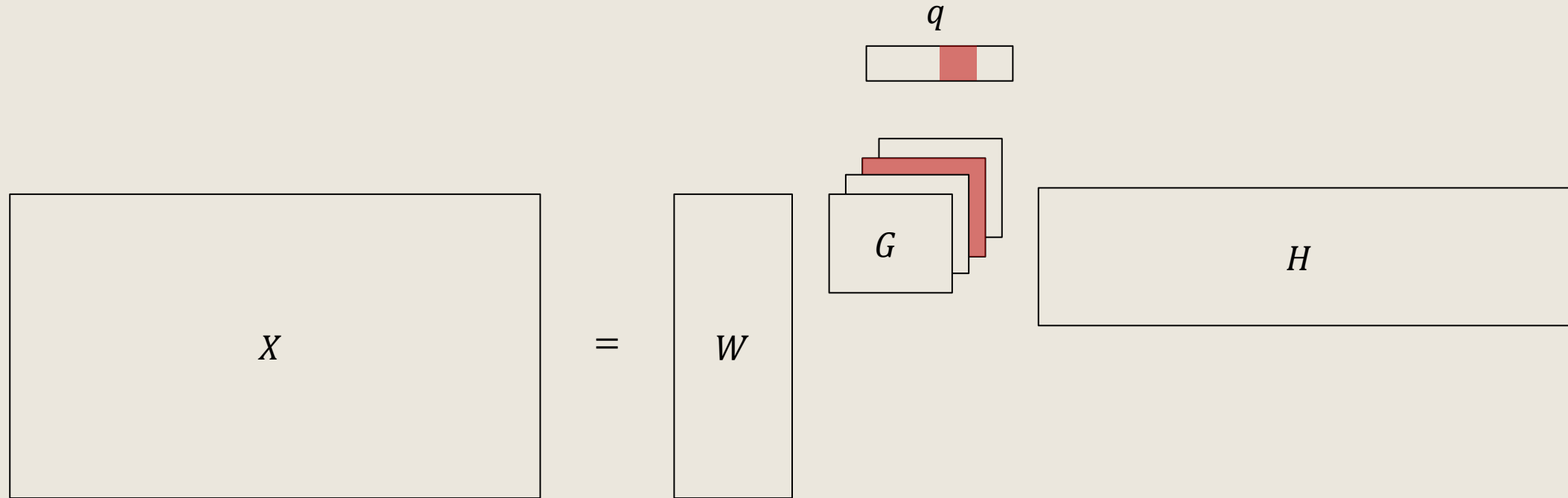
$$X = W \left(\sum_i q_{[i]} G_{[:,i]} \right) H$$

Combining patterns



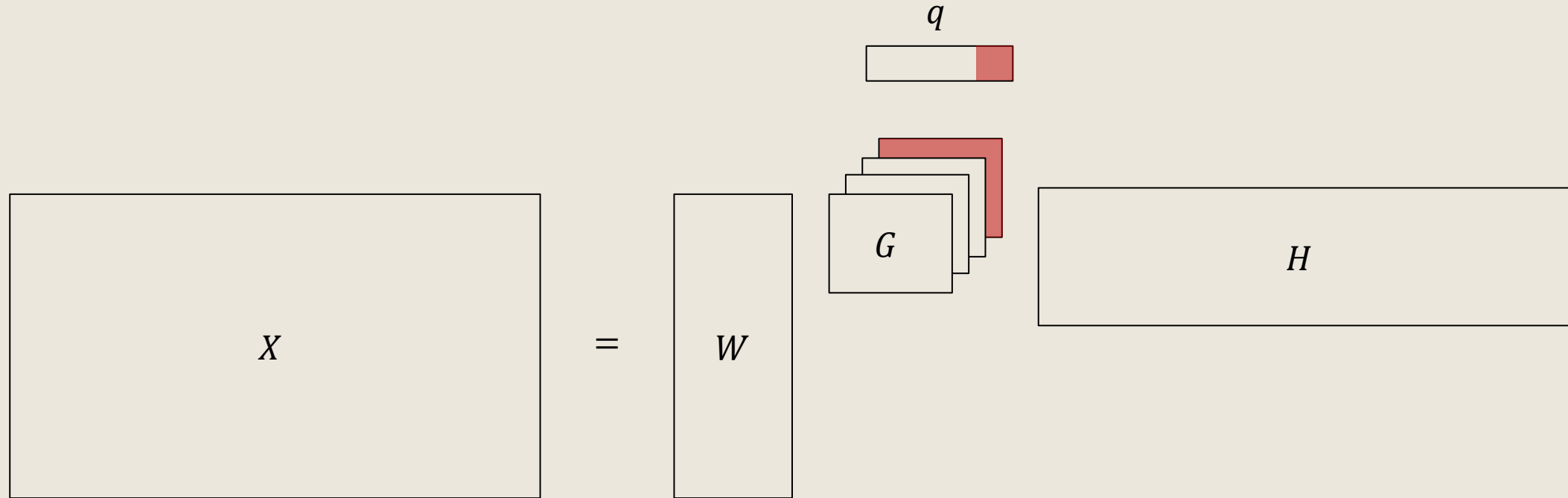
$$X = W \left(\sum_i q_{[i]} G_{[:,i]} \right) H$$

Combining patterns



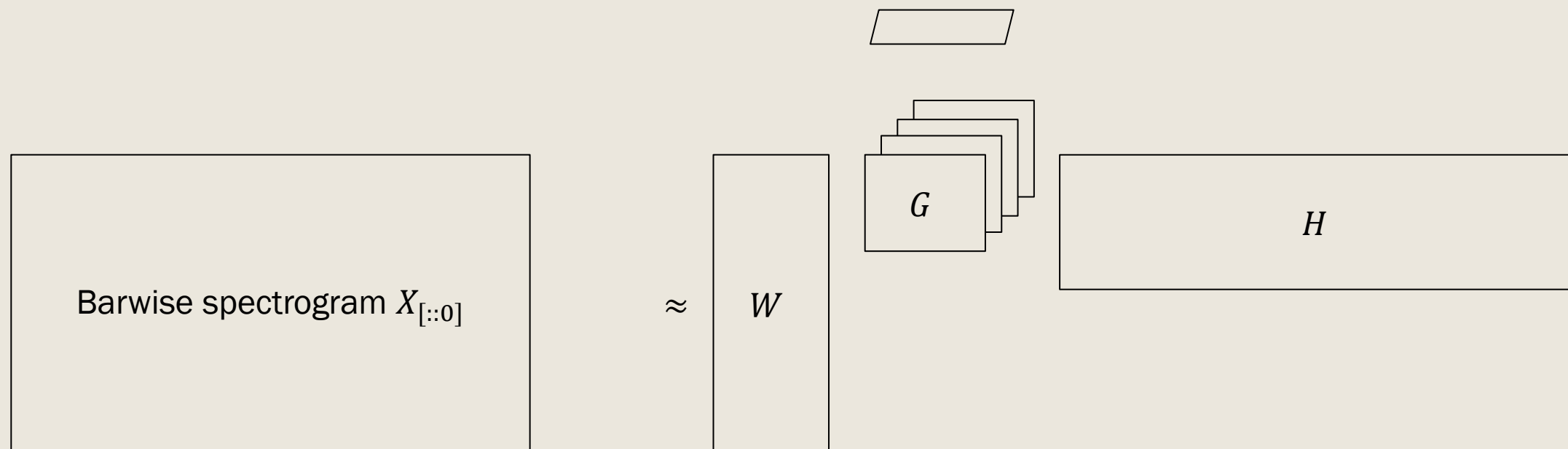
$$X = W \left(\sum_i q_{[i]} G_{[:,i]} \right) H$$

Combining patterns

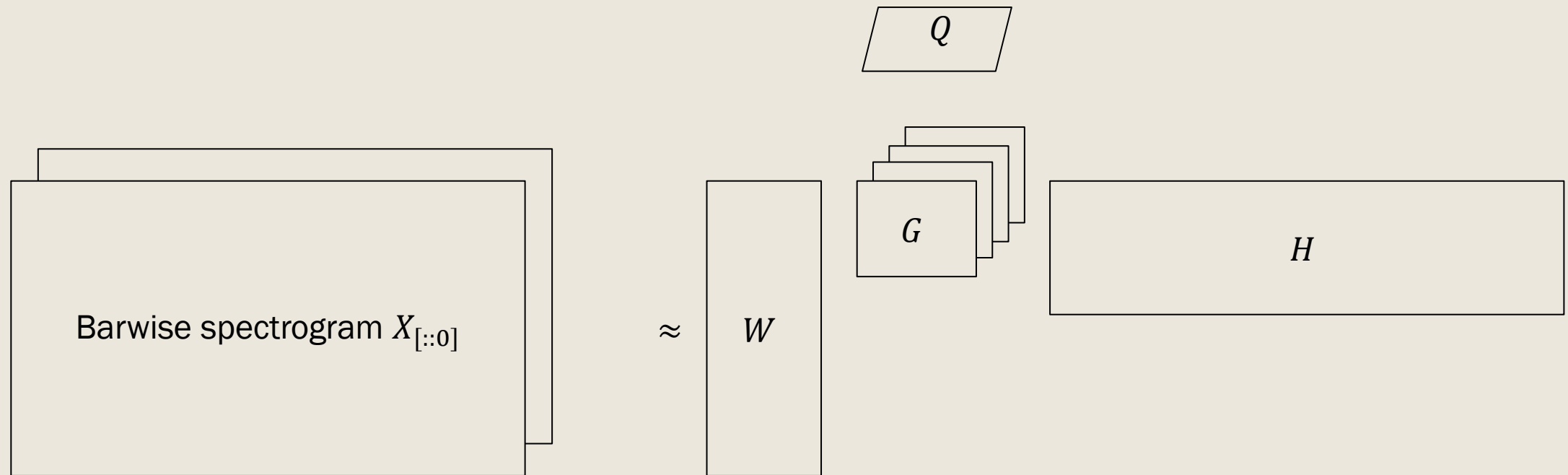


$$X = W \left(\sum_i q_{[i]} G_{[:,i]} \right) H$$

Bars, expressed with patterns

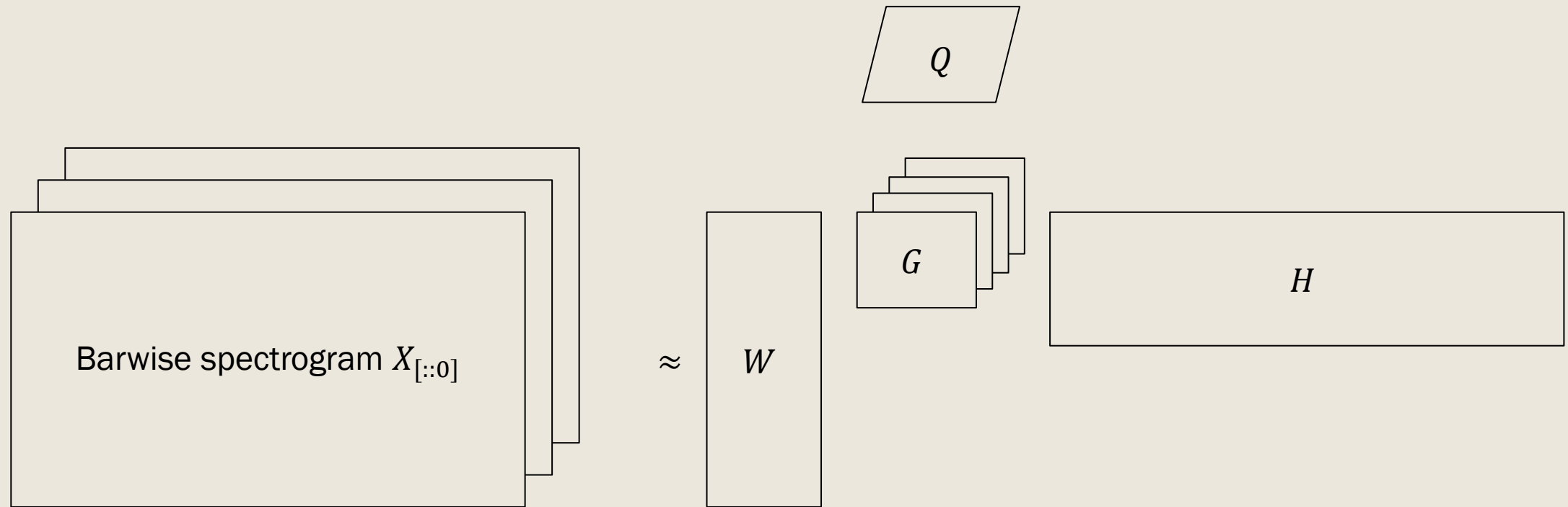


Bars, expressed with patterns



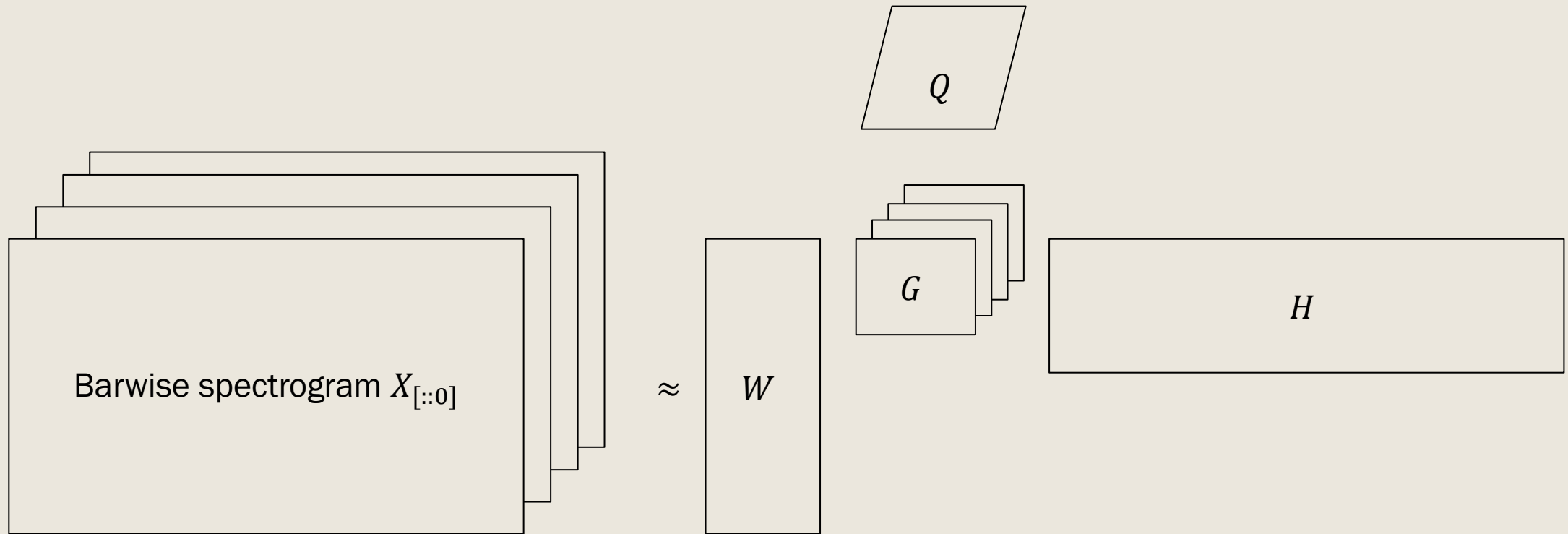
$$X_{[:,k]} = W \left(\sum_i Q_{[ki]} G_{[:,i]} \right) H$$

Bars, expressed with patterns



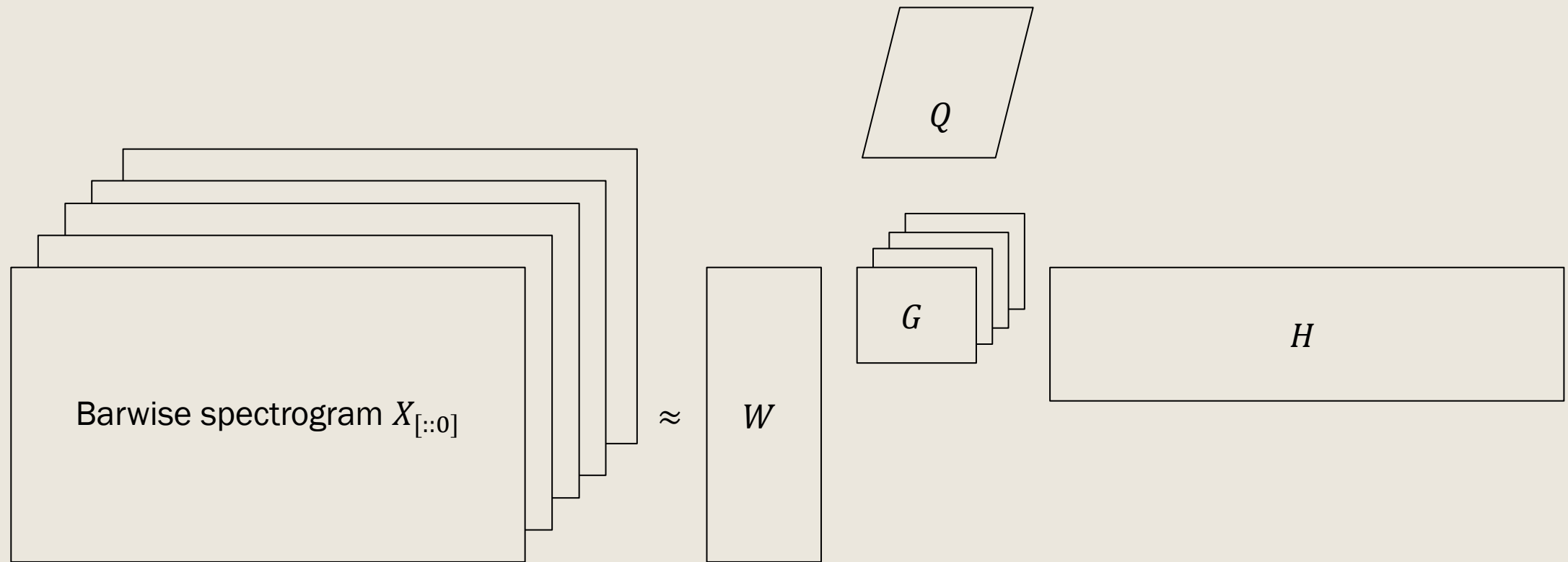
$$X_{[:,k]} = W \left(\sum_i Q_{[ki]} G_{[:,i]} \right) H$$

Bars, expressed with patterns



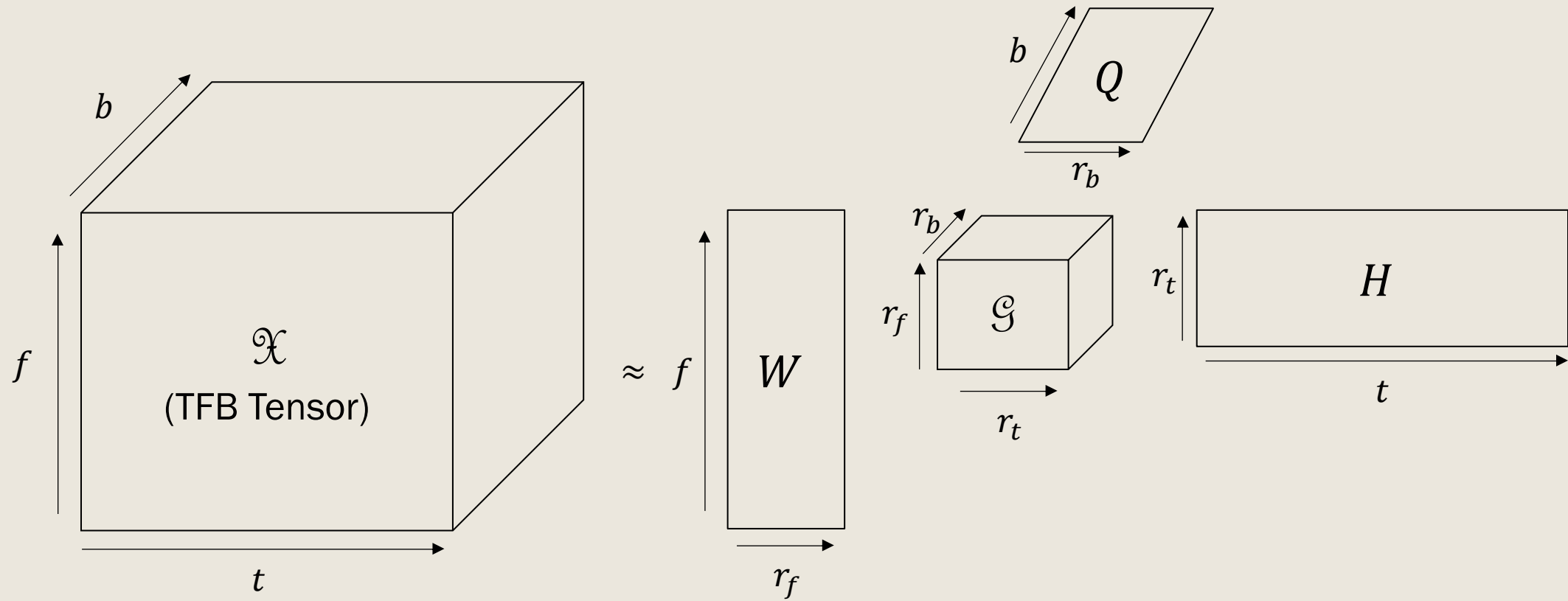
$$X_{[:,k]} = W \left(\sum_i Q_{[ki]} G_{[:,i]} \right) H$$

Bars, expressed with patterns



$$X_{[:,k]} = W \left(\sum_i Q_{[ki]} G_{[:,i]} \right) H$$

NTD: Nonnegative Tucker Decomposition

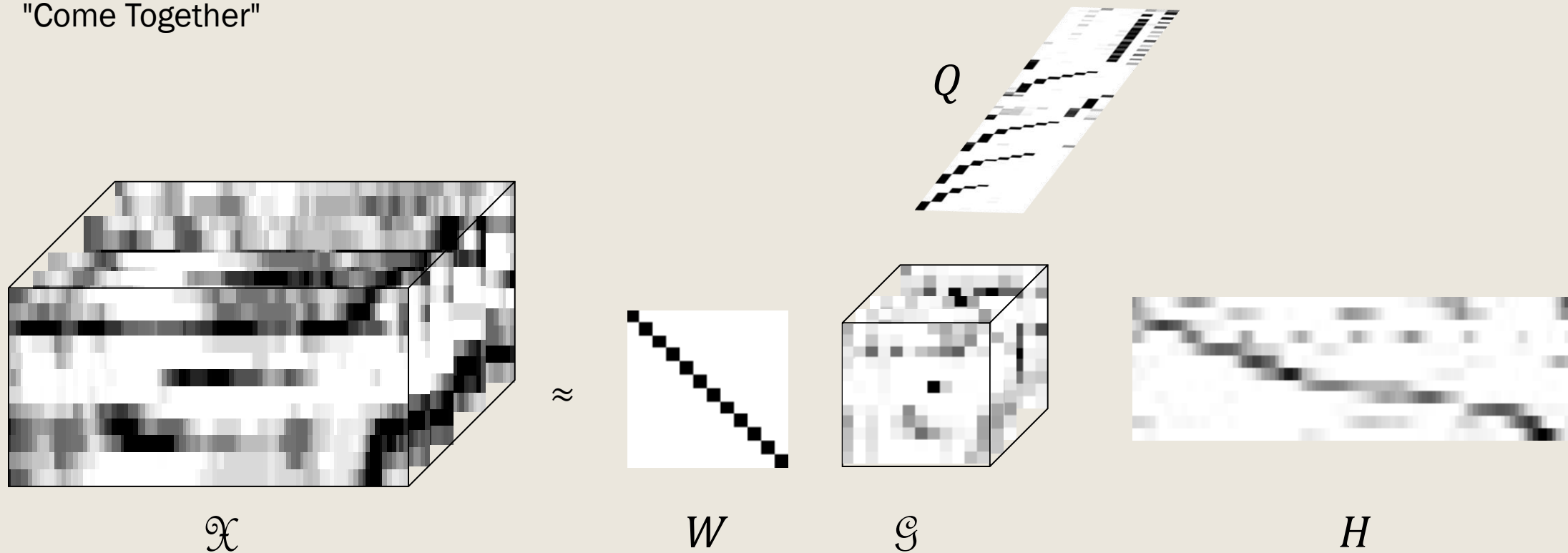


$$\mathcal{X} = \mathcal{G} \times_1 W \times_2 H \times_3 Q$$

- All factors are nonnegative, r_f, r_t, r_b are hyperparameters (dimensions: order of magnitude of 20)

NTD in practice: chromagram

"Come Together"



NTD computation

$$\operatorname{argmin}_{\mathcal{G} \geq 0, W \geq 0, H \geq 0, Q \geq 0} d(\mathcal{X}, \mathcal{G} \times_1 W \times_2 H \times_3 Q)$$

■ Euclidean norm [Mar+20]:

- Solved with HALS [GG12]

$$d_{\text{euc}}(x|y) = \|x - y\|_2^2$$

■ β -divergence [Mar+22]:

- Solved with Multiplicative Updates [FI11]

$$d_{\beta}(x|y) = \begin{cases} \frac{x}{y} - \log\left(\frac{x}{y}\right) - 1 & \beta = 0 \\ & \text{(Itakura - Saïto)} \\ x \log\left(\frac{x}{y}\right) + (y - x) & \beta = 1 \\ & \text{(Kullback - Leibler)} \\ \frac{x^{\beta} + (\beta - 1)y^{\beta} - \beta xy^{\beta-1}}{\beta(\beta - 1)} & \beta \in \mathbb{R} \setminus \{0, 1\} \end{cases}$$

[FI11] Févotte, C. and Idier, J., « Algorithms for nonnegative matrix factorization with the β -divergence », in: *Neural computation* 23.9 (2011), pp. 2421–2456.

[GG12] Gillis, N. and Glineur, F., « Accelerated multiplicative updates and hierarchical ALS algorithms for nonnegative matrix factorization », in: *Neural computation* 24.4 (2012), pp. 1085–1105.

[Mar+20] Marmoret, A., Cohen, J., Bertin, N., & Bimbot, F. Uncovering Audio Patterns in Music with Nonnegative Tucker Decomposition for Structural Segmentation. In 2020 ISMIR.

[Mar+22] Marmoret, A., et al.. Nonnegative Tucker Decomposition with Beta-divergence for Music Structure Analysis of audio signals. GRETSI2022.

NTD computation

$$\operatorname{argmin}_{\mathcal{G} \geq 0, W \geq 0, H \geq 0, Q \geq 0} d(\mathcal{X}, \mathcal{G} \times_1 W \times_2 H \times_3 Q)$$

■ Euclidean norm [Mar+20]:

- Solved with HALS [GG12]

$$d_{\text{euc}}(x|y) = \|x - y\|_2^2$$

■ β -divergence [Mar+22]:

- Solved with Multiplicative Updates [FI11]

$$d_{\beta}(x|y) = \begin{cases} \frac{x}{y} - \log\left(\frac{x}{y}\right) - 1 & \beta = 0 \\ & (\text{Itakura - Saïto}) \\ x \log\left(\frac{x}{y}\right) + (y - x) & \beta = 1 \\ & (\text{Kullback - Leibler}) \\ \frac{x^{\beta} + (\beta - 1)y^{\beta} - \beta xy^{\beta-1}}{\beta(\beta - 1)} & \beta \in \mathbb{R} \setminus \{0, 1\} \end{cases}$$

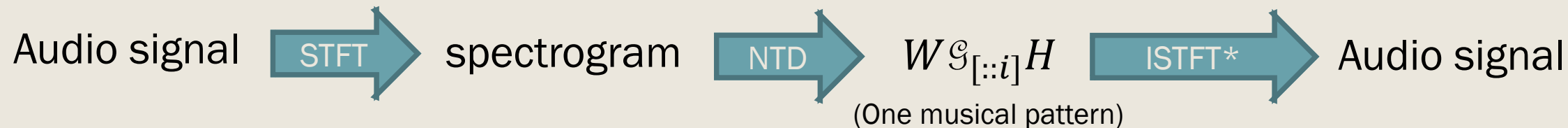
[FI11] Févotte, C. and Idier, J., « Algorithms for nonnegative matrix factorization with the β -divergence », in: *Neural computation* 23.9 (2011), pp. 2421–2456.

[GG12] Gillis, N. and Glineur, F., « Accelerated multiplicative updates and hierarchical ALS algorithms for nonnegative matrix factorization », in: *Neural computation* 24.4 (2012), pp. 1085–1105.

[Mar+20] Marmoret, A., Cohen, J., Bertin, N., & Bimbot, F. Uncovering Audio Patterns in Music with Nonnegative Tucker Decomposition for Structural Segmentation. In 2020 ISMIR.

[Mar+22] Marmoret, A., et al.. Nonnegative Tucker Decomposition with Beta-divergence for Music Structure Analysis of audio signals. GRETSI2022.

Musical patterns $W^{\mathcal{G}}_{[:,i]}H?$



- 1st bar of Come Together



- Hand-picked pattern from NTD

- *Euclidean norm*



- *Kullback-Leibler divergence*



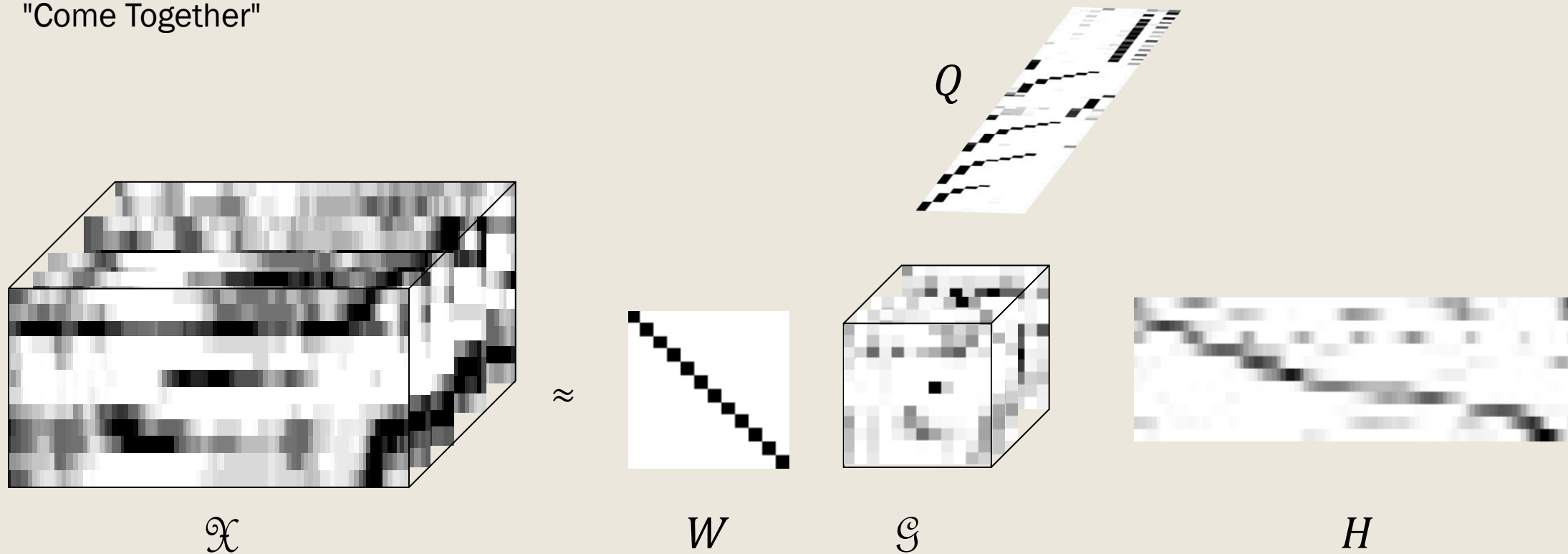
- *Itakura-Saito divergence*



*Phase information: from the original STFT

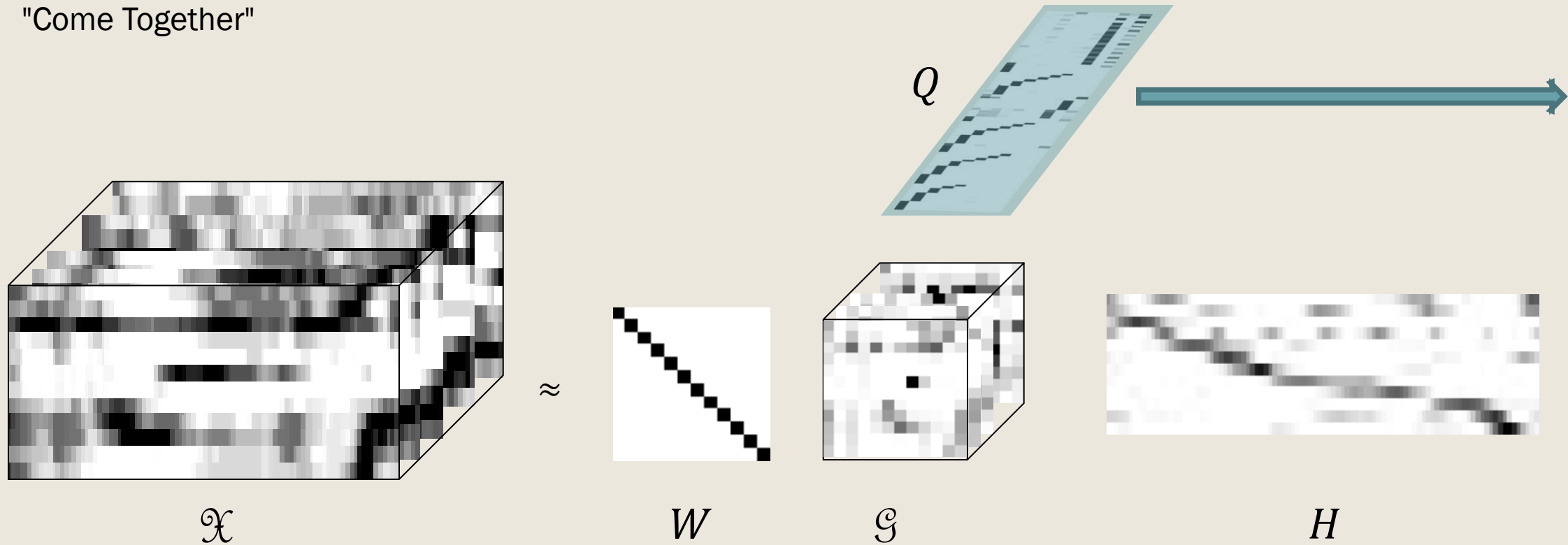
NTD in practice: chromagram

"Come Together"

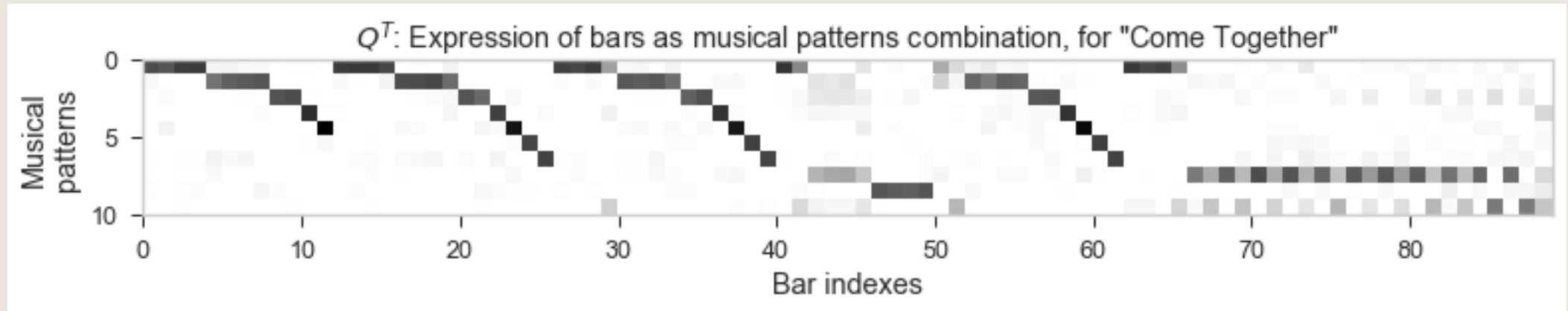


NTD in practice: chromagram

"Come Together"

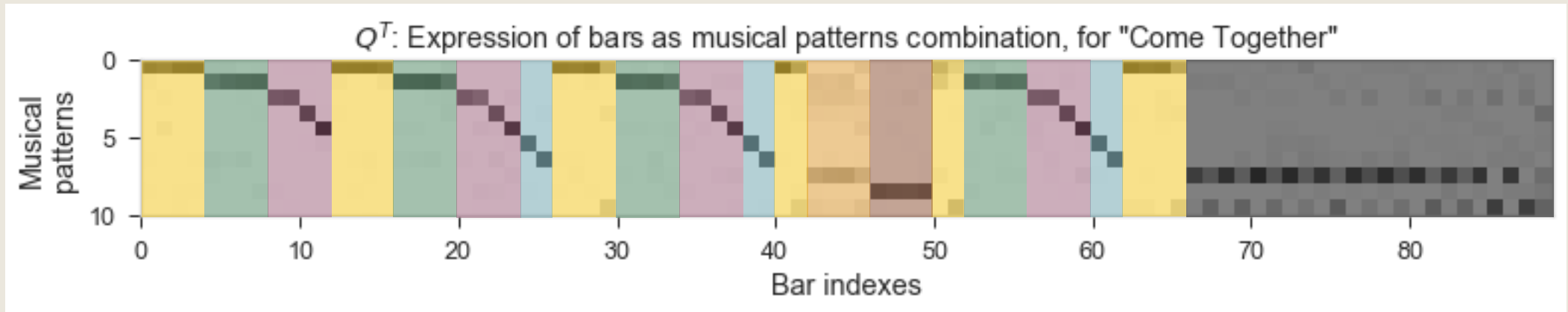


Q matrix



- Most bars appear as a sparse combination of musical patterns

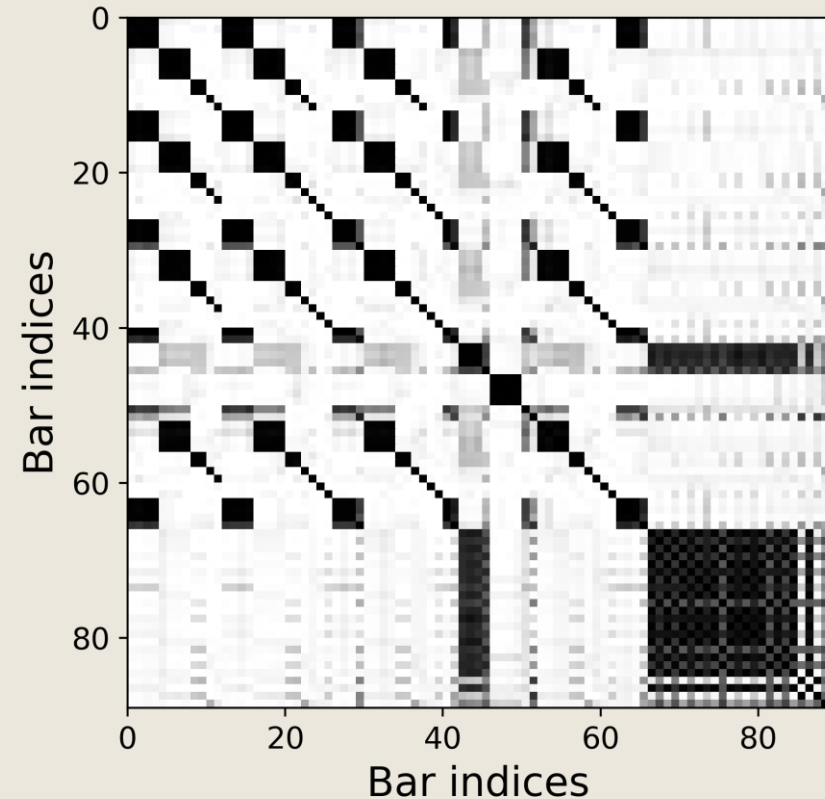
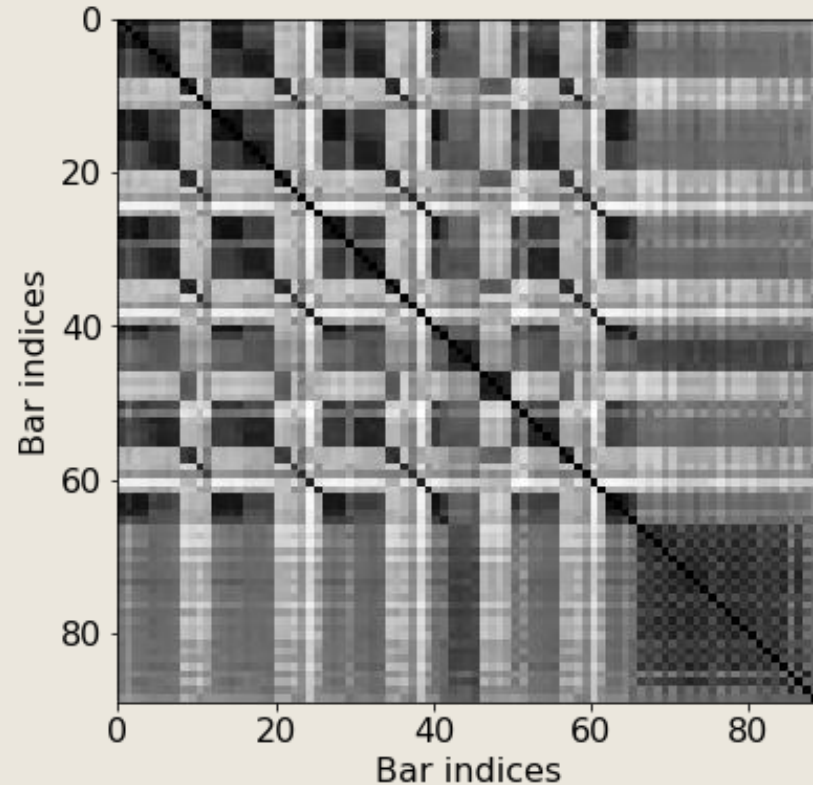
Q matrix



- Most bars appear as a sparse combination of musical patterns

Autosimilarities: Uncompressed vs NTD

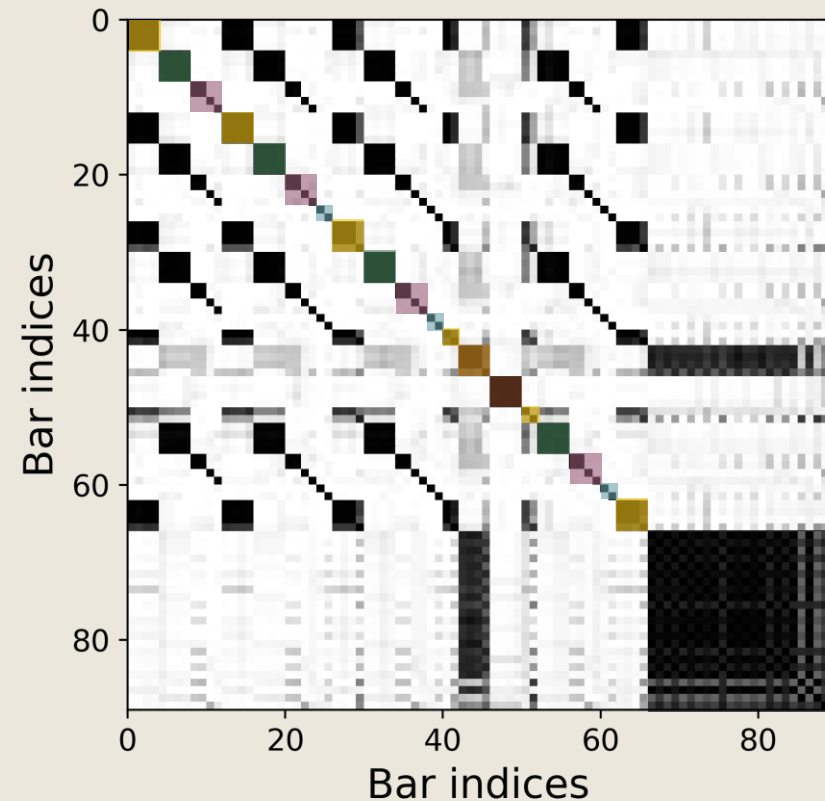
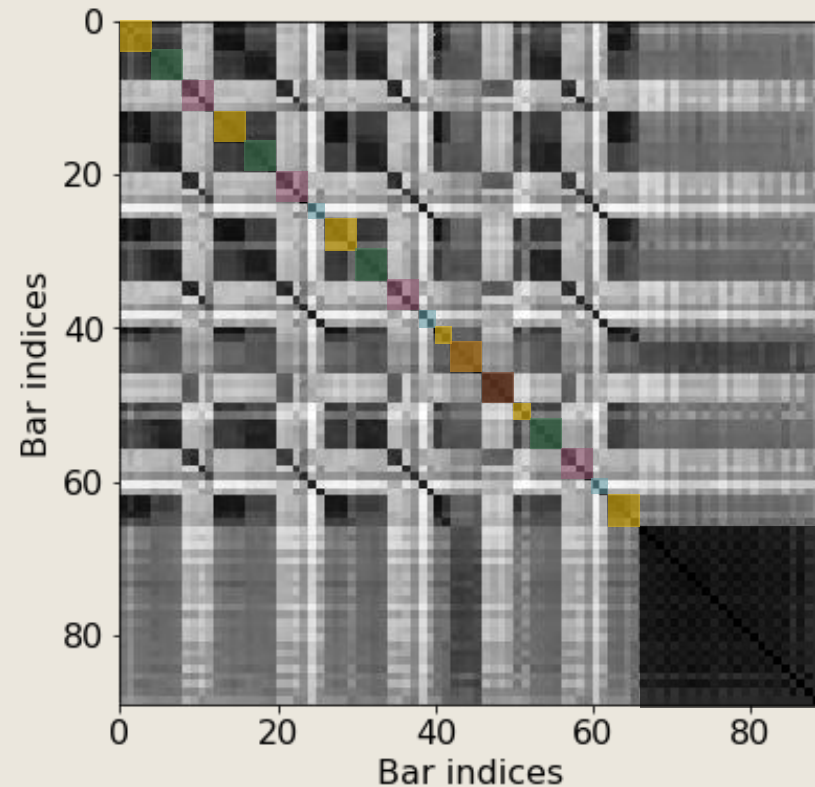
(Cosine)



➔ Higher contrast with NTD (as desired)

Autosimilarities: Uncompressed vs NTD

(Cosine)

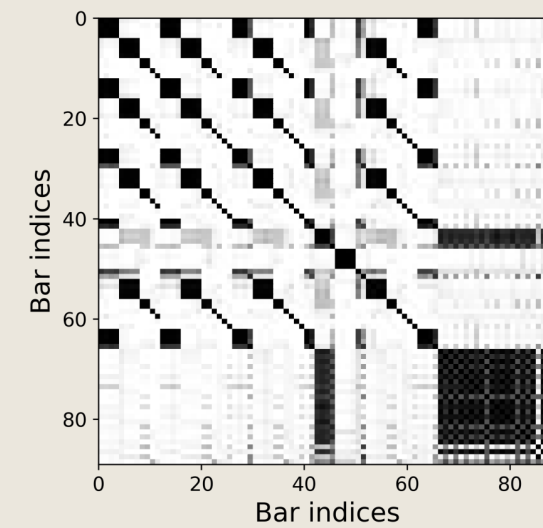
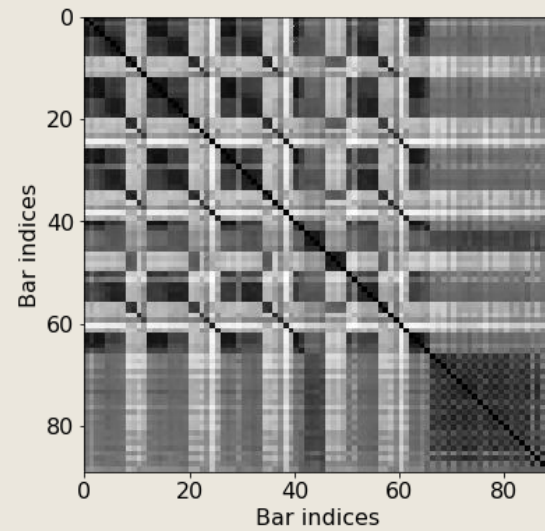


➔ Higher contrast with NTD (as desired)

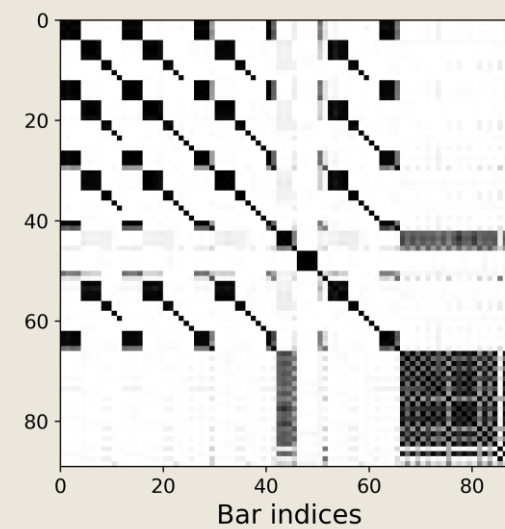
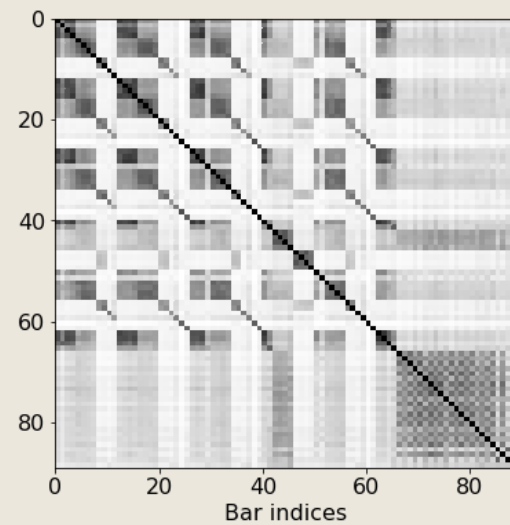
Uncompressed

NTD

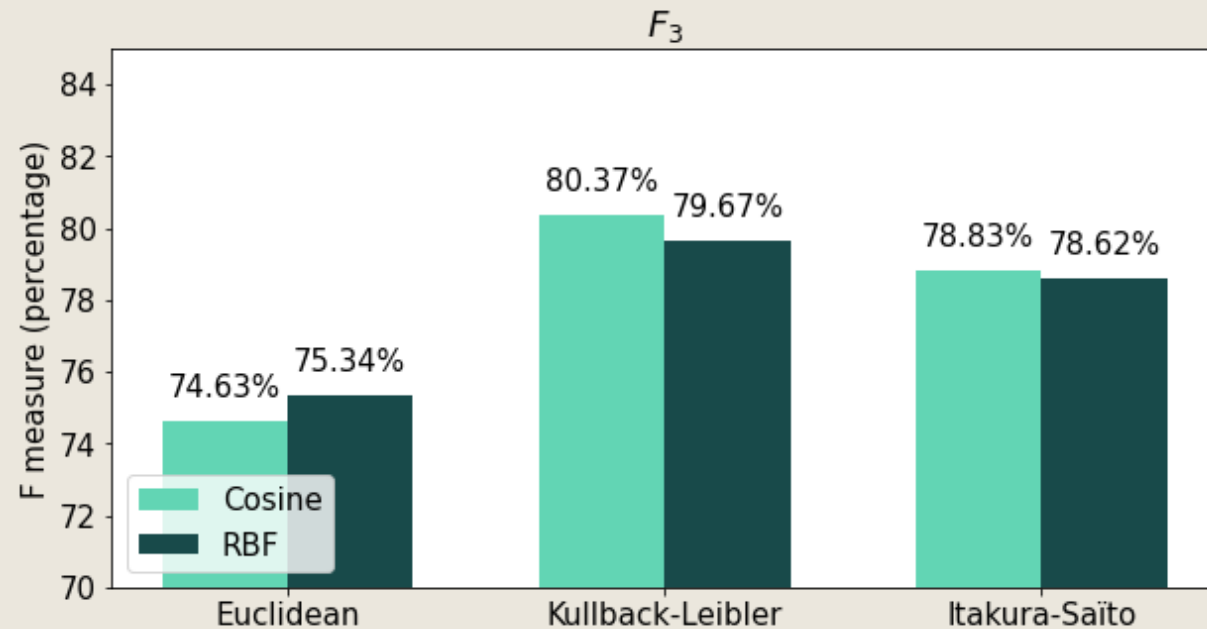
Cosine



RBF



NTD segmentation results



?

Similarity function

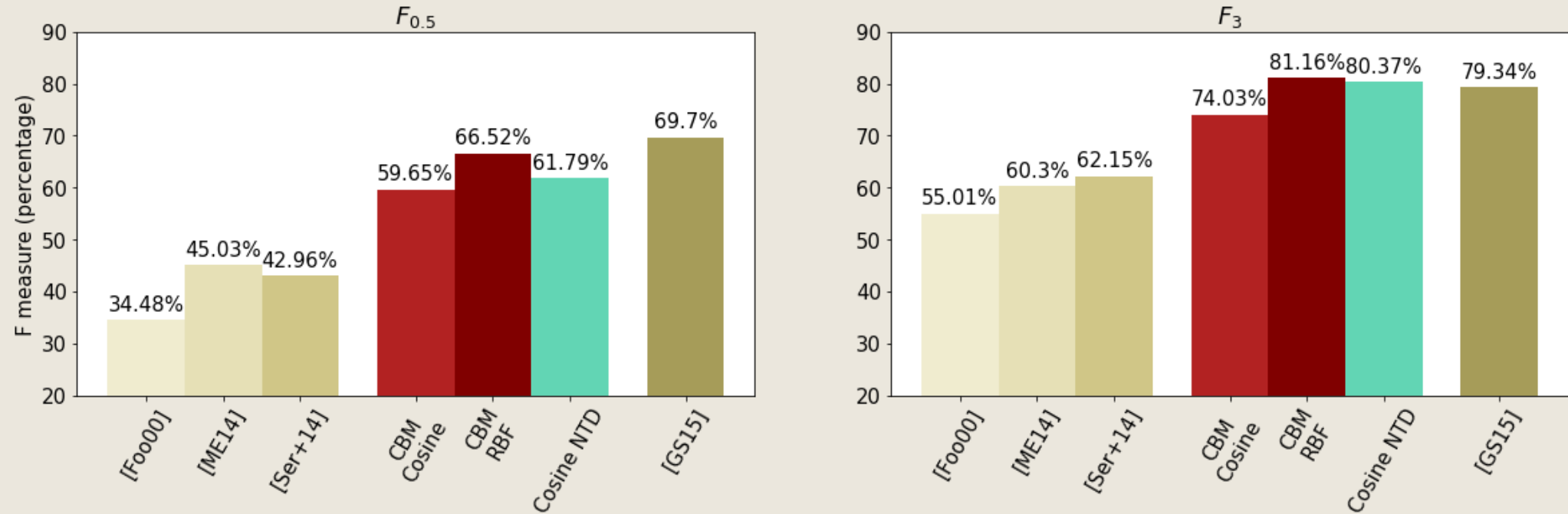
!

Objective function

– *Kullback-Leibler* ($\beta = 1$)

Segmentation results, according to parameters (RWC Pop dataset)

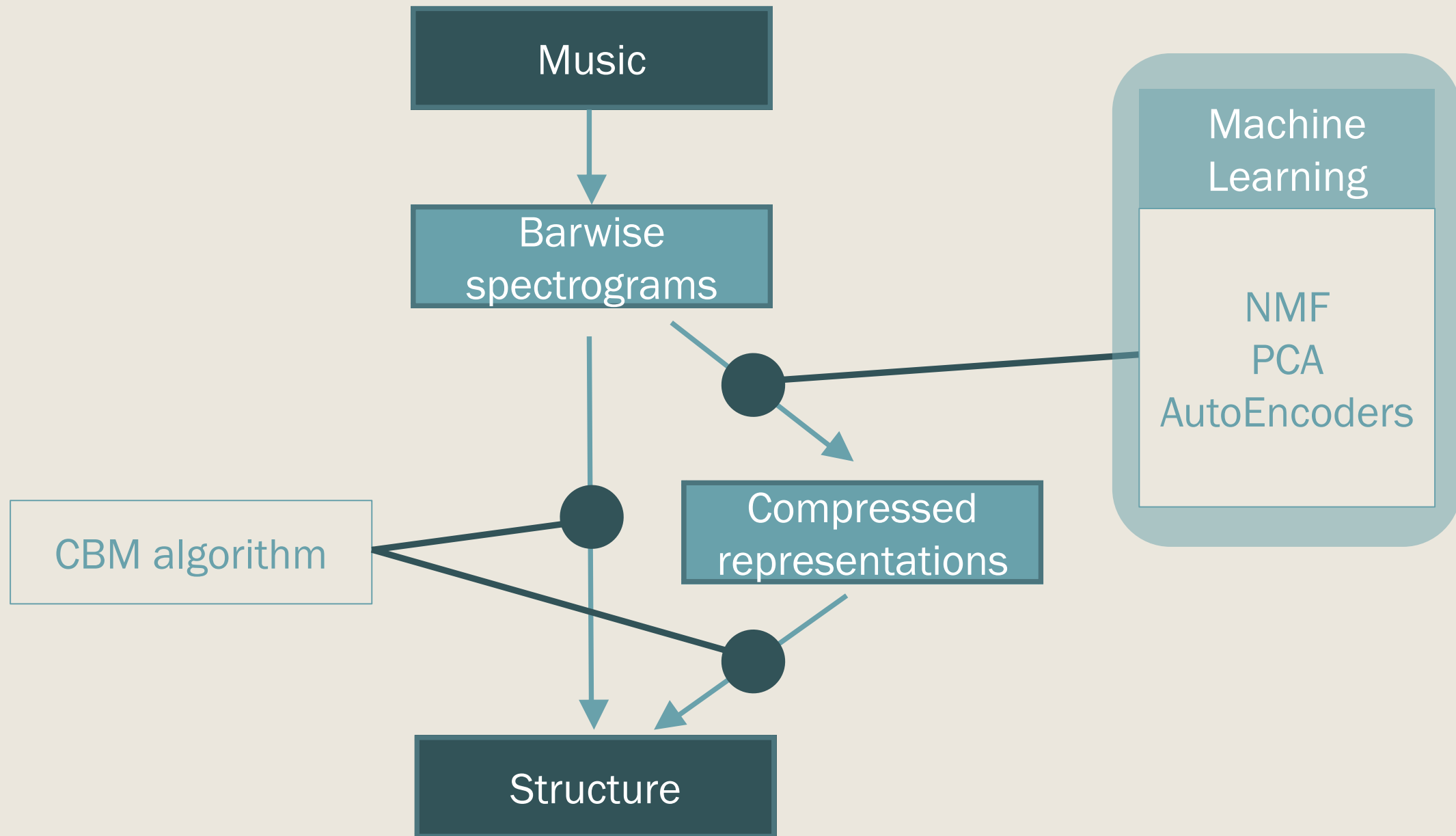
NTD segmentation results



Segmentation results, compared with State-of-the-art (RWC Pop dataset)

- Cosine: NTD improves uncompressed results

- RBF: NTD does not improve uncompressed results
 - *NTD dimensions are hard to fit*
 - *hyperparameters for CBM were fitted for uncompressed matrices*



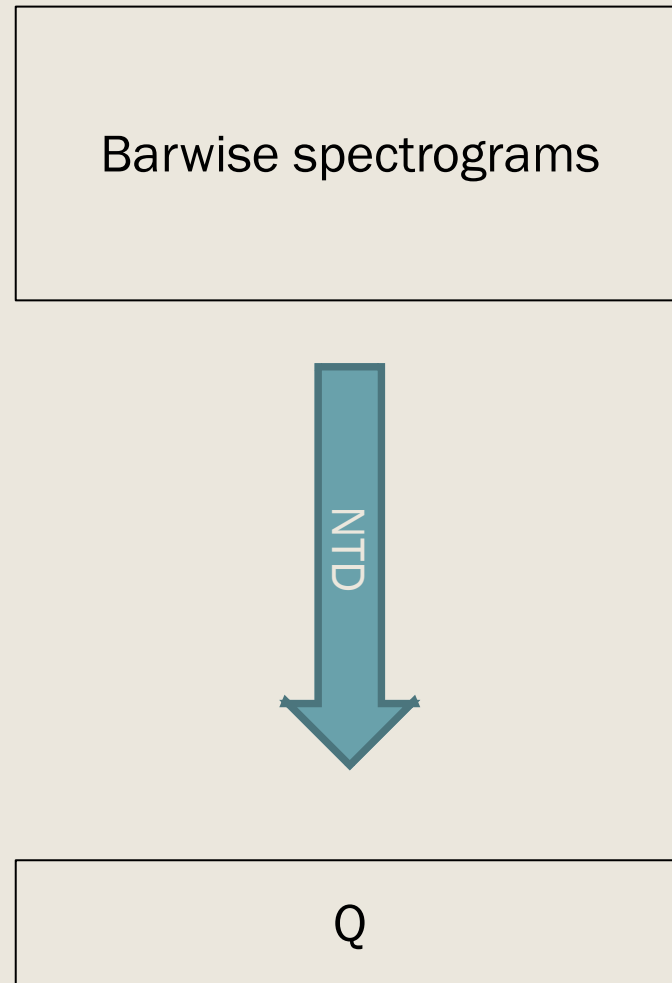
Linear and Nonlinear Barwise Compression Schemes

[MCB22b]

[MCB22b] **Marmoret, A.**, Cohen, J.E., and Bimbot, F., « Barwise Compression Schemes for Audio-Based Music Structure Analysis », in: *19th Sound and Music Computing Conference, SMC 2022*, Sound and music Computing network, 2022.

Linear and Nonlinear Barwise Compression Schemes

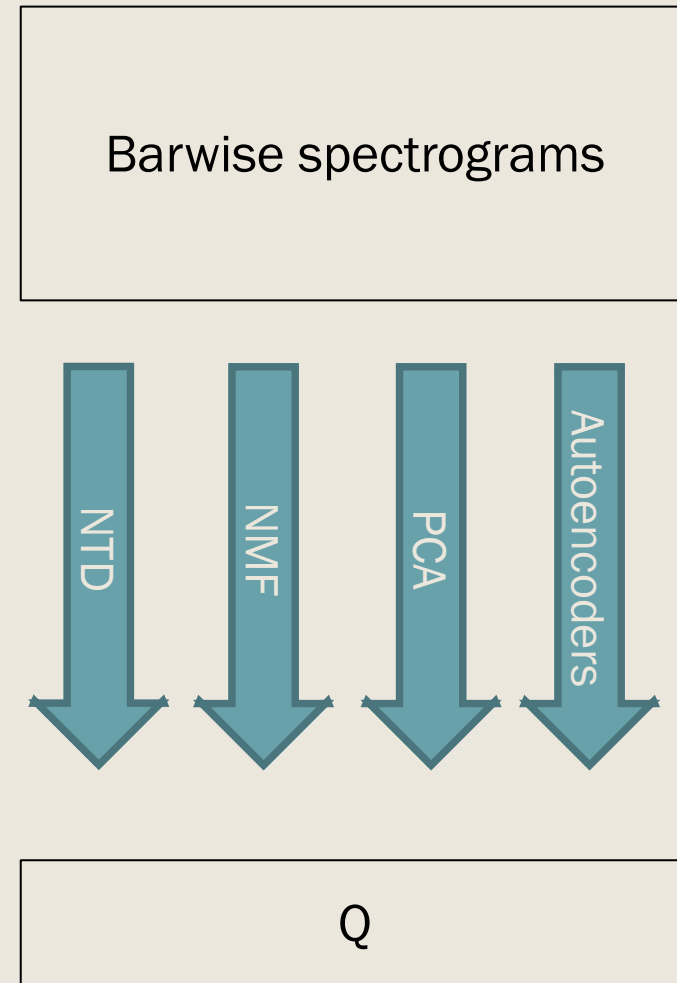
[[MCB22b](#)]



[[MCB22b](#)] **Marmoret, A.**, Cohen, J.E., and Bimbot, F., « Barwise Compression Schemes for Audio-Based Music Structure Analysis », in: *19th Sound and Music Computing Conference, SMC 2022*, Sound and music Computing network, 2022.

Linear and Nonlinear Barwise Compression Schemes

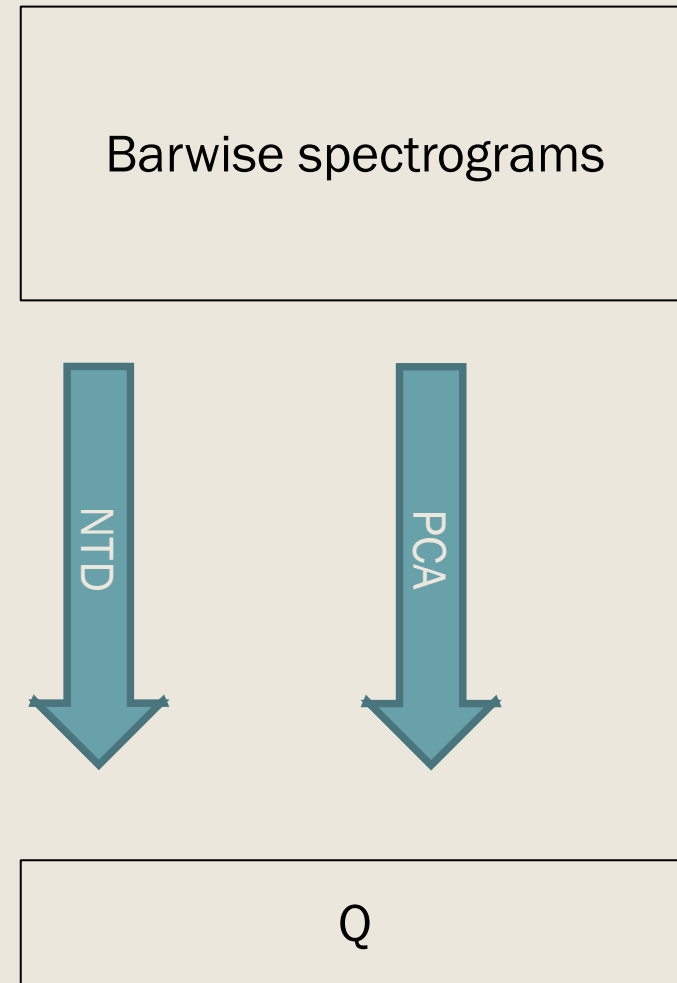
[MCB22b]



[MCB22b] Marmoret, A., Cohen, J.E., and Bimbot, F., « Barwise Compression Schemes for Audio-Based Music Structure Analysis », in: *19th Sound and Music Computing Conference, SMC 2022*, Sound and music Computing network, 2022.

Linear and Nonlinear Barwise Compression Schemes

[[MCB22b](#)]



[[MCB22b](#)] **Marmoret, A.**, Cohen, J.E., and Bimbot, F., « Barwise Compression Schemes for Audio-Based Music Structure Analysis », in: *19th Sound and Music Computing Conference, SMC 2022*, Sound and music Computing network, 2022.

PCA

- Computes directions of maximal variance ("principal components")
 - *Not interpretable!*
- Projection on these principal components

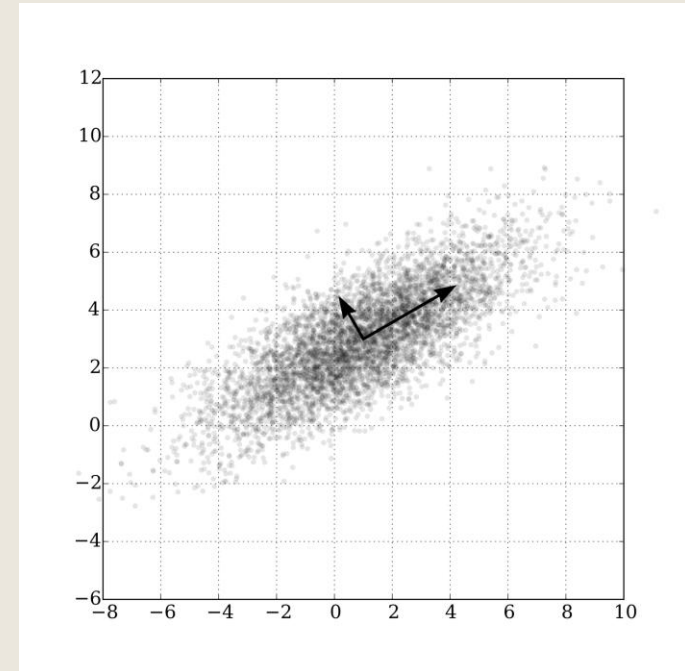
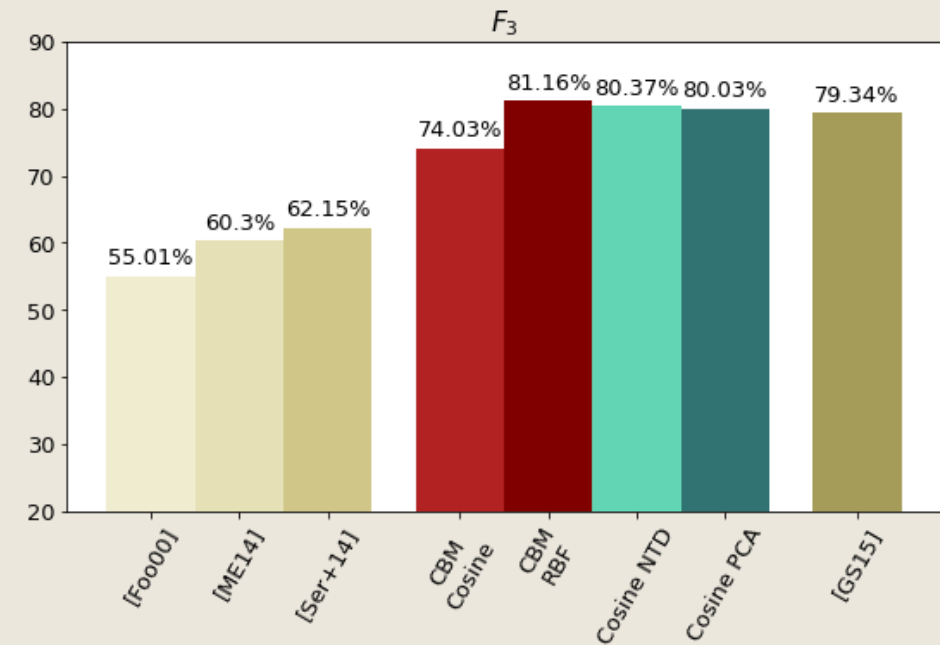
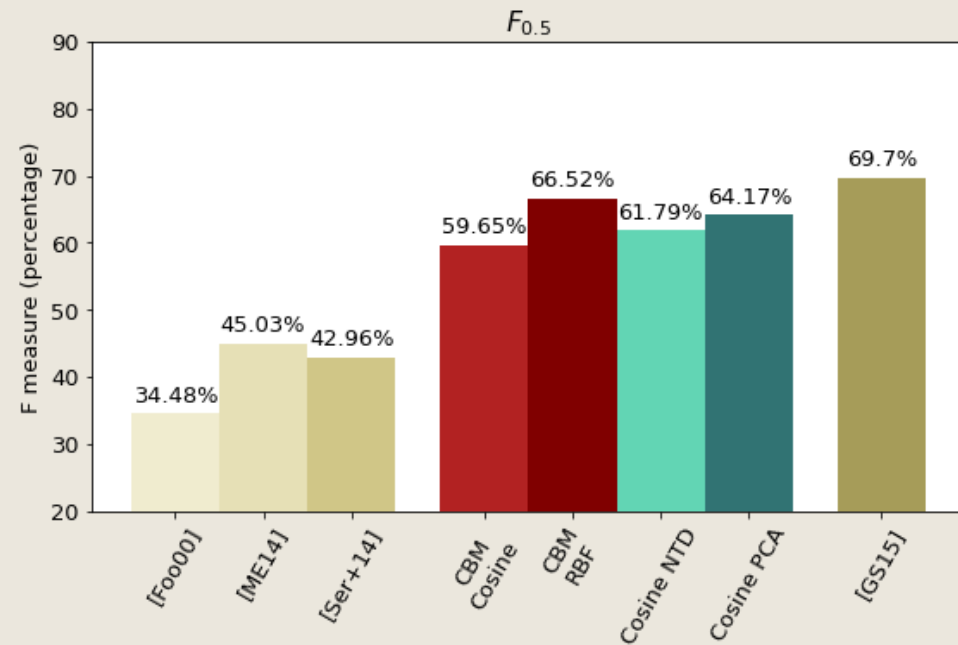
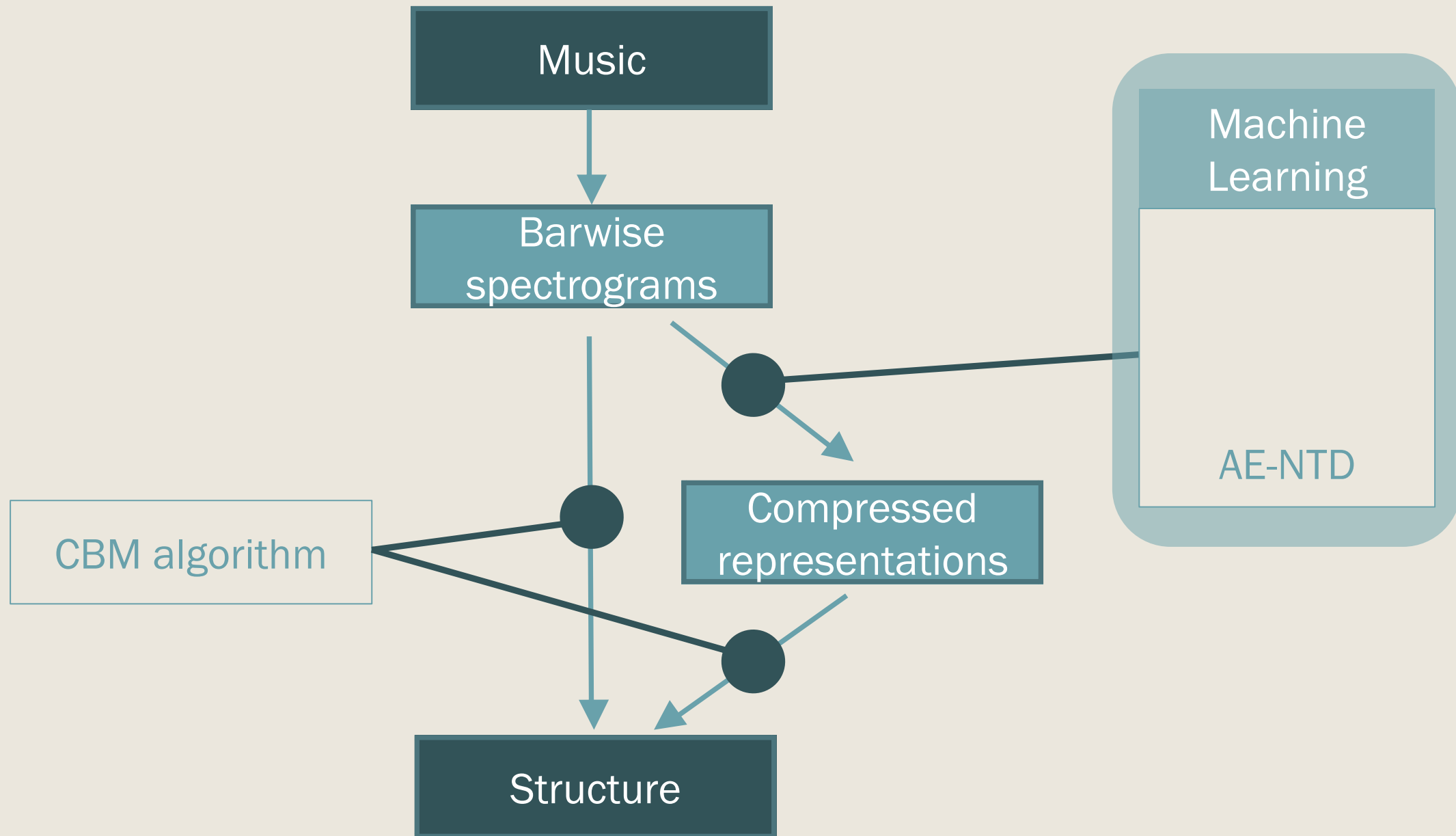


Image by Nicoguardo - Own work, CC BY 4.0,
<https://commons.wikimedia.org/w/index.php?curid=46871195>

PCA segmentation results



- Improves NTD results, at the cost of interpretability
- Does not outperform best condition for uncompressed representations
 - Compression dimension is hard to fit
 - Parameters for CBM were fitted for uncompressed matrices

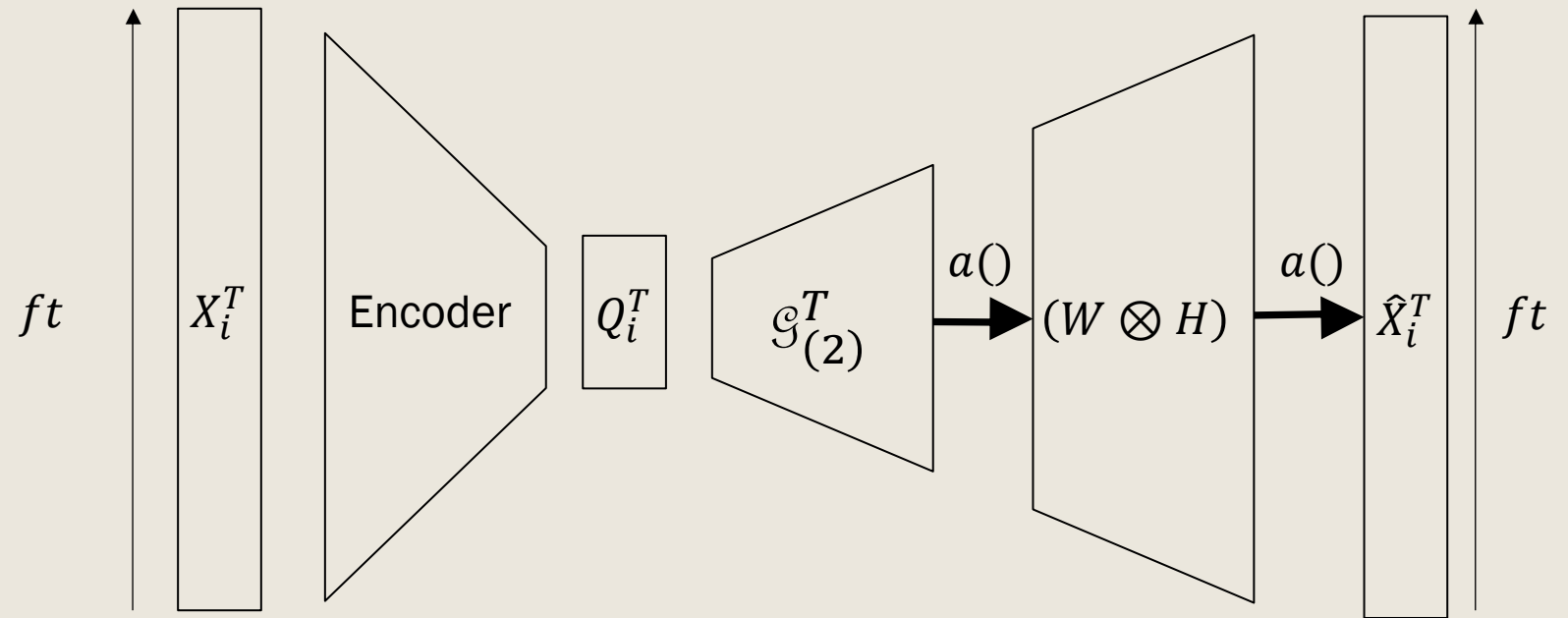


AE-NTD

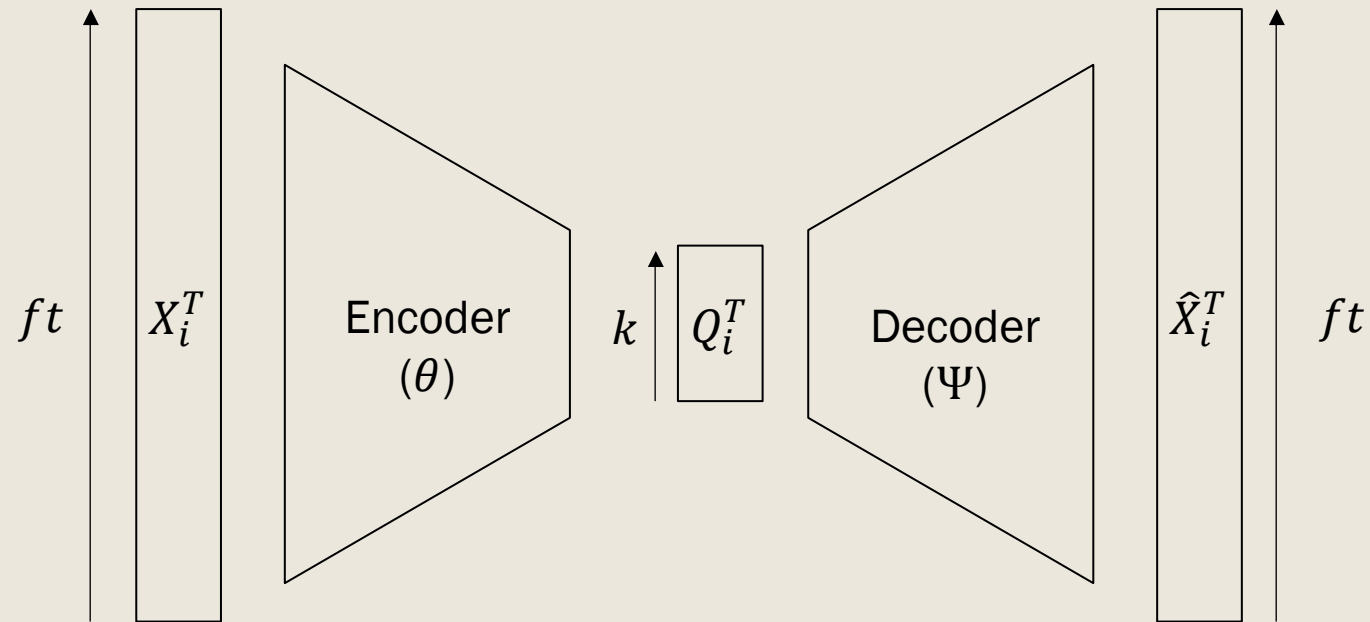
Autoencoder & NTD

AE-NTD

Autoencoder & NTD



AutoEncoders



Objective function: $\operatorname{argmin}_{\theta, \Psi} d(X_i, \hat{X}_i)$

- Encoding: compression
 - Latent representation Q_i for each bar
- Decoding: reconstruction
- Both are based on matrix products

NTD matricisation

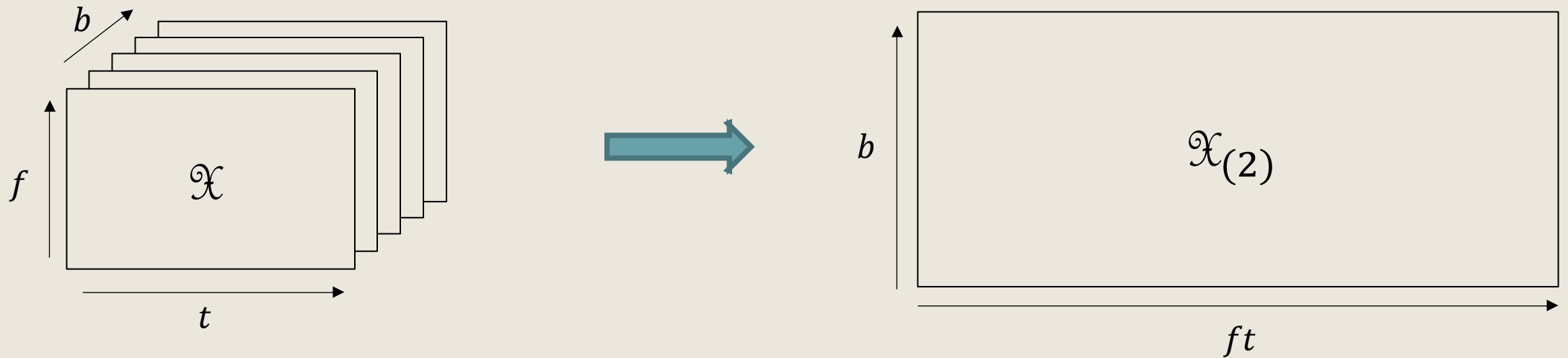
- NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q$

NTD matricisation

■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q$ $\Leftrightarrow \mathcal{X}_{(2)} \approx Q \mathcal{G}_{(2)} (W \otimes H)^T$

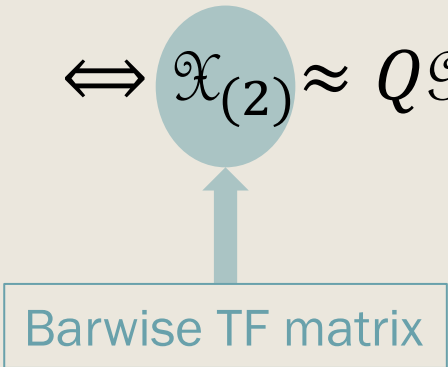
"Matricisation"

Kronecker product



NTD matricisation

■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q \iff \mathcal{X}_{(2)} \approx Q \mathcal{G}_{(2)} (W \otimes H)^T$


("unfolding" of the tensor)

NTD matricisation

■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q \iff \mathcal{X}_{(2)} \approx Q \mathcal{G}_{(2)} (W \otimes H)^T$

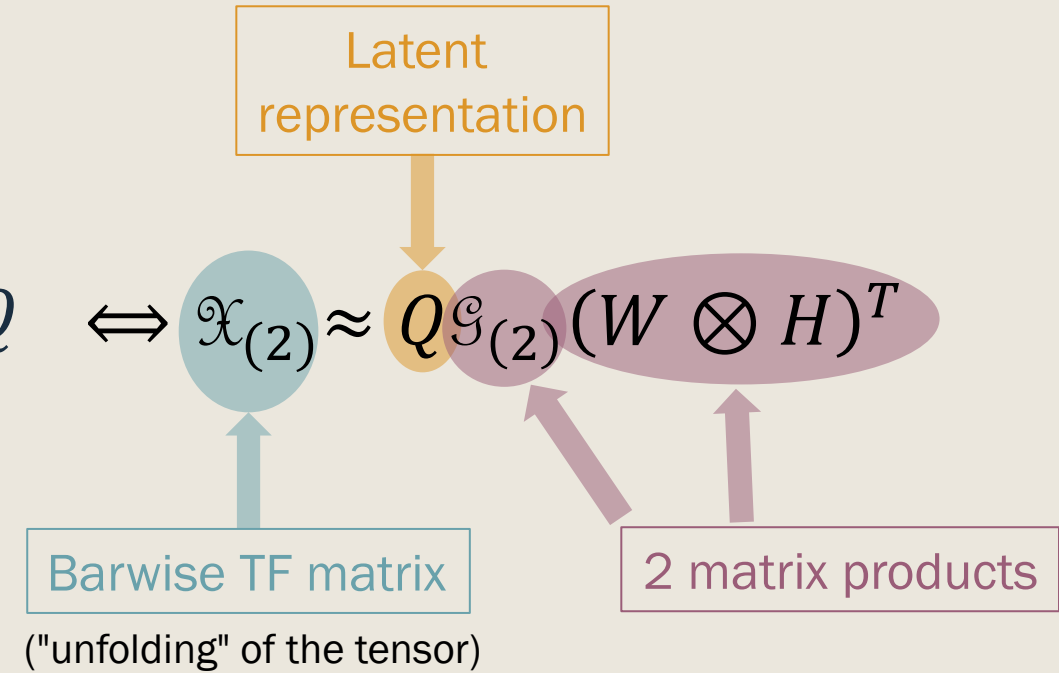
Barwise TF matrix

("unfolding" of the tensor)

Latent
representation

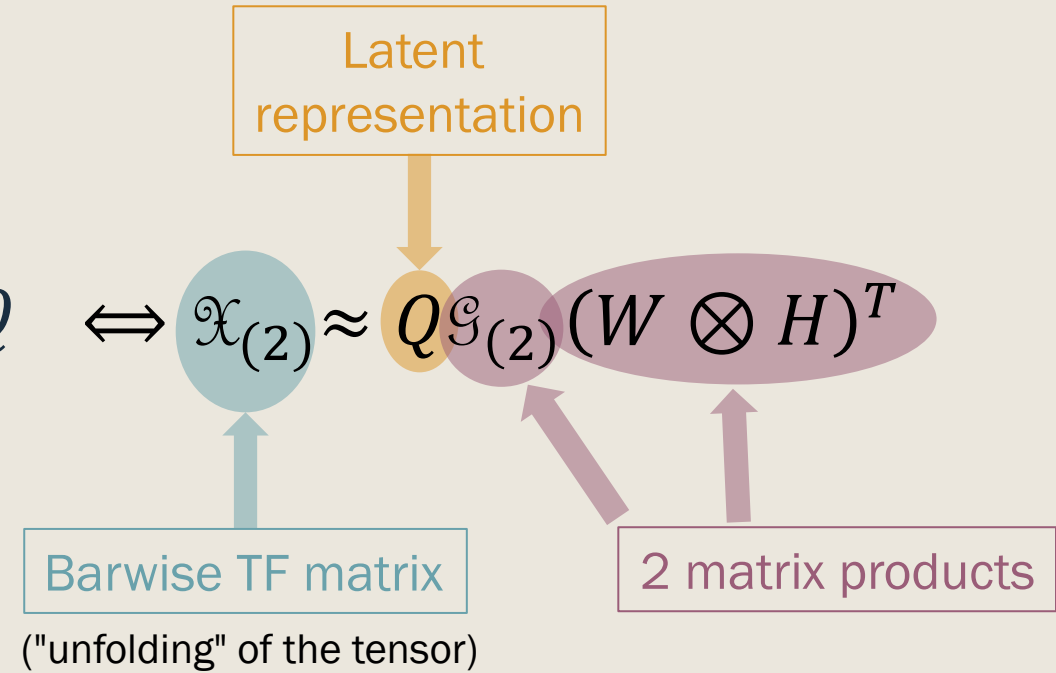
NTD matricisation

■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q$



NTD matricisation

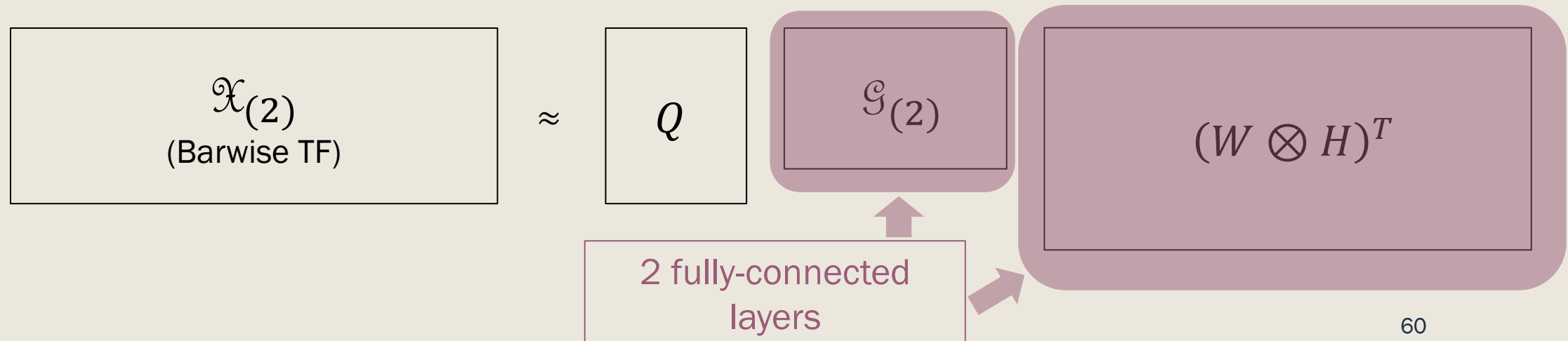
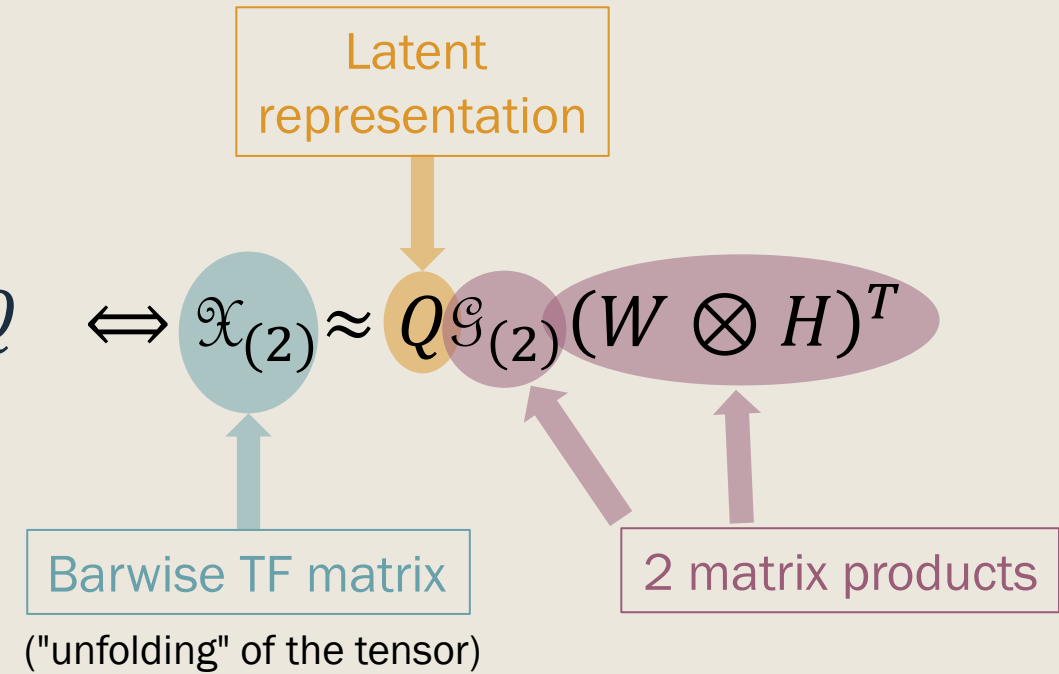
■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q \iff \mathcal{X}_{(2)} \approx Q \mathcal{G}_{(2)} (W \otimes H)^T$



$$\begin{array}{|c|} \hline \mathcal{X}_{(2)} \\ \hline \text{(Barwise TF)} \\ \hline \end{array} \approx \begin{array}{|c|} \hline Q \\ \hline \end{array} \begin{array}{|c|} \hline \mathcal{G}_{(2)} \\ \hline \end{array} \begin{array}{|c|} \hline (W \otimes H)^T \\ \hline \end{array}$$

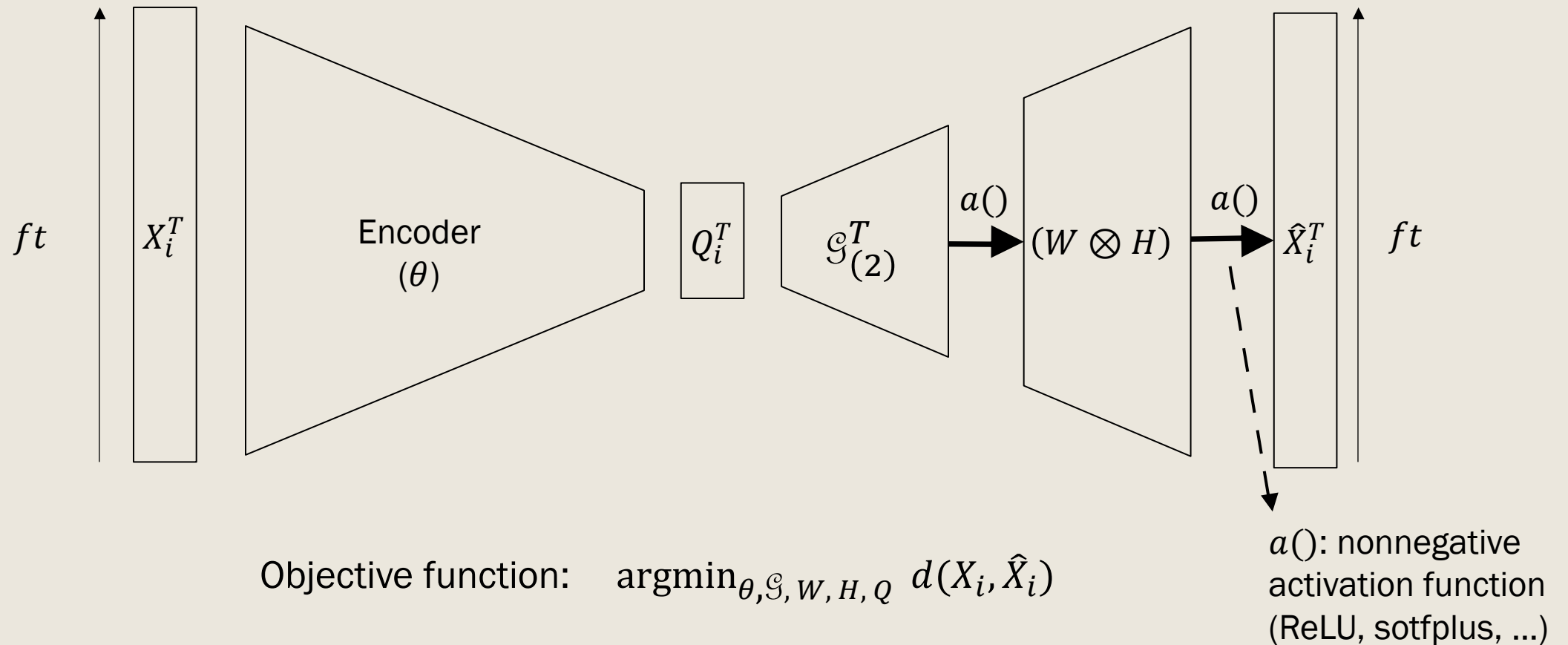
NTD matricisation

■ NTD: $\mathcal{X} \approx \mathcal{G} \times_1 W \times_2 H \times_3 Q \iff \mathcal{X}_{(2)} \approx Q \mathcal{G}_{(2)} (W \otimes H)^T$



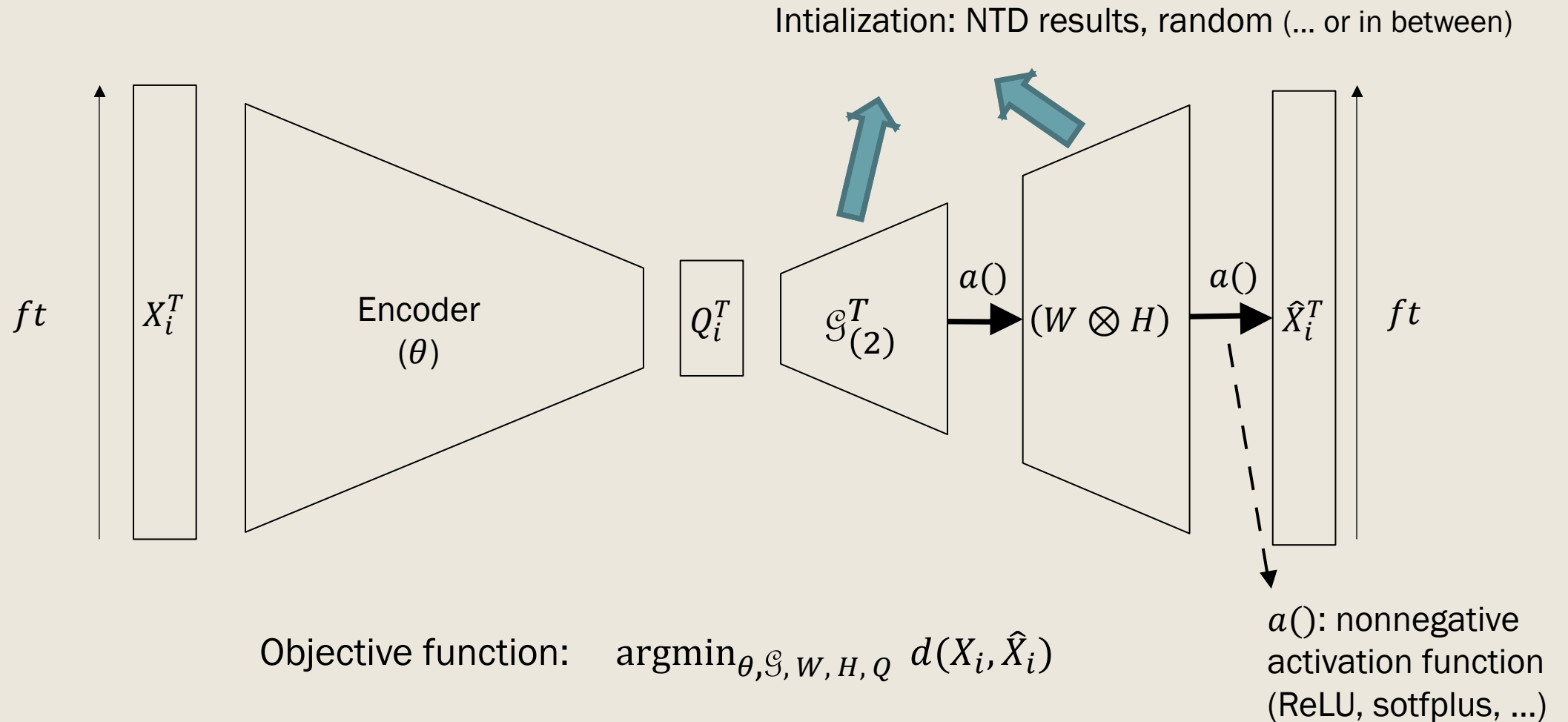
NTD-AutoEncoder:

NTD as a 2-layer decoder



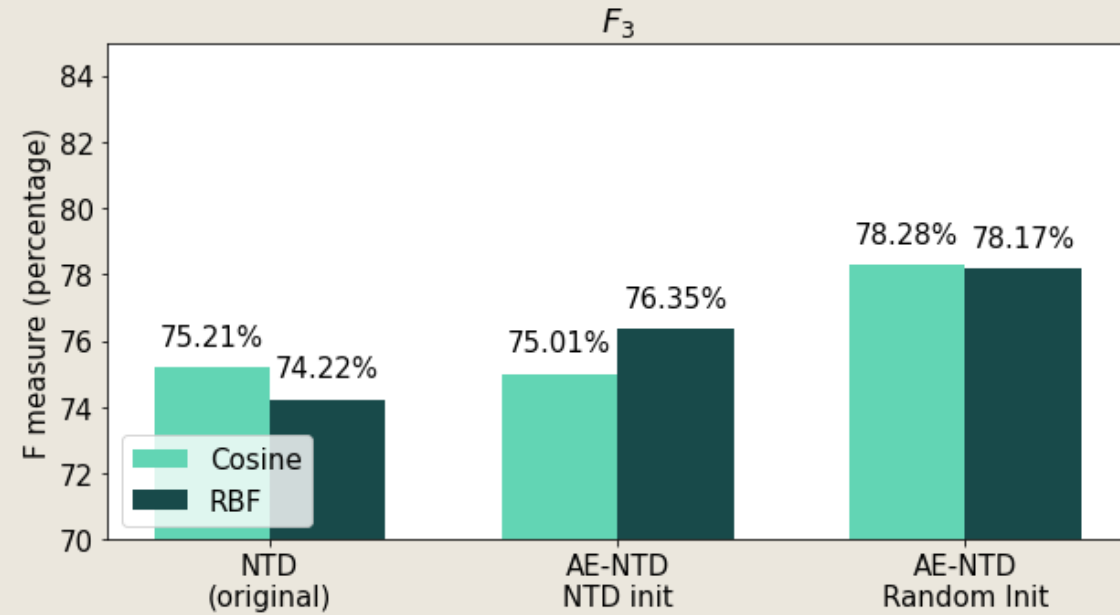
NTD-AutoEncoder:

NTD as a 2-layer decoder



Exploratory results

- Segmentation:
(RWC Pop)



- Pattern $W^{\mathcal{G}}_{[:,i]}H$:

NTD (original)	Decoder: NTD Init	Decoder: Random init
		

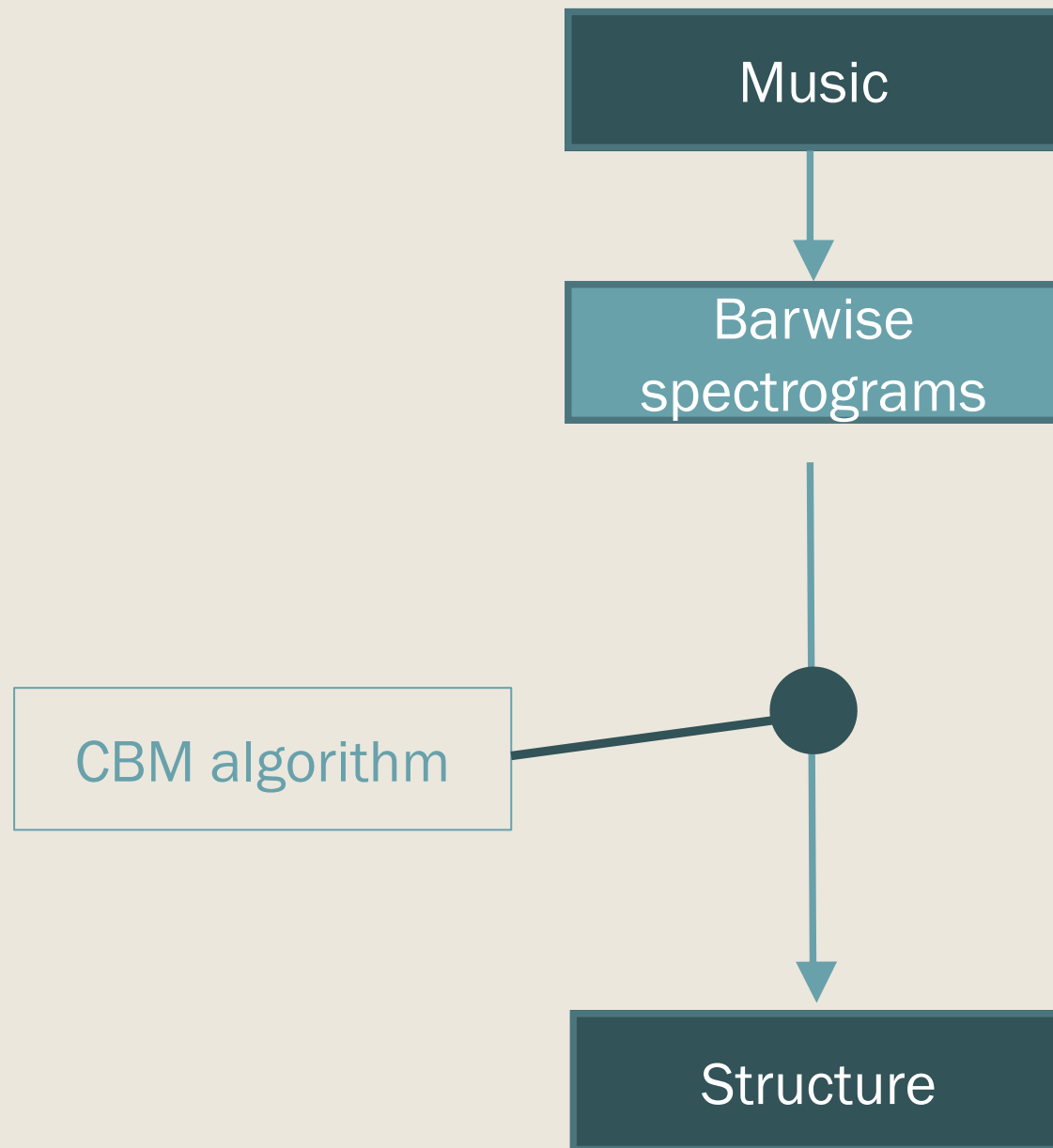
➔ Empirical trade-off between segmentation performance and interpretability

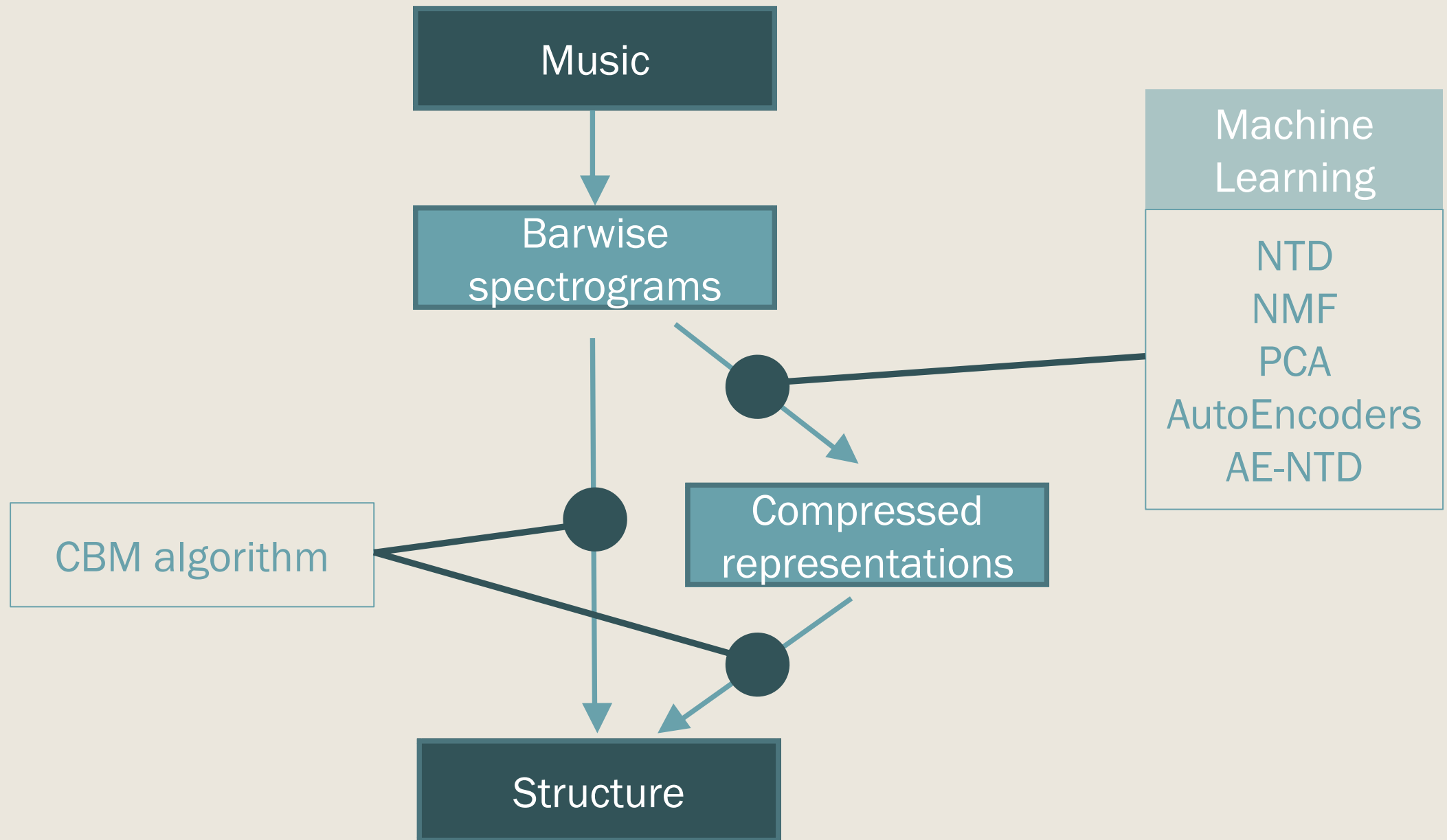
Music

Music



Barwise
spectrograms





Overview of segmentation results

- Structure is exhibited in **similarity** and **dissimilarity** notions
- Two strategies for better representations:
 - *Kernel-based similarity function*
 - *Machine Learning (compression methods)*
- Machine Learning methods: trade-off between performance and interpretability
 - *Methods present different pros and cons*
- NTD: uncovers **musical patterns**

Perspectives

Dimensioning compression

- Dimensions required prior to Machine Learning
- No obvious link between **dimensions** and **segmentation results**

Domain knowledge

- Incorporating domain knowledge to **enhance representations**
 - Constraints
 - Supervision/semi-supervision
 - Dictionary learning
 - ...

Extension of results

- Different musical contexts
- *Results are only **quantitative***
 - *Need for **musical analysis** of results*

UNSUPERVISED MACHINE LEARNING PARADIGMS FOR THE REPRESENTATION OF MUSIC SIMILARITY AND STRUCTURE



What could constitute structure?

- Lyrics?

- *Around the World - Daft Punk*



- Timbre?

- *Come Together – The Beatles*

- Melody/Harmony?

- *Lambé An Dro – Matmatah*

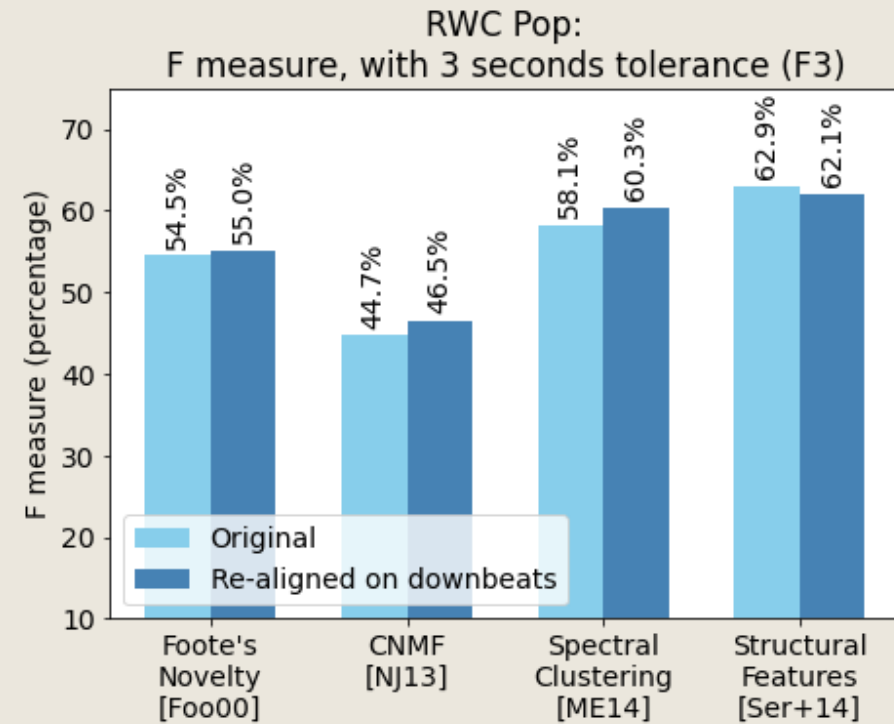
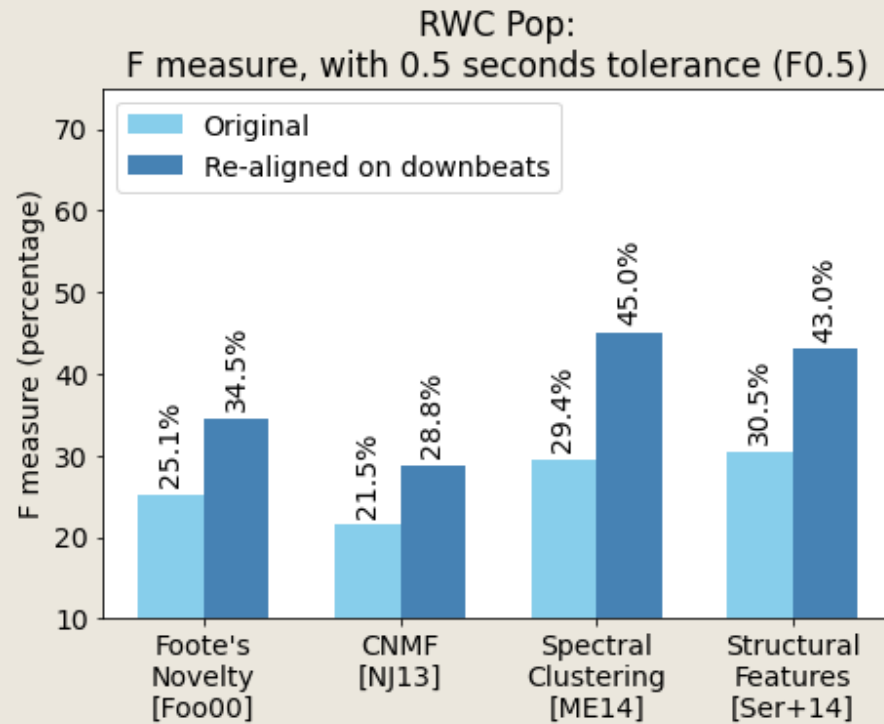


Example of pattern, with melody



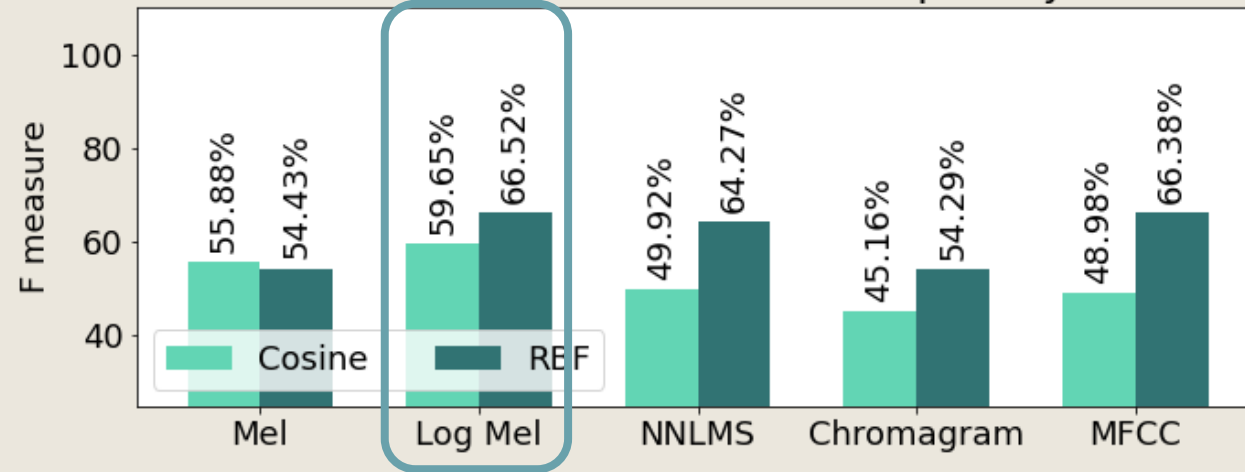
Barwise point of view

- Hypothesis 2: boundaries fall on downbeats
 - *Valid for results of state-of-the-art on RWC Pop with madmom*

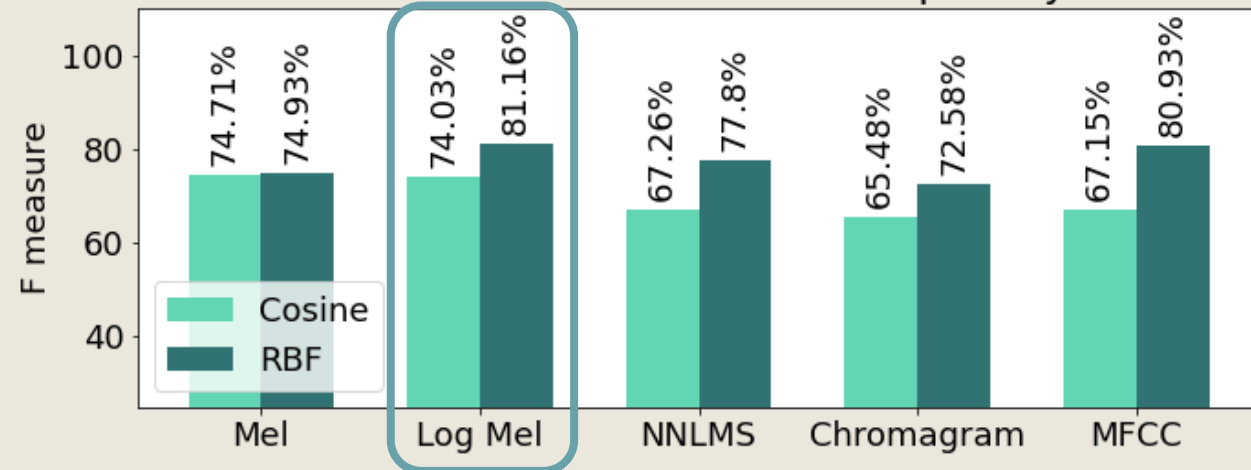


CBM Results, according to features

F0.5 on RWC Pop with different features,
7-bands kernel and modulo 8 penalty.



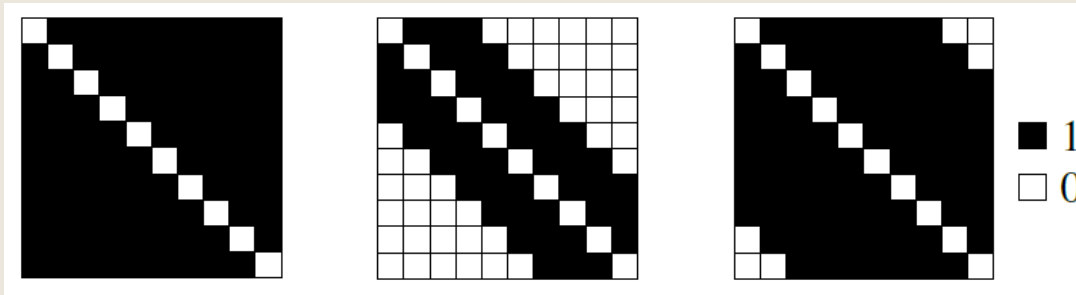
F3 on RWC Pop with different features,
7-bands kernel and modulo 8 penalty.



Convolutional "block-matching" (CBM) algorithm

■ Principle:

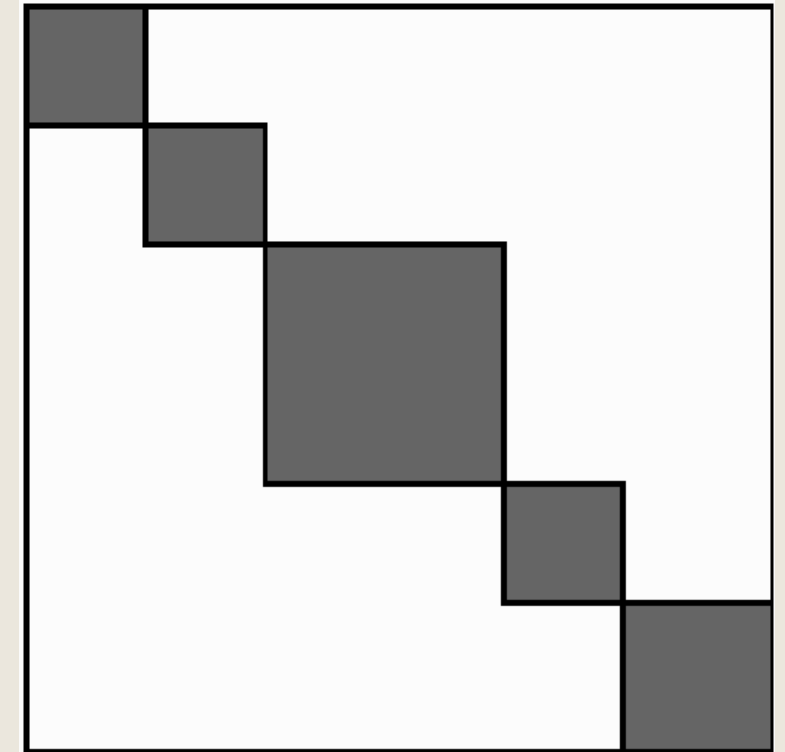
- associating a **score** for **each segment** $[b_i, b_j[$ as a convolution



$$\gamma([b_i, b_j]) = \frac{1}{j-i} \sum_{k=0}^{i-1} \sum_{l=0}^{j-1} A(X)_{i+k, i+l} K_{k,l} + p(j-i)$$

- **maximizing** their sum via **dynamic programming**

$$Z^* = \operatorname{argmax}_Z \sum_{S_i \in Z} \gamma(S_i)$$



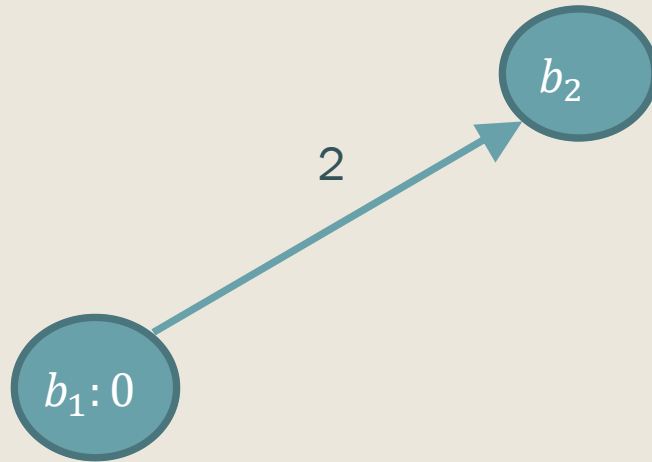
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph



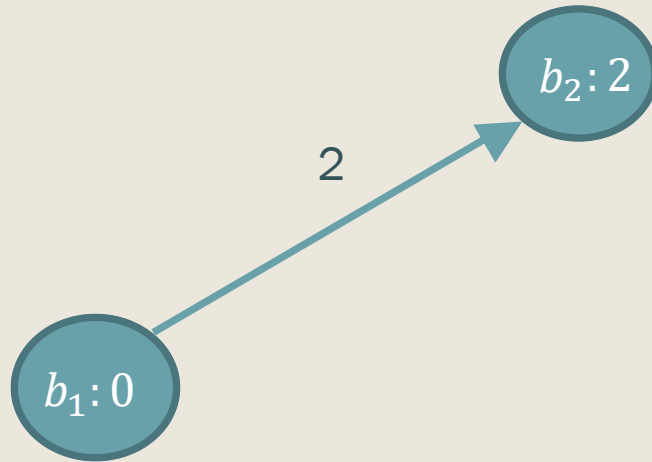
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph



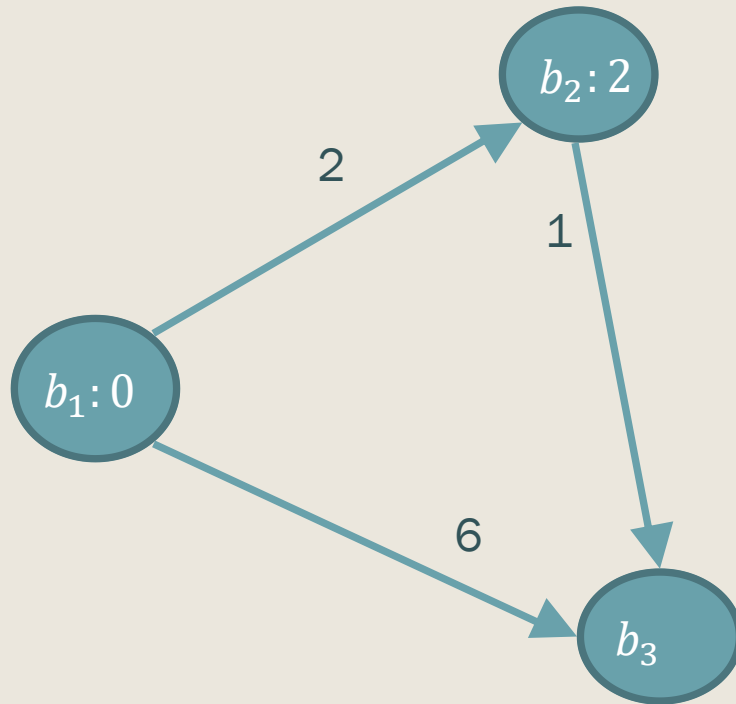
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph



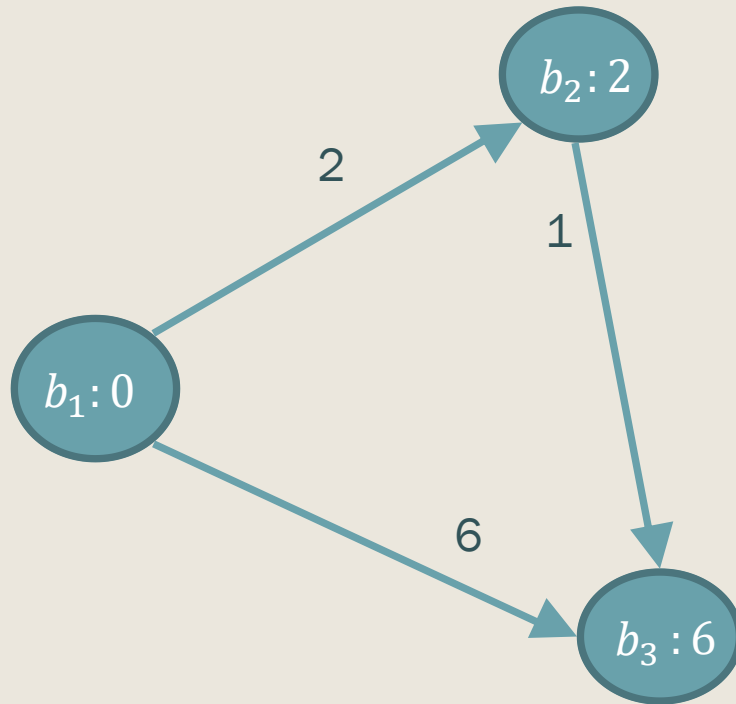
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph



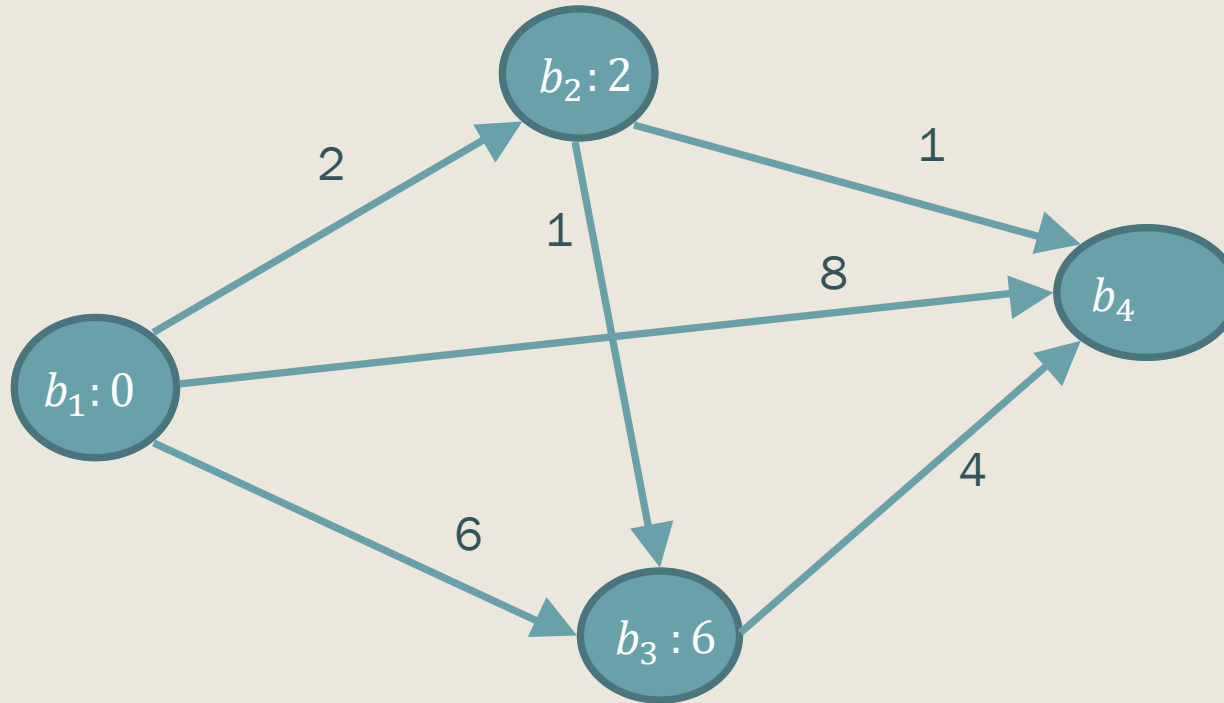
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph



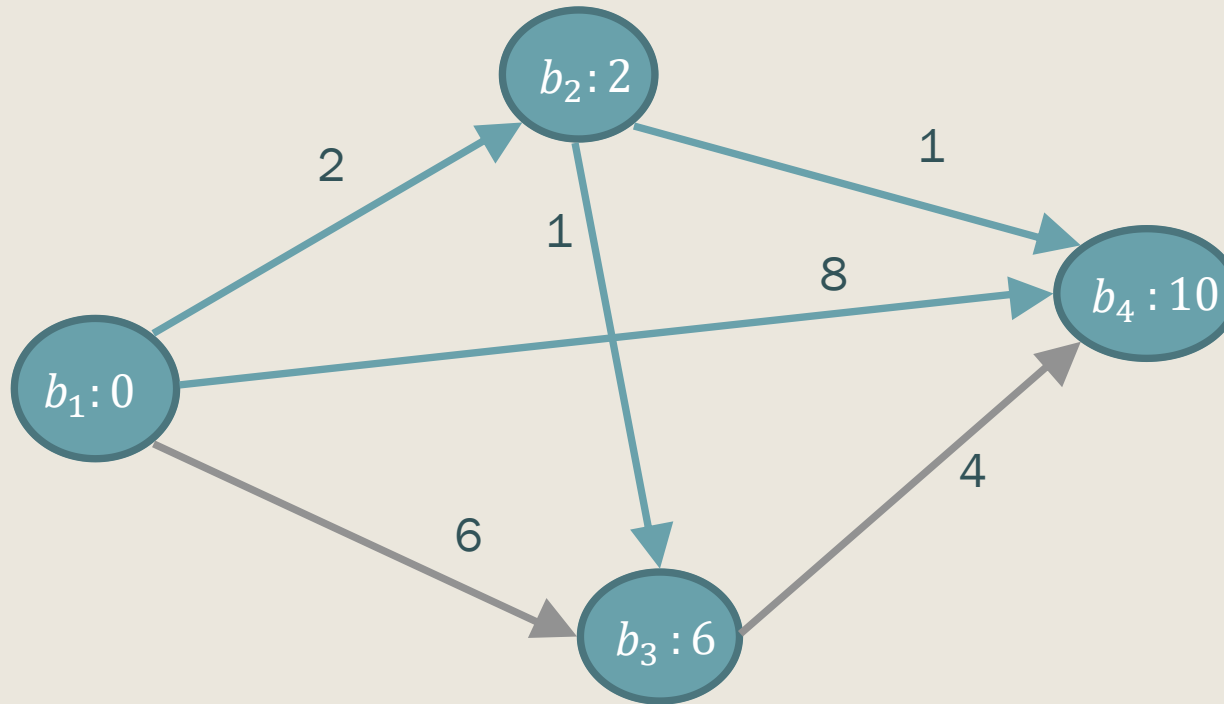
Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph

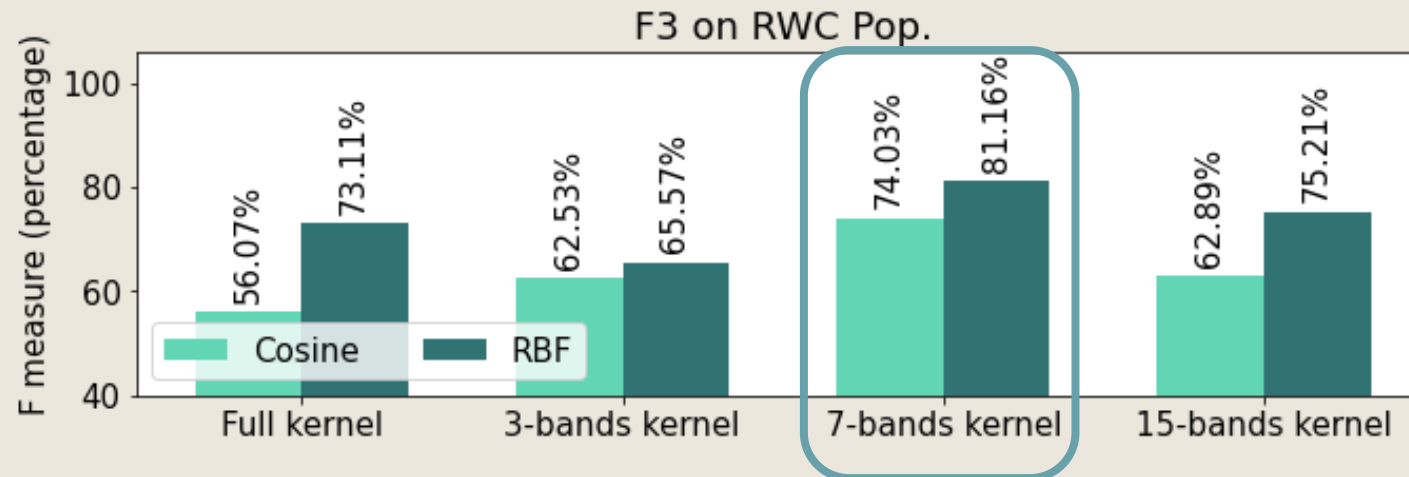
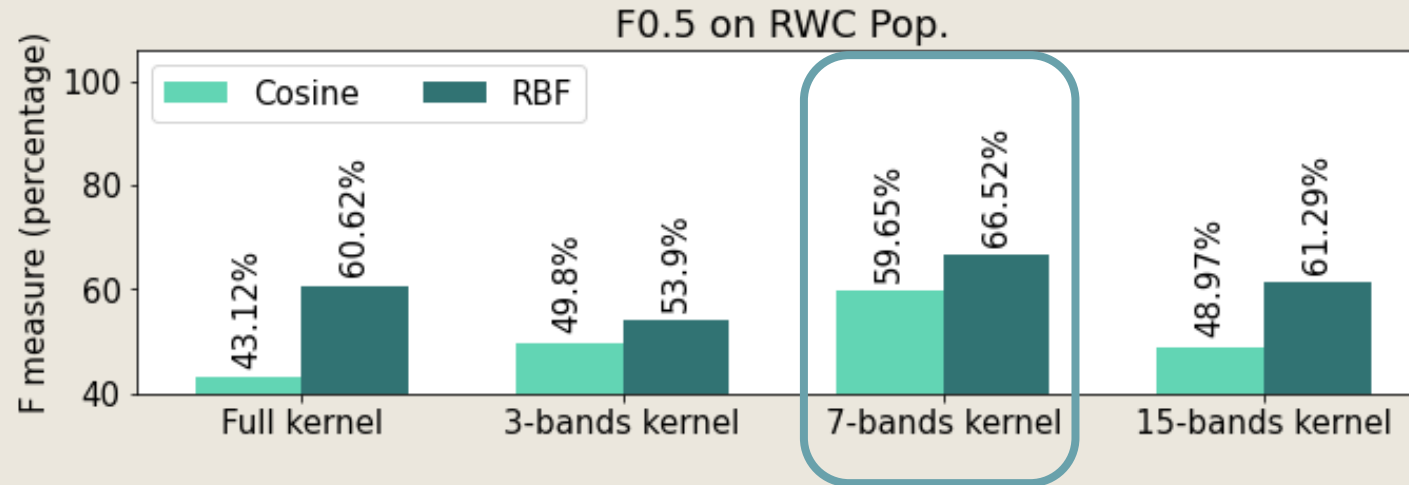


Dynamic Programming (CBM)

- Dynamic programming, can be interpreted as a longest-path retrieving in a graph

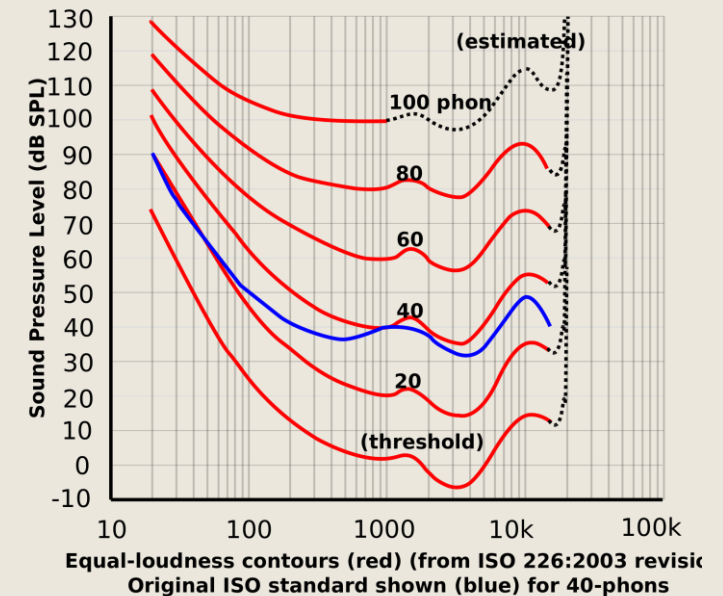
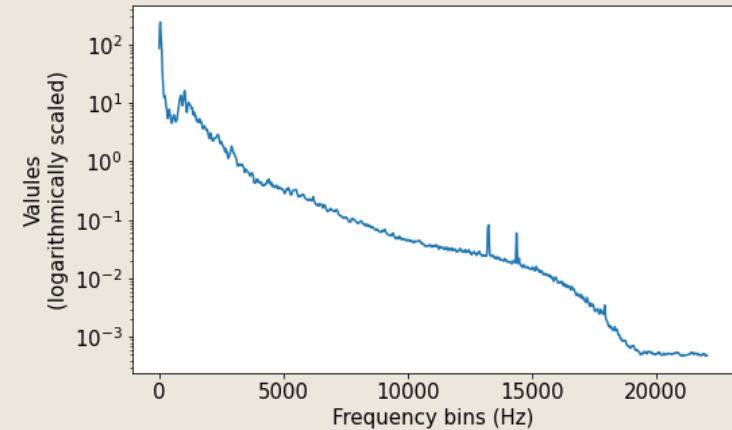


Results, according to kernels



Audio – power discrepancies

- Low-frequency coefficients are generally larger in magnitude than high-frequency coefficients
- Components of low intensity can **perceptually** be as important as components of high intensity
 - *perception of loudness in the human ear*



β -divergence

$$d_{\beta}(x|y) = \begin{cases} \frac{x}{y} - \log\left(\frac{x}{y}\right) - 1 & \beta = 0 \\ & \text{(Itakura - Saïto)} \\ x \log\left(\frac{x}{y}\right) + (y - x) & \beta = 1 \\ & \text{(Kullback - Leibler)} \\ \frac{x^{\beta} + (\beta - 1)y^{\beta} - \beta xy^{\beta-1}}{\beta(\beta - 1)} & \beta \in \mathbb{R} \setminus \{0,1\} \end{cases} \quad (\beta = 2: \text{Euclidean norm})$$

For any $\beta \in \mathbb{R}, \lambda \in \mathbb{R}, d_{\beta}(\lambda x|\lambda y) = \lambda^{\beta} d_{\beta}(x|y)$

$$\beta = 2 \Rightarrow d_2(\lambda x|\lambda y) = \lambda^2 d_2(x|y)$$

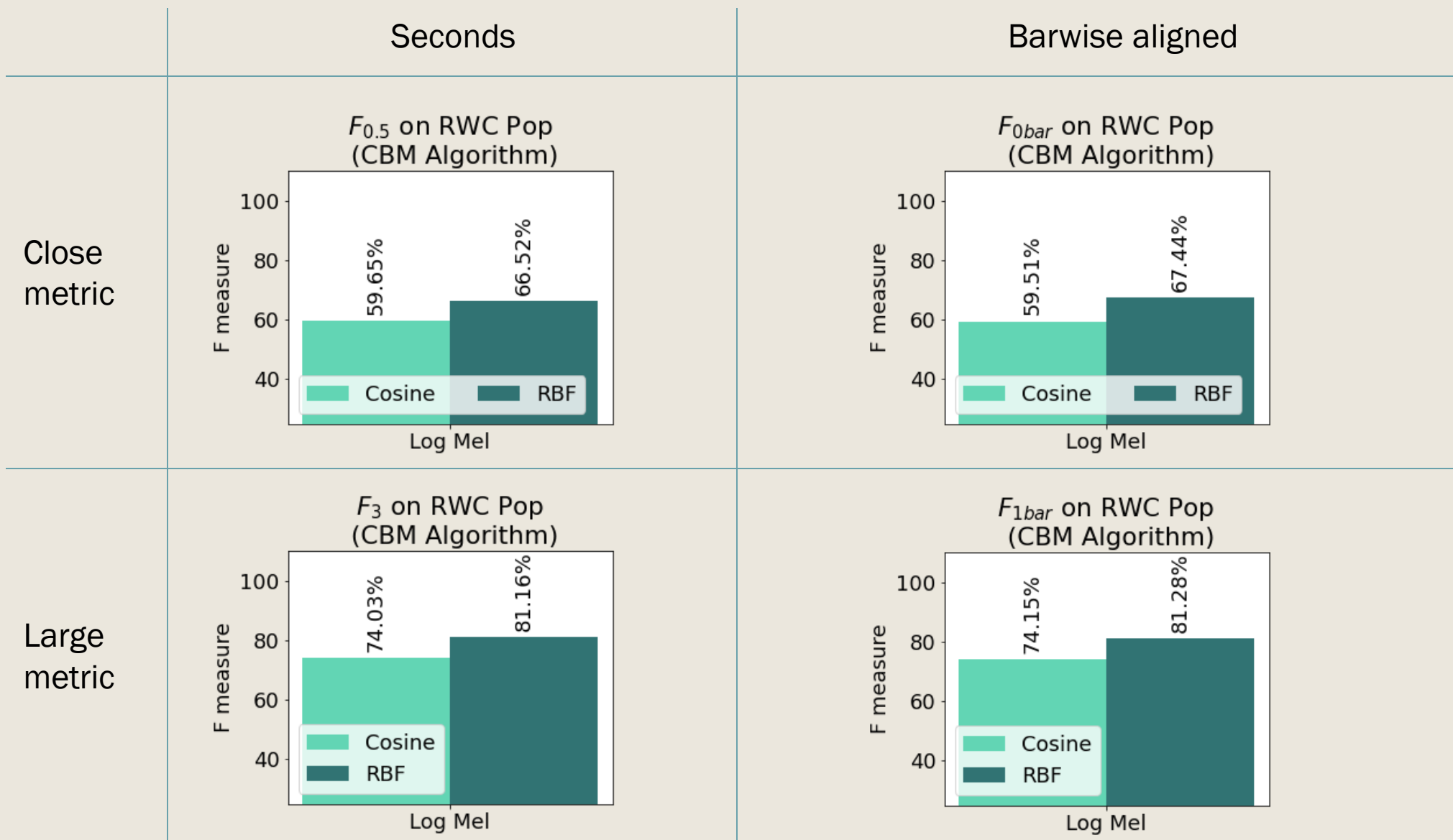
$$\beta = 1 \Rightarrow d_1(\lambda x|\lambda y) = \lambda d_1(x|y)$$

$$\beta = 0 \Rightarrow d_0(\lambda x|\lambda y) = d_0(x|y)$$

(Example: $d_{\beta}(100|10)$ vs $d_{\beta}(0.1|0.01)$)

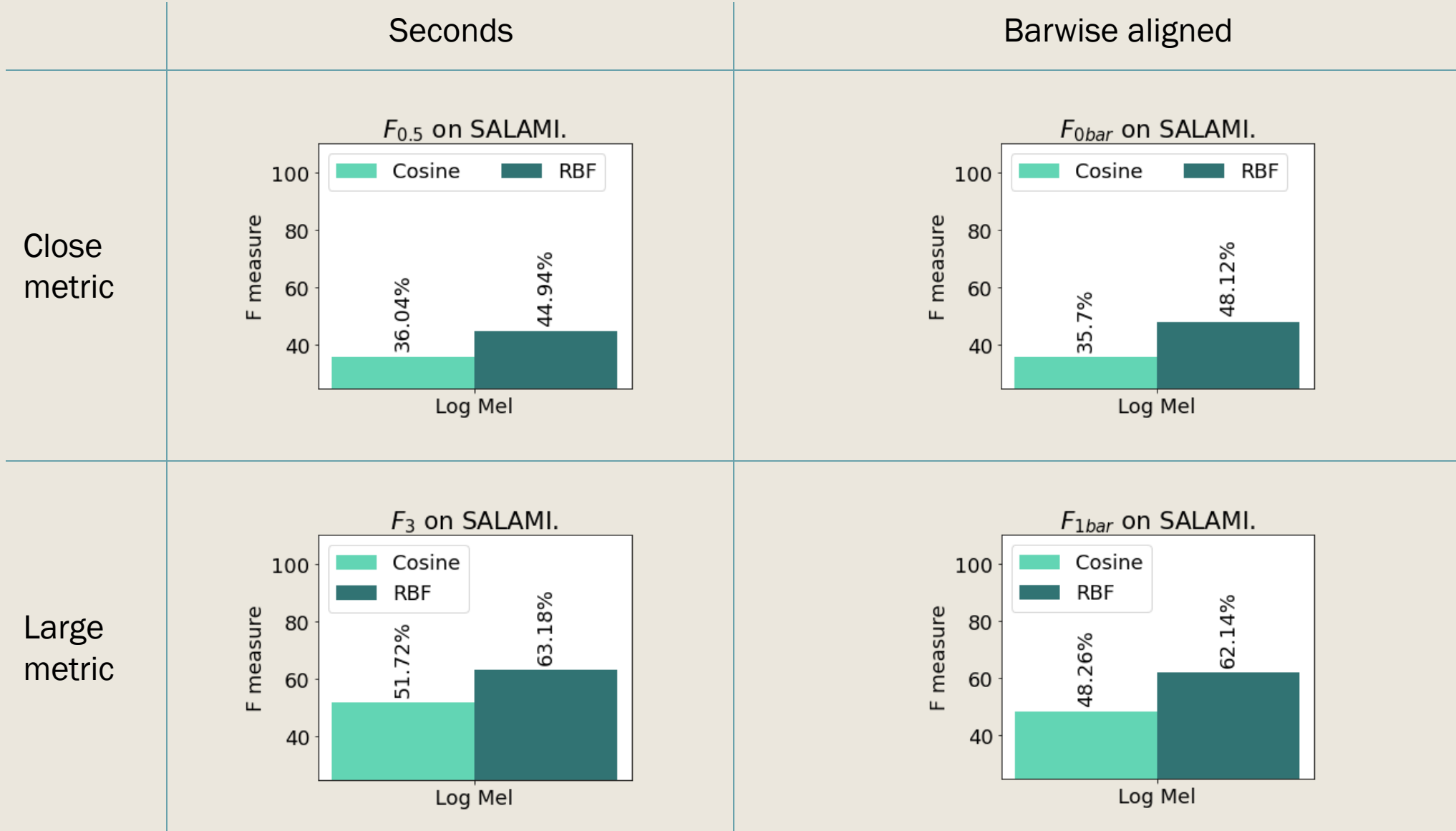
Results for CBM, scores aligned on bars

RWC POP dataset



Results for CBM, scores aligned on bars

SALAMI dataset



Why mixing AE and NTD?

Performance-Interpretability

- Combining:
 - segmentation results of AutoEncoders
 - interpretability of NTD

Objective: interpretable patterns in the decoder

Expressiveness

- Greater subspace of solutions with AE than with NTD
 - activations functions
 - encoder design
 - ...
- Ease of implementing constraints (ex: *sigmoid in the latent space*)

Objective: improve the NTD outputs, and follow state-of-the-art research

Technical tools

- Using toolboxes (PyTorch, TensorFlow) with high-performance computing

Objective: benefiting from recent developments in Neural networks computation

PCA

- Computes directions of maximal variance ("principal components")
 - *Not interpretable!*

$$\hat{X} = USV^T$$

($\hat{X}$: centered X matrix)

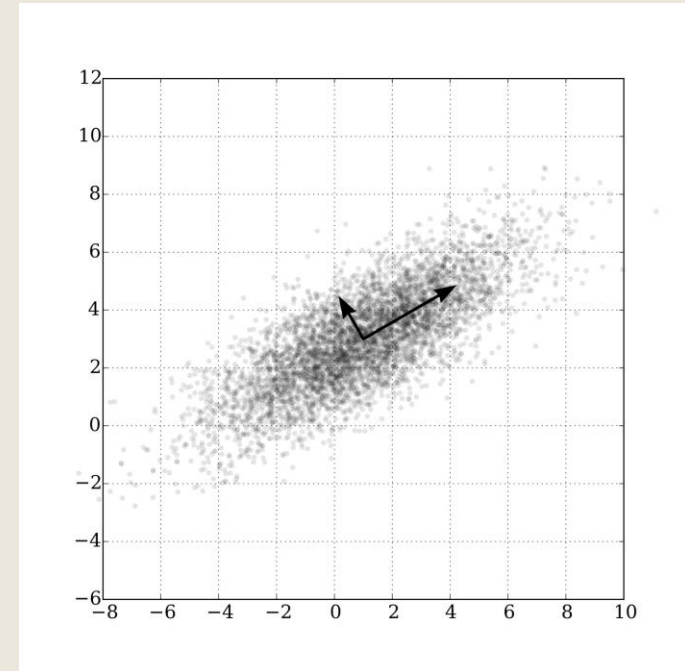
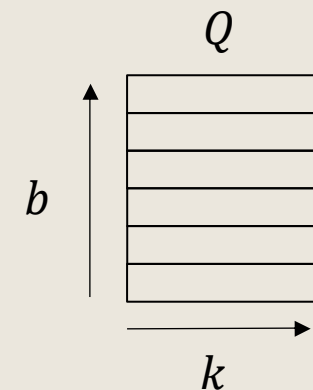
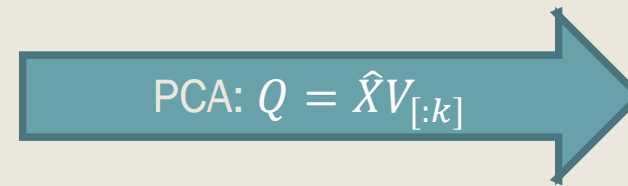
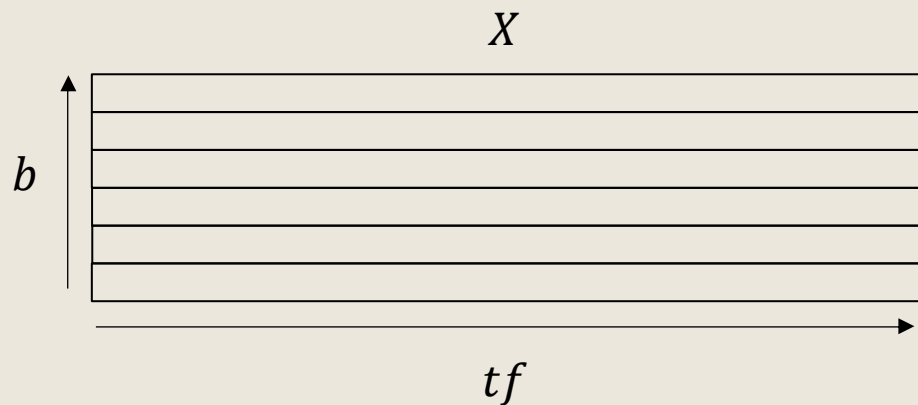
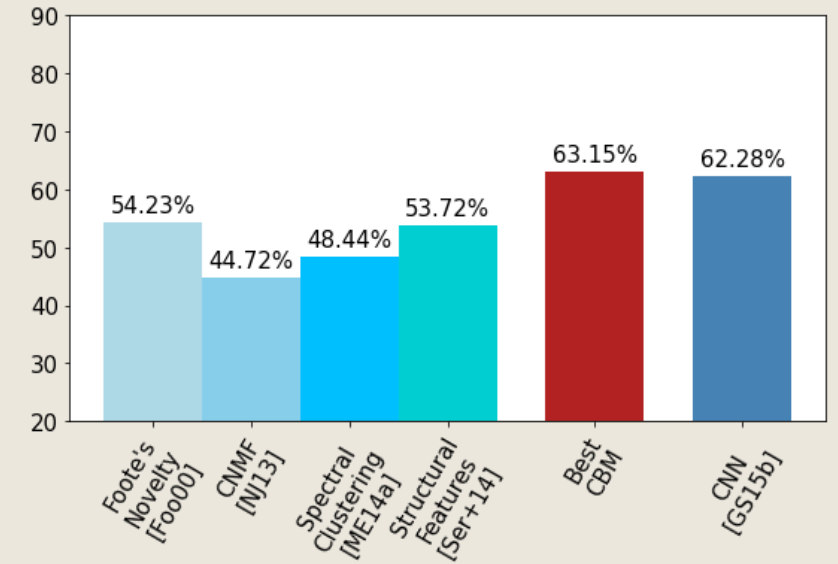
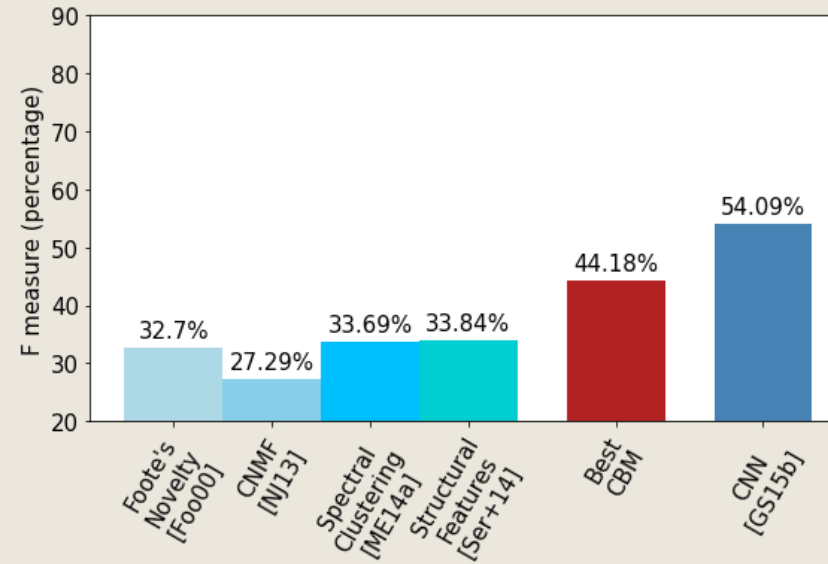


Image by Nicoguardo - Own work, CC BY 4.0,
<https://commons.wikimedia.org/w/index.php?curid=46871195>

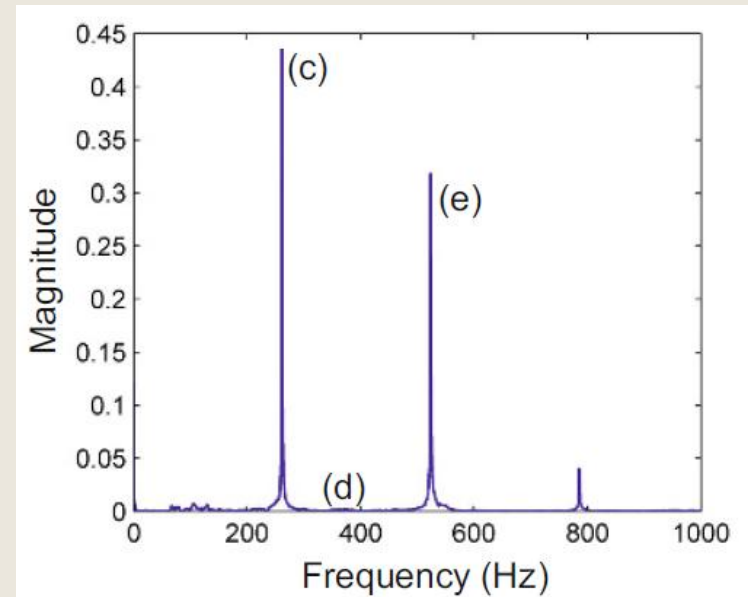
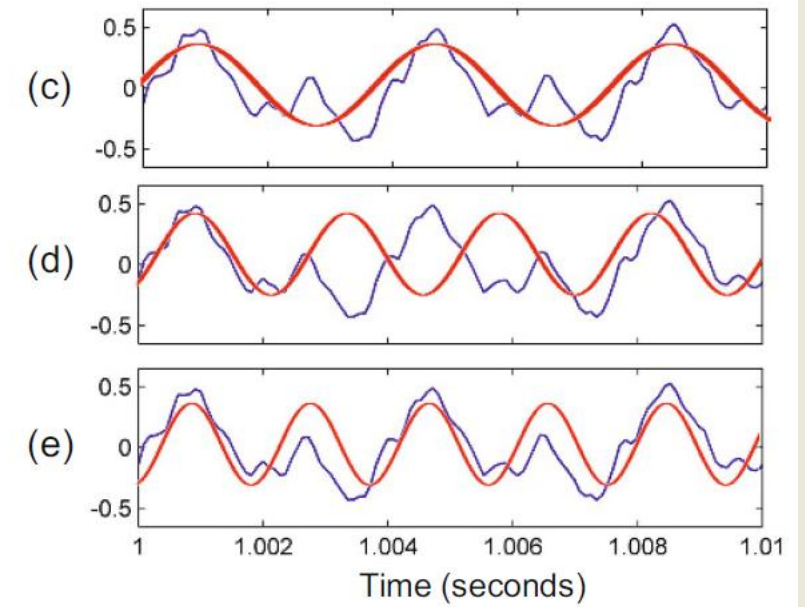
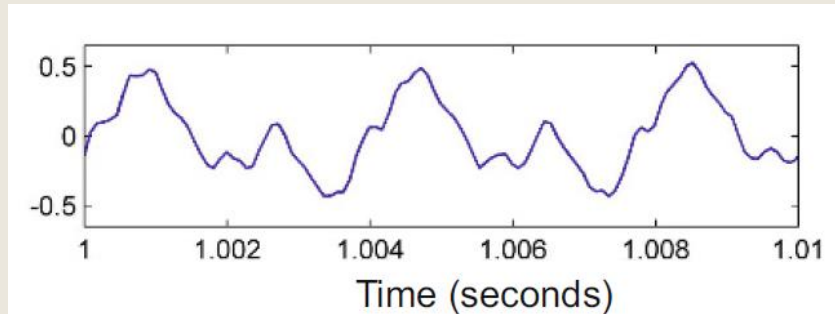
Results on SALAMI



Method	Best conditions	RWC Pop		SALAMI	
		$F_{0.5}$	F_3	$F_{0.5}$	F_3
Barwise TF	Log Mel, RBF	66.52%	81.16%	44.94%	63.18%
NTD	NNLM, KL, Cosine	61.79%	80.37%	-	-
NMF	NNLM, KL, Cosine	61.64%	79.75%	43.42%	61.84%
PCA	Log Mel, Cosine	64.17%	80.03%	43.55%	61.79%
Conv SSAE	MFCC, Euclidean, Covariance	63.14% $\pm 2.25\%$	79.50% $\pm 1.64\%$	44.04% $\pm 1.33\%$	61.91% $\pm 0.55\%$
Foote [Foo00]		34.48%	55.01%	32.70%	54.23%
CNMF [NJ13]		28.81%	46.53%	27.29%	44.72%
Spectral Clustering [ME14a]		45.01%	60.30%	33.69%	48.44%
Structural Features [Ser+14]		42.96%	62.15%	33.84%	53.72%
CNN [GS15b]		69.70%	79.34%	54.09%	62.28%

Table 5.5 – Comparison of all best-performing methods.

Fourier analysis



Kronecker product

$$A \boxtimes B = \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1J}B \\ a_{21}B & a_{22}B & \cdots & a_{2J}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{I1}B & a_{I2}B & \cdots & a_{IJ}B \end{bmatrix}, \quad (11)$$

or, by its explicit form:

$$A \boxtimes B = \begin{bmatrix} a_{11}b_{11} & a_{11}b_{12} & \cdots & a_{11}b_{1L} & \cdots & \cdots & a_{1J}b_{11} & a_{1J}b_{12} & \cdots & a_{1J}b_{1L} \\ a_{11}b_{21} & a_{11}b_{22} & \cdots & a_{11}b_{2L} & \cdots & \cdots & a_{1J}b_{21} & a_{1J}b_{22} & \cdots & a_{1J}b_{2L} \\ \vdots & \vdots & \ddots & \vdots & & & \vdots & \vdots & \ddots & \vdots \\ a_{11}b_{K1} & a_{11}b_{K2} & \cdots & a_{11}b_{KL} & \cdots & \cdots & a_{1J}b_{K1} & a_{1J}b_{K2} & \cdots & a_{1J}b_{KL} \\ \vdots & \vdots & & \vdots & \ddots & & \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots & & \ddots & \vdots & \vdots & & \vdots \\ a_{I1}b_{11} & a_{I1}b_{12} & \cdots & a_{I1}b_{1L} & \cdots & \cdots & a_{IJ}b_{11} & a_{IJ}b_{12} & \cdots & a_{IJ}b_{1L} \\ a_{I1}b_{21} & a_{I1}b_{22} & \cdots & a_{I1}b_{2L} & \cdots & \cdots & a_{IJ}b_{21} & a_{IJ}b_{22} & \cdots & a_{IJ}b_{2L} \\ \vdots & \vdots & \ddots & \vdots & & & \vdots & \vdots & \ddots & \vdots \\ a_{I1}b_{K1} & a_{I1}b_{K2} & \cdots & a_{I1}b_{KL} & \cdots & \cdots & a_{IJ}b_{K1} & a_{IJ}b_{K2} & \cdots & a_{IJ}b_{KL} \end{bmatrix} \quad (12)$$

References

- [Böc+16] Böck, S et al. Madmom: A new python audio and music signal processing library. In 2016 Proc. of the 24th ACM international conference on Multimedia.
- [FI11] Févotte, C. and Idier, J., « Algorithms for nonnegative matrix factorization with the β -divergence », *in: Neural computation* 23.9 (2011), pp. 2421–2456.
- [Foo00] Foote, J. Automatic Audio Segmentation using a Measure of Audio Novelty. In 2000 Proc. IEEE International Conference on Multimedia and Expo.
- [GG12] Gillis, N. and Glineur, F., « Accelerated multiplicative updates and hierarchical ALS algorithms for nonnegative matrix factorization », *in: Neural computation* 24.4 (2012), pp. 1085–1105.
- [GS15] Grill, T. and Schlüter, J., « Music Boundary Detection Using Neural Networks on Combined Features and Two-Level Annotations. », *in: ISMIR*, 2015, pp. 531–537
- [Got+02] Goto, M., Hashiguchi, H., Nishimura, T., and Oka, R., « RWC Music Database: Popular, Classical and Jazz Music Databases », *in: ISMIR*, vol. 2, 2002, pp. 287–288.

References

- [KC07] Kim, Y. D., & Choi, S. Nonnegative tucker decomposition. In 2007 *IEEE conference on computer vision and pattern recognition*.
- [**Mar+20**] **Marmoret, A.**, Cohen, J., Bertin, N., & Bimbot, F. Uncovering Audio Patterns in Music with Nonnegative Tucker Decomposition for Structural Segmentation. In 2020 *ISMIR*.
- [**Mar+22**] **Marmoret, A.**, et al.. Nonnegative Tucker Decomposition with Beta-divergence for Music Structure Analysis of audio signals. *GRETSI2022*.
- [**MCB22a**] **Marmoret, A.**, Cohen, J. E., & Bimbot, F. (2022). Convolutional Block-Matching Segmentation Algorithm with Application to Music Structure Analysis. *arXiv preprint arXiv:2210.15356*. (Submitted to ICASSP 2023)
- [**MCB22b**] **Marmoret, A.**, Cohen, J.E., and Bimbot, F., « Barwise Compression Schemes for Audio-Based Music Structure Analysis », *in: 19th Sound and Music Computing Conference, SMC 2022, Sound and music Computing network, 2022*.
- [ME14] McFee, B., & Ellis, D. Analyzing Song Structure with Spectral Clustering. In 2014 *ISMIR*.

References

- [Nie+20] Nieto, O *et al.*, Audio-based music structure analysis: Current trends, open challenges, and applications. 2020 *Transactions of the International Society for Music Information Retrieval*.
- [Ser+14] Serra, J., Müller, M., Grosche, P., and Arcos, J.LI., « Unsupervised music structure annotation by time series structure features and segment similarity », *in: IEEE Transactions on Multimedia* 16.5 (2014), pp. 1229–1240.
- [SG18] Smith, J. B., & Goto, M.. Nonnegative tensor factorization for source separation of loops in audio. In *2018 IEEE ICASSP*.
- [Smi+11] Smith, J.B.L., Burgoyne, J.A., Fujinaga, I., De Roure, D., and Downie, J.S., « Design and creation of a large-scale database of structural annotations. », *in: ISMIR*, vol. 11, Miami, FL, 2011, pp. 555–560.
- [S03] Smaragdis, B., and Brown, J., “Non-negative matrix factorization for polyphonic music transcription,” in 2003 Proc. IEEE Workshop Applicat. Signal Process. Audio Acoust. (WASPAA)
- [**Wu+22**] Wu, H., **Marmoret, A.**, & Cohen, J. E. Semi-Supervised Convolutional NMF for Automatic Music Transcription. 2022, *arXiv preprint arXiv:2202.04989*.